

포스코그룹

WX Agent 입문과정



교육커리큘럼

모듈	주제	학습내용	시간
M1. AI Agent 개발 방법론 이해 (1.5hr)	1. AI 개념 및 기술 이해	▪ AI 개념, AI Agent 이해, AI 주요 기술 이해	0.5
	2. 웹 이해	▪ 프론트엔드와 백엔드, HTML/CSS/JS, 서버/DB ▪ API/JSON 이해	1
M2. 바이브코딩 활용 앱/웹 개발 (8.5hr)	3. 바이브코딩 환경 구축	▪ 바이브코딩 및 Cursor AI 개발 환경 이해 ▪ Cursor AI 활용 실습	1
	4. 바이브코딩 기반 앱 개발	▪ 프롬프트 작성 방법, 웹과 앱의 차이 ▪ PDF 분할/병합(PDF 자료 업로드) ▪ 이미지 다운로드, 사이즈 색상 및 크기 조정 ▪ PDF를 PPT로 변환	2
	5. 바이브코딩 기반 웹 개발	▪ API Key 활용, 웹페이지 개발(PDF번역 웹페이지) * IP주소, 포트이해 ▪ DB활용, 웹페이지 개발(물품공유 웹페이지) ▪ DB 및 파일 활용, 예측 프로그램 개발 ▪ 번외. 구글 AI Studio에서의 웹앱 개발	3.5
	6. 바이브코딩 스킬 활용	▪ 컨텍스트 윈도우/토큰/Skill 이해 ▪ Skill을 활용해서 아이디어 구체화-프롬프트 제작-웹 개발 실습	2
M3. 바이브코딩 활용 RAG 구현 (3.5hr)	7. RAG 감잡기	▪ RAG 개념(오픈북 시험)과 동작 구조, 컨텍스트 윈도우, 의미 검색(임베딩) ▪ 실습 : 문서 유무에 따른 답변 비교, 의미 검색 체험	1
	8. RAG 구현	▪ Cursor AI 활용 RAG 구현 : ① General ② Local LLM·임베딩 모델 활용 ③ 계약서 검토	2.5
M4. 종합실습(2.5hr)	9. 막혔을 때 빠져나오는 법	▪ 에러 대응 3단계(STOP·COPY·TELL), 실습 : 버그가 심어진 앱 고치기	1
	10. AI Agent 맛보기	▪ 에이전트 4단계와 MCP, 실습 : 출장비 정산 검토 에이전트 만들기	1.5

M1. AI Agent 개발 방법론 이해

1. AI 개념 및 기술 이해

Artificial Intelligence(AI, 인공지능)의 과거와 현재

1950s~1960s



1950 | 튜링테스트 |
기계도 생각할 수 있는가?

1950

1956



1956 | 최초 챗봇 |
ELIZA 개발

1966

1970s~1990s



1980 | 전문가 시스템 |
지식베이스/추론 엔진

1974

1980

1986



1987 | AI 칩 체계 |
2차 칩 체계

1987

2000s~2010s

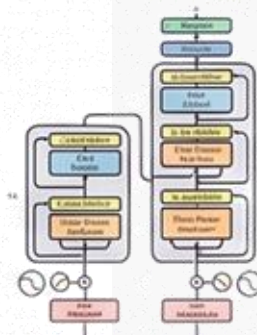


2012 | 알렉스넷 우승 |
이미지 경연 대회

2000

2012

2016



2017 | 트랜스포머 |
생성형 AI 기반

2017

2020s~



2025 | 바이브코딩 |
느낌으로 코딩

2025~



1956 | 다트머스 회의 |
「인공지능」 용어 탄생



1974 | AI 칩 체계 |
1차 칩 체계



1986 | 역전파 알고리즘 |
신경망 부활



2000 | 머신러닝 시대 |
인터넷 발달, 빅데이터 및 검색 적용



2016 | 알파고 vs 이세돌 |
알파고 우승



2022 | ChatGPT |
생성형 AI 등장

GENERATIVE AI 에서 AGENTIC AI로..

2012 ALEXNET



PERCEPTION AI

SPEECH RECOGNITION
DEEP RECSYS
MEDICAL IMAGING

GENERATIVE AI

DIGITAL MARKETING
CONTENT CREATION

AGENTIC AI

CODING ASSISTANT
CUSTOMER SERVICE
PATIENT CARE

PHYSICAL AI

SELF-DRIVING CARS
GENERAL ROBOTICS

말하는 AI에서, "일하는 AI"로의 거대한 전환

단순한 대화형 인터페이스에서 실질적인 업무를 수행하는 에이전트(Agent)로 진화

대화형 AI (Conversational AI)



- 수동적 정보 제공 및 검색
- 단순 질의응답(Q&A) 및 잡담
- '무엇인가요?'에 대한 답변 중심

[인터페이스(Interface) 역할]

진화 (EVOLUTION)

실행형 AI (Action-Oriented AI)



- 능동적 업무 수행 및 워크플로우 자동화
- 복합적 문제 해결 및 도구 활용(Tool Use)
- '해주세요'에 대한 결과 창출 중심

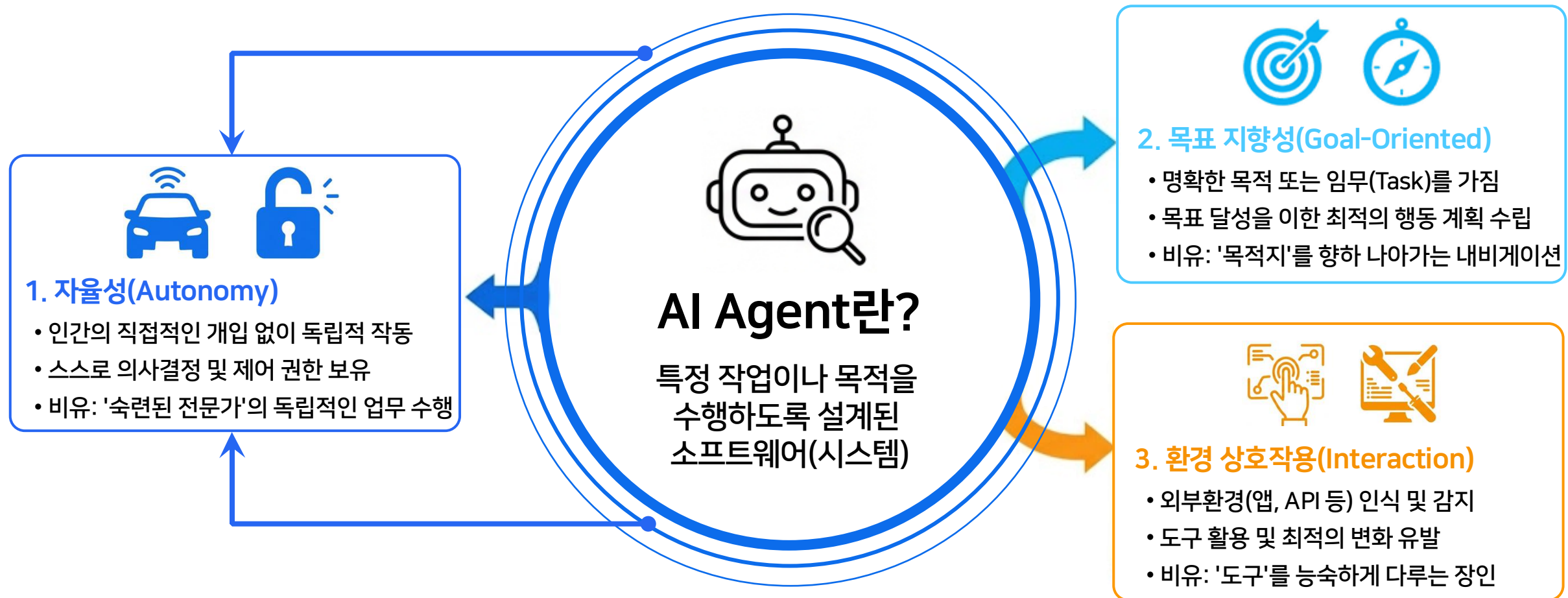
[파트너(Partner) & 에이전트(Agent)]

사람·기관·업종별 AI Agent 정의

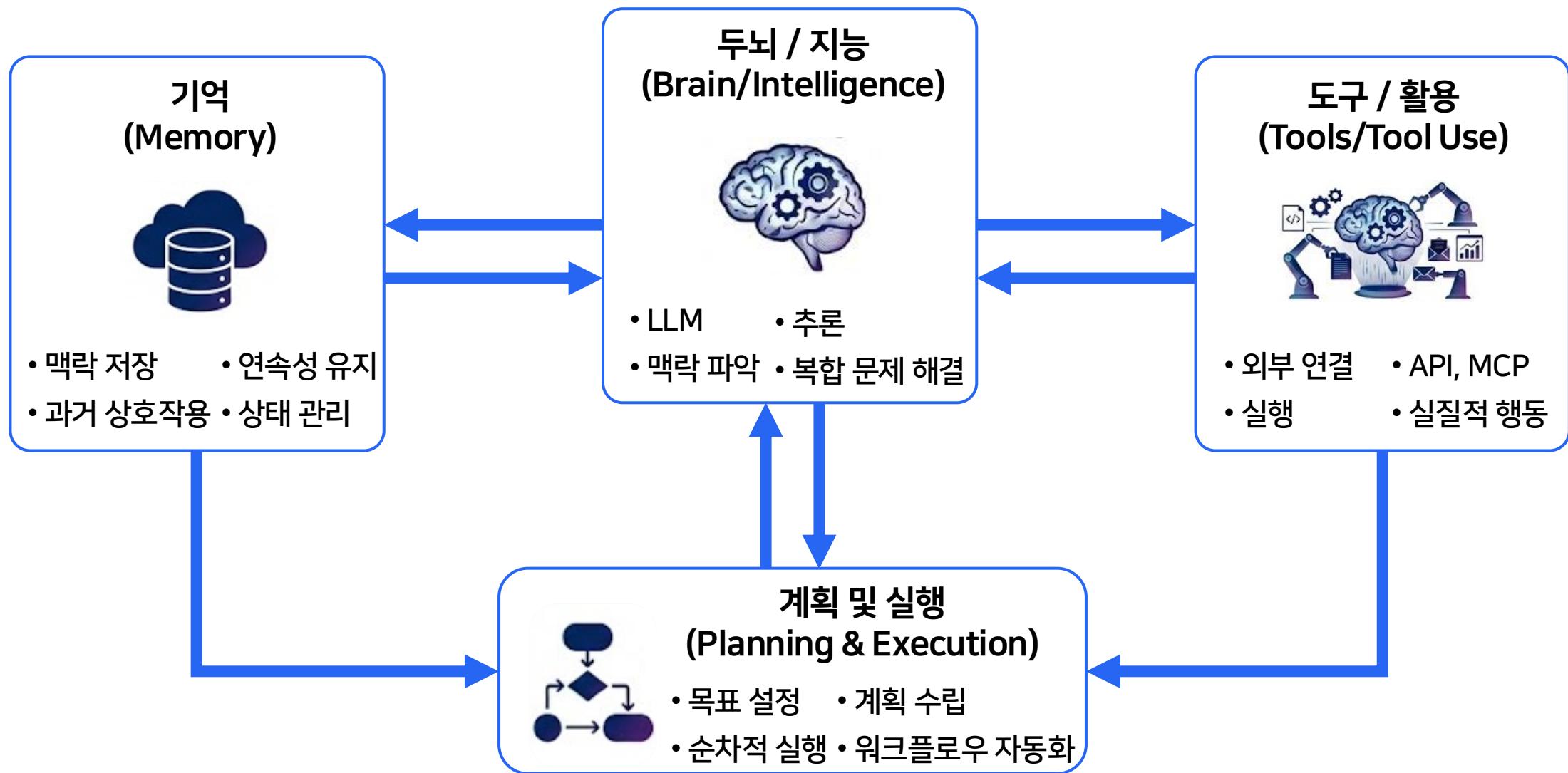
목표를 이해하고, 계획을 세우고, 도구를 사용해 행동하고, 결과를 바탕으로 반복하는 자율 AI 시스템

사람 / 학계 관점	AI 모델·개발 플랫폼	클라우드 / 빅테크	엔터프라이즈 / 산업 활용
Russell & Norvig 환경을 지각하고 목표 달성을 위해 행동하는 시스템 ※ 인공지능 분야의 대표적인 교과서 저자 - Artificial Intelligence : A Modern Approach(AIMA)	OpenAI LLM이 도구를 사용해 복잡한 작업을 단계적으로 수행하는 시스템	Google Cloud 사용자를 대신해 목표를 추구하고 작업을 완료하는 AI 소프트웨어	IBM 도구를 활용해 워크플로를 만들고 자율적으로 업무를 수행하는 프로그램
Andrew Ng 계획·실행·반성·도구 사용을 반복하며 결과를 개선하는 AI ※ Google Brain의 공동 창립자 겸 책임자	Anthropic 사용자와 협업하며 계획하고 행동하는 AI 동료	Microsoft 목표를 이해하고 계획하며 사용자를 대신해 행동하는 동적 소프트웨어	Salesforce 고객 맥락을 이해하고 업무를 자동 처리하는 비즈니스 AI 에이전트
	LangChain LLM이 도구를 선택해 반복적으로 문제를 해결하는 구조	AWS 사람이 목표를 주면 달성 방법을 스스로 선택하는 소프트웨어	Amazon Science 채팅을 넘어 실제 세계의 일을 대신 수행하는 AI
		NVIDIA 목표를 받아 추론하고 계획하며 실행하는 디지털 작업자	

AI Agent 정의



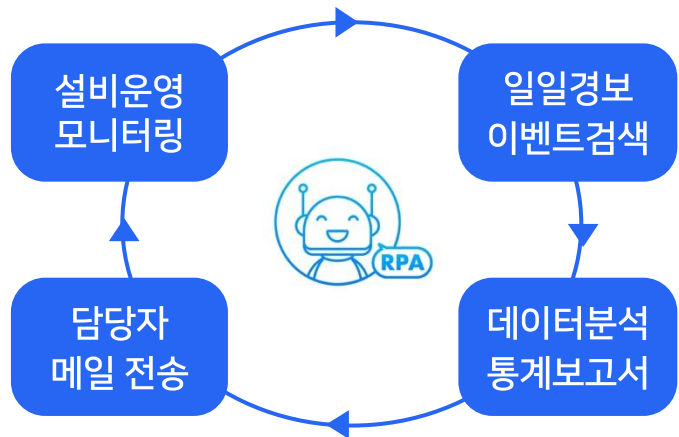
AI Agent 구성요소



AI 주요기술 _ RPA

RPA(Robotic Process Automation)는 비즈니스 프로세스에서 반복적이고 규칙적인 작업을 자동화하기 위해 **소프트웨어 로봇을 사용하는 기술**

【설비 모니터링 자동화 예시】



☞ 활용 : 설비상태 모니터링 분석 보고, 뉴스수집 메일링, 법인카드 전표 처리, 일일 보고서 작성

- RPA 주요기능
작업 자동화, 데이터 처리, 작업 스케줄링, 모니터링
 - RPA 구성 요소
워크플로우 정의도구, 실행 로봇(S/W), 오케스트레이터
- ※ 파이썬 패키지 : `pyautogui`, `pywinauto`, `openpyxl`

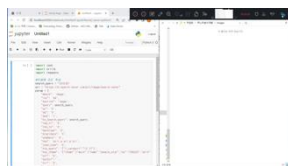
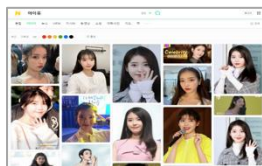
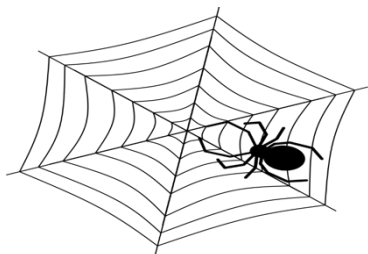
RPA 절차



AI 주요기술 _ 웹크롤링

인터넷 상에 존재하는 웹 페이지를 자동으로 탐색하고 숫자, 이미지, 영상 등
모든 데이터를 자유롭게 수집하는 기술

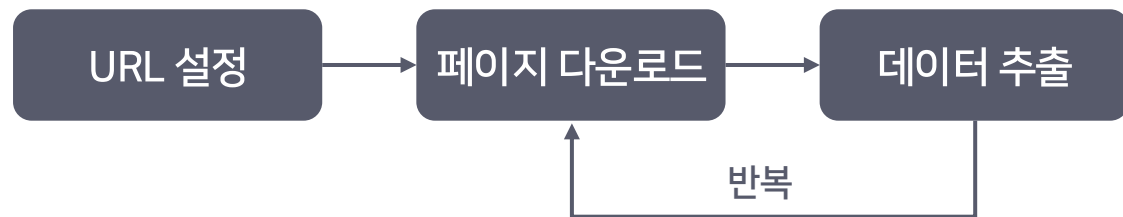
【웹크롤링】



↑ 네이버에서 특정 이미지를
빠르게 다운로드

- 크롤링 의미: 거미가 거미줄(web) 위를 기어다니며(crawl) 먹이를 찾는 것처럼 봇이 인터넷을 돌아다니며 정보를 수집하는 것을 비유
 - 크롤링을 수행하는 S/W가 네이버, 구글과 같은 검색 포털을 무작위로 돌아다니며 정보를 수집/분류/저장
 - 검색 엔진은 크롤링한 데이터를 정렬하여 보여줌
- ※ 파이썬 패키지 : `requests`, `beautifulsoup`, `selenium`

웹크롤링 절차



☞ 활용 : 뉴스 및 트렌드 수집, 제품 리뷰 평판 수집,
웹 사이트 모니터링, 이미지 다운로드, 티켓 예매

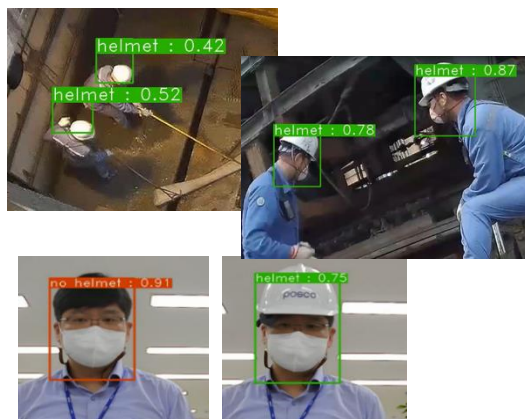
AI 주요기술 _ 컴퓨터 비전

컴퓨터 비전(Computer Vision)은 **컴퓨터가 디지털 이미지, 비디오 및 기타 시각적 입력으로부터 정보를 추출하고 이해하도록 하는 기술**

【Classification】



【Object Detection】



- 이미지처리 : 디지털 이미지를 변환, 개선 분석
 - 객체탐지 : 이미지나 비디오에서 특정 객체를 탐지, 위치 표시
 - 객체분할 : 이미지에서 객체를 픽셀 단위로 분리
 - 이미지분류 : 이미지를 분석해 특정 클래스나 라벨을 예측
- ※ 파이썬 패키지 : opencv, tensorflow, pytorch, torchvision

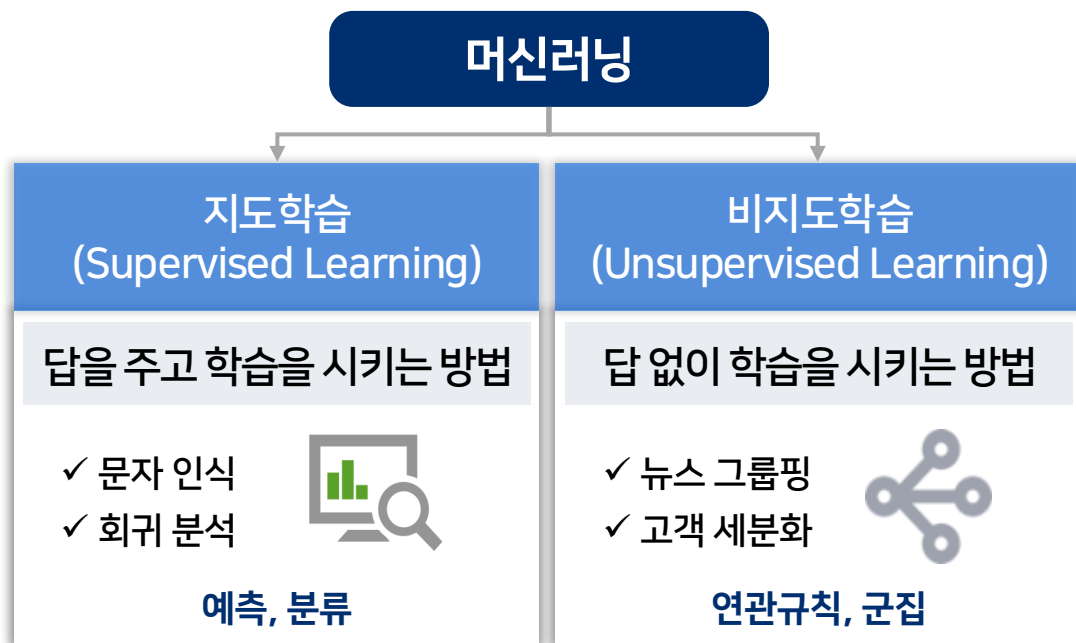
컴퓨터 비전 절차



☞ 활용 : 제품 표면 결함 판별, 작업자 안전상태 확인, 자율 주행, 침입자 감지, 의료 영상 판별

AI 주요기술 _ 머신러닝

머신러닝은 **통계적 방법과 알고리즘을 사용하여 데이터에서 패턴을 발견하고 이를 기반으로 예측, 분류, 군집 등의 의사 결정을 도와주는 기술**



☞ **활용** : 제품 수요 예측, 에너지 사용량 예측, 이상 거래 탐지, 스팸 메일 분류, 고객 세분화, 상품 배치 최적화

- **예측** : 과거 데이터를 기반으로 미래값을 추정
- **분류** : 데이터가 속할 범주나 클래스를 판별
- **군집** : 유사한 특성을 가진 데이터를 그룹으로 묶는 작업
- **연관** : 데이터간의 숨겨진 패턴이나 관계를 발견하는 작업

※ 파이썬 패키지 : `scikit-learn`, `statsmodels`, `pandas`, `numpy`

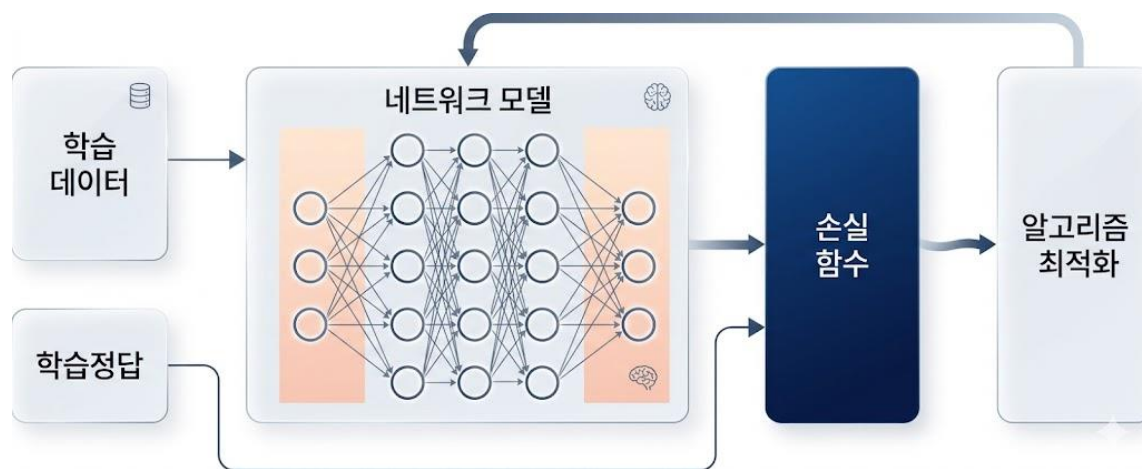
머신러닝 절차



AI 주요기술 _ 딥러닝

딥러닝은 인공신경망을 기반으로 한 머신러닝의 한 분야로, 컴퓨터가 **데이터로부터 스스로 복잡한 패턴과 표현을 학습**할 수 있도록 설계된 기술

【심층신경망 구조】



- 모델 : 데이터의 특성 학습, 뉴런과 레이어로 구성
- 손실함수 : 모델의 예측값과 실제값 간의 오차 측정
- 최적화 : 손실을 최소화 하기위해 모델의 가중치 업데이트
- 주요 모델 : CNN, RNN, GAN, Transformer, RL
- ※ 파이썬 패키지 : Tensorflow, Pytorch, Keras

딥러닝 절차



👉 활용 : 의료영상 분석, 이미지 복원 및 변환, 감정분석, 문서 요약, 음성인식, 통역, 이미지/텍스트 생성

AI 주요기술 _ Transformer

트랜스포머는 문장 속 단어들의 '관계(문맥)'를 파악하고,
데이터를 '동시에 처리(병렬)' 하는 인공지능 신경망 구조

【단어 선택】

입력: "바다에서 파도 소리를 들으며, 싱싱한 [?]"

문장 안의 단어들이 서로 얼마나 끈끈하게 연결되어 있는지(Attention) 점수를 계산

Attention: 바다, 파도, 싱싱한

1. 회 (45%), 2. 해산물 (30%), 4. 생선 (15%), 5. 커피 (2%), 6. 노트북 (0.001%)

바다에서 파도 소리를 들으며, 싱싱한 회

- 핵심 원리: 트랜스포머는 문장을 통째로 외우는 것이 아니라, 앞의 문맥을 수학적으로 계산해 다음에 올 가장 자연스러운 단어를 '확률'로 선택합니다

- 문맥에 따른 확률 변화

- "아침부터 굶었더니 배가 너무 [?]"
- "섬에 들어가기 위해 항구에서 배를 [?]"

- 릴레이 방식

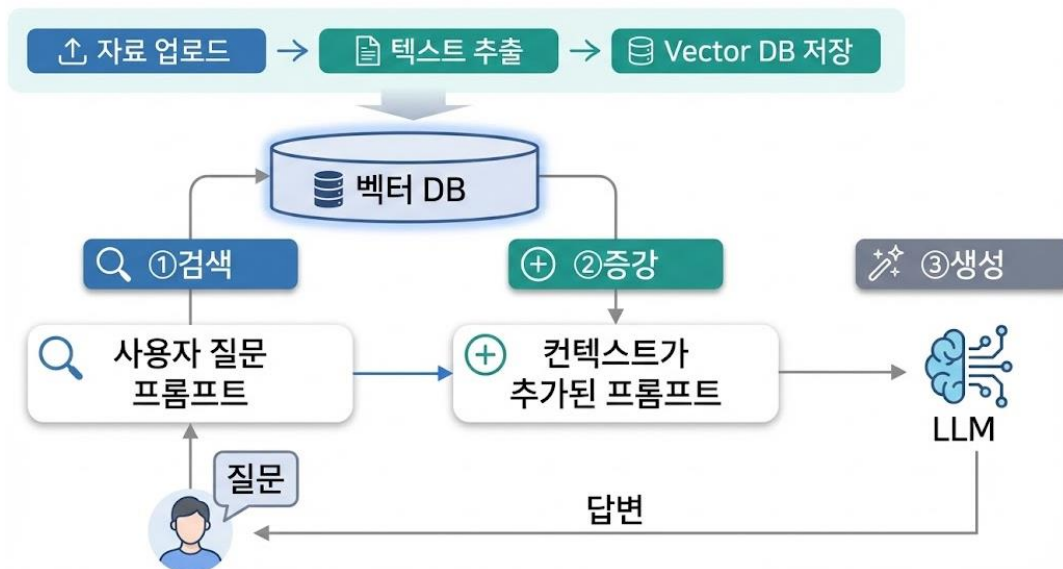
... 싱싱한 회를

... 싱싱한 회를 먹었다

AI 주요기술 _ Rag

RAG(Retrieval-Augmented Generation)는 대형 언어 모델(LLM)과 외부 데이터 소스를 결합하여 보다 정확하고 맥락적으로 풍부한 답변을 생성하는 기술

【RAG 시스템】



👉 활용 : 지식기반 고객상담 챗봇, 맞춤형 튜터링, 특정 법률, 규정, 판례 검색, 맞춤형 보고서 생성

- 검색단계 : 사용자의 질문에 대한 정보를 벡터DB에 저장한 외부 데이터 소스에서 검색
- 증강단계 : 검색된 데이터를 사용자가 입력한 질문과 결합하여 언어모델이 더 풍부하게 답변할 수 있도록 보조
- 생성단계 : 증강된 프롬프트를 바탕으로 사용자의 질문에 대한 최종 답변을 생성

※ 파이썬 패키지 : [Langchain](#), [Langgraph](#), [Openai](#)

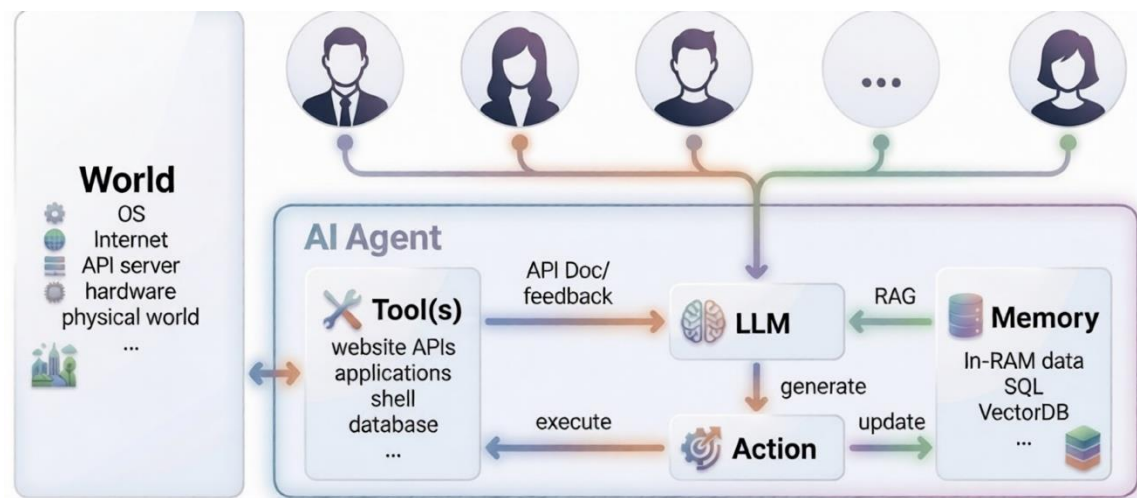
RAG 장점

- 외부 데이터 소스에서 필요한 정보를 실시간으로 가져올 수 있음
- 일반적인 언어 모델보다 더 정확하고 신뢰할 수 있는 답변을 제공

AI 주요기술 _ AI Agent

AI 에이전트(AI Agent)는 인공지능 기술을 기반으로 **특정 작업이나 목적을 수행하도록 설계된 소프트웨어** 또는 시스템을 의미

【AI Agent】



☞ **활용** : 공장 자동화, 로봇 제어, 시장 분석, 의료 상담,
맞춤형 학습경로 제안, 개인화된 제품 추천

- **Tool** : AI 에이전트가 특정 작업을 수행하거나 외부 시스템과 상호작용하기 위해 사용하는 기능 또는 인터페이스
- **Action** : AI 에이전트가 도구를 활용하여 실제 작업을 수행하거나 환경 상태를 변경하는 행위
- **Memory** : AI 에이전트가 과거 경험, 데이터를 저장하고 이를 학습하거나 활용하여 더 나은 의사결정을 할 수 있도록 하는 기능

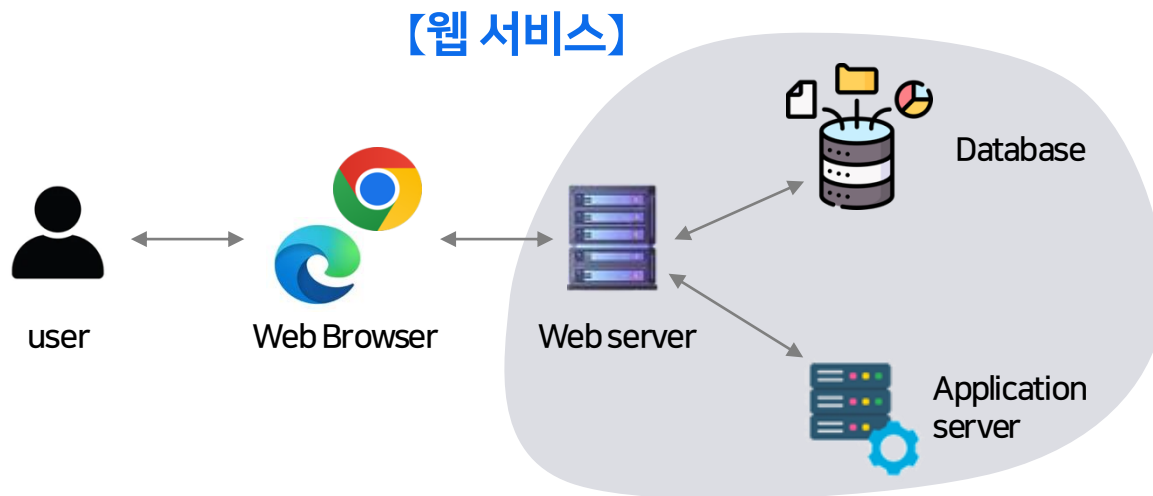
※ 파이썬 패키지 : Langchain, LangGraph, Openai, CrewAI

AI Agent 특징

- 자율적 의사결정과 행동을 통해 복잡한 문제를 해결하거나 인간의 작업을 보조하는 데 있어 뛰어난 성능을 발휘

AI 주요기술 _ 웹 프레임워크

웹 프레임워크는 웹사이트나 서비스를 개발할 때 필요한 기본 구조와 기능 (라우팅, 데이터베이스, 보안 등)을 미리 제공해주는 도구



- 풀 스택 프레임워크
 - 웹 서비스 개발에 필요한 모든 도구와 기능 제공
 - Django(python), Spring(java), Ruby on Rails(ruby)
 - 마이크로 프레임워크
 - 웹 서비스의 필수적인 기능만 제공, 간결하고 유연한 구조
 - Flask(python), FastAPI(python), Node.js(javascript)
- ※ 파이썬 패키지 : Django, FastAPI, flask, streamlit

※ 파이썬으로 간단하게 데이터 시각화 및 대시보드 웹 애플리케이션을 구현하는 “streamlit”

- 추가적인 서버/프론트엔드 개발 없이 Python 코드로 즉시 실행 가능한 애플리케이션 생성

AI 주요기술 _ 데이터베이스

데이터베이스(Database)는 데이터를 체계적으로 저장하고 관리하는 시스템으로 여러 사용자가 효율적이고 안전하게 데이터를 접근하고 활용할 수 있도록 설계

관계형 데이터베이스

- 데이터를 표(Table) 형태로 저장하며 테이블 간의 관계를 키를 통해 표현, 구조적 데이터 처리
- MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server

비관계형 데이터베이스

- 정형화 되지 않은 데이터를 저장하고 처리, 고정된 스키마가 없으며 다양한 데이터 형식을 지원
- MongoDB, Redis, Apache Cassandra, Neo4j *벡터DB : Chroma, Faiss, Weaviate

하이브리드 데이터베이스

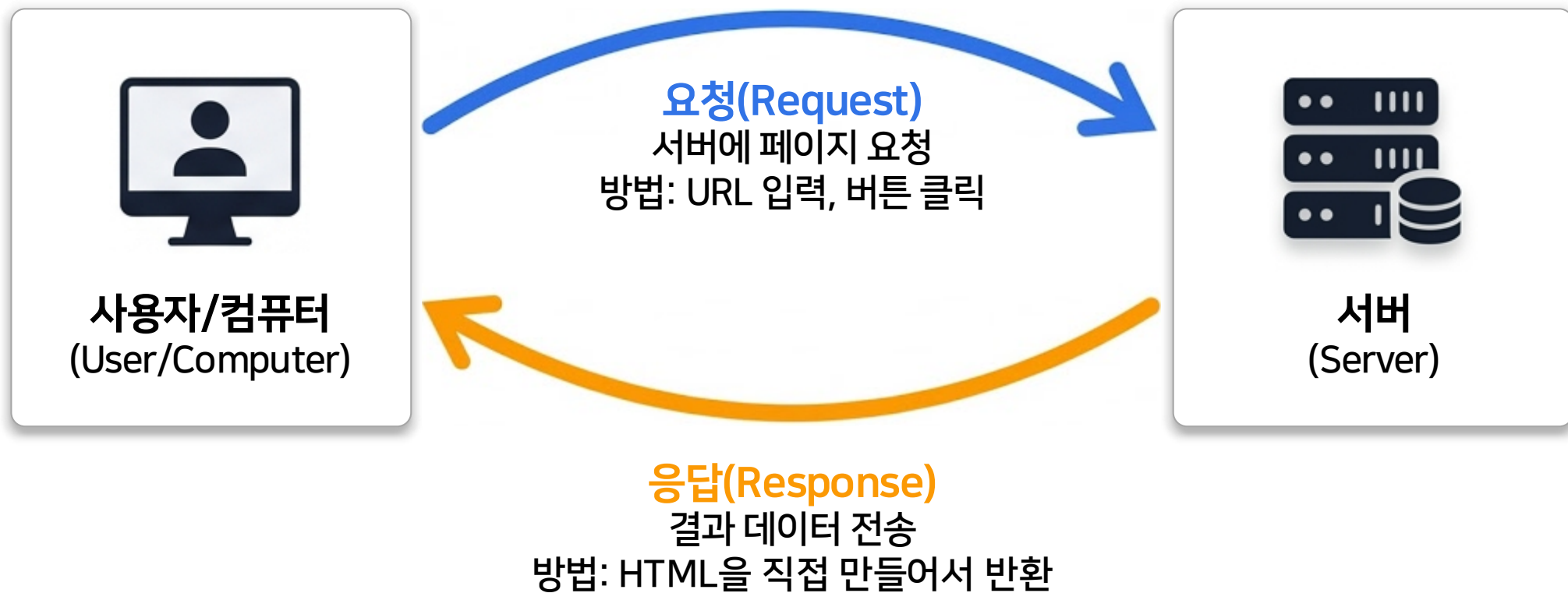
- 관계형 데이터와 비관계형 데이터를 혼합하여 저장 및 관리, 유연한 데이터 처리가 필요할 때 적합
- Amazon Aurora, Azure Cosmos DB

※ 파이썬 패키지 : sqlite3(파이썬 표준 라이브러리), mysql-connector-python, psycopg2, cx_Oracle, pymongo, elasticsearch

M1. AI Agent 개발 방법론 이해

2. 웹 이해

웹 통신방식: 요청과 응답



웹은 일방적인 방송이 아닌, 사용자의 행동(클릭)에 따라 서버가 반응하는 양방향 대화

웹의 구조

프론트엔드



클라이언트(Client / Browser)

사용자의 눈에 보이고
직접 상호작용하는 영역

UI & Interaction

백엔드



서버(Server / System)

눈에 보이지 않는 곳에서
데이터를 처리하고 저장하는 영역

Logic & Data

프론트엔드 구성 3대 요소

프론트엔드: 사용자가 웹사이트나 애플리케이션에서 직접 보고 상호작용하는 앞단 화면(GUI)

HTML

뼈대
(Skeleton)



구조와 배치
'여기는 제목, 여기는 버튼'

CSS

살과 옷
(Skin & Clothing)



디자인과 스타일
'버튼은 파란색, 모서리는 둥글게'

JavaScript

근육과 신경
(Muscle & Nerves)



동작과 생명력
'버튼을 누르면 팝업창을 띄워라'

HTML(HyperText Markup Language)

'무슨 내용을(WHAT)' 보여줄 것인가? 를 정의

- 웹 콘텐츠의 구조와 의미를 문단, 제목, 표, 삽입 이미지, 동영상 등의 태그로 정의하고 부여
- HTML Tag는 제목, 단락, 목록 등과 같이 웹문서 본문의 구조적 의미를 정의함
- Markup Language : 태그, 구분자 등을 이용해 문서나 데이터의 구조를 표기하는 언어



HTML Tag(태그)

<태그> 데이터 </태그>

- **웹페이지의 뼈대**가 되는 **HTML**에서는 모든 내용을 태그(Tag)로 작성하고 구조화
 - 웹 콘텐츠의 구조와 의미를 문단, 제목, 표, 이미지, 동영상 등에 해당하는 태그로 정의하고 부여
 - 태그는 사전에 약속된 암호와 같음. 태그를 사용하여 데이터를 나열하면 웹 브라우저가 해석 후 시각화
- **태그는 시작과 끝이 있음** : 시작 <태그명>, 끝 </태그명>
 - <html> 내용 </html>, <head> 내용 </head>, <p> ~ </p>

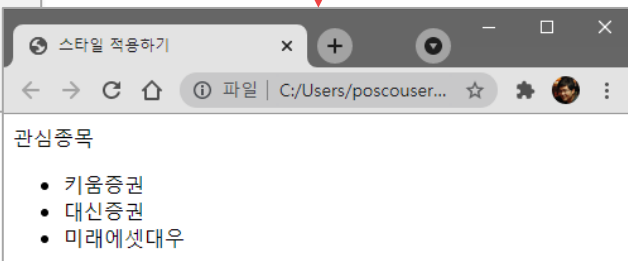
관심종목

키움증권

대신증권

미래에셋대우

렌더링



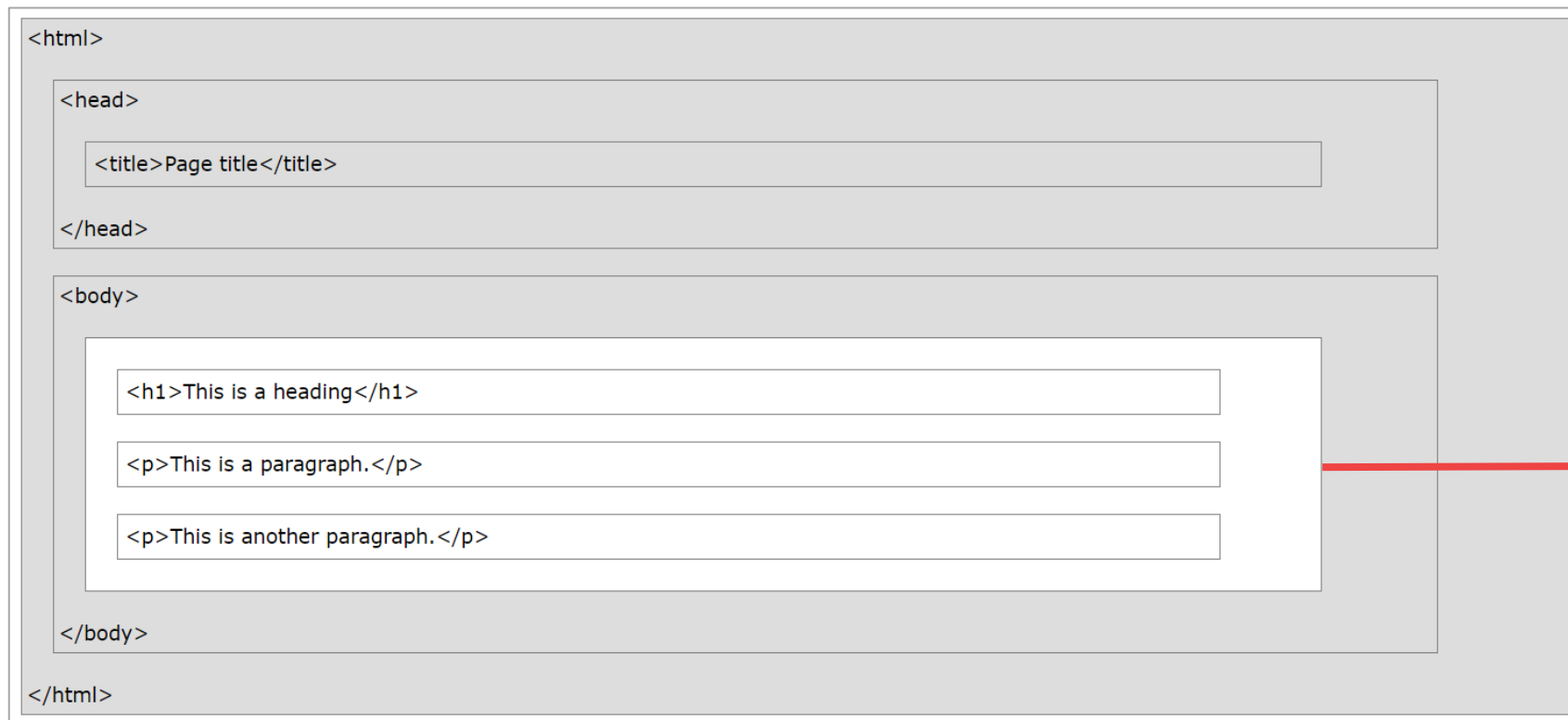
```
<p> == $0
<b>하이퍼텍스트 마크업 언어</b>
"("
  <span lang="en">
    "HyperText Markup Language, "
    <b>HTML</b>
  </span>
  ", "
  <span style="font-size: smaller;">
    <a href="/wiki/%E8%AC%B8%ED%99%94%E9%96%B4" title="문화어">문화어</a>
    " "
  </span>
  " 초본문표식달기언어, 하이퍼본문표식달기언어)는 "
  <a href="/wiki/%EC%98%B9_%ED%8E%98%E9%9D%B4%E7%80" title="웹 페이지">웹 페이지</a>
  "를 위한 지배적인 "
  <a href="/wiki/%E8%A7%88%ED%81%AC%EC%97%85_%E9%96%B8%EC%96%B4" title="마크업 언어">마크업 언어</a>
  "다. 또한, HTML은 제목, 단락, 목록 등과 같은 본문을 위한 구조적 의미를 나타내는 것뿐만 아니라 링크, 인용과 그 밖의 항목으로
  <a href="/wiki/%E8%B5%AC%EC%A1%B0%EC%A0%B1_%E8%AC%B8%EC%84%9C" title="구조적 문서">구조적 문서</a>
  "를 만들 수 있는 방법을 제공한다."
```

렌더링

하이퍼텍스트 마크업 언어(HyperText Markup Language, **HTML**, **문화어**: 초본문표식달기언어, 하이퍼본문표식달기언어)는 **웹 페이지**를 위한 지배적인 **마크업 언어**다. 또한, HTML은 제목, 단락, 목록 등과 같은 본문을 위한 구조적 의미를 나타내는 것뿐만 아니라 링크, 인용과 그 밖의 항목으로 **구조적 문서**를 만들 수 있는 방법을 제공한다.

HTML Tag의 기본 구조

- HTML 페이지는 <html> 태그 하위에 크게 <head>와 <body> 태그로 구분 됨
- <head> ~ </head> → 본문에 보여지지 않음 (메타데이터 등 문서의 속성을 정의)
- <body> ~ </body> → 본문에 보여지며 문서의 실제 내용이 하위 태그들로 작성되어 있음



<body> 섹션(흰색 영역) 내의
내용만 웹 브라우저 본문에 표시

HTML Tag의 기본 구조

<!-- 기본 HTML 구조 이해하기 -->

<html>

<head>

<title> My First Web Page </title>

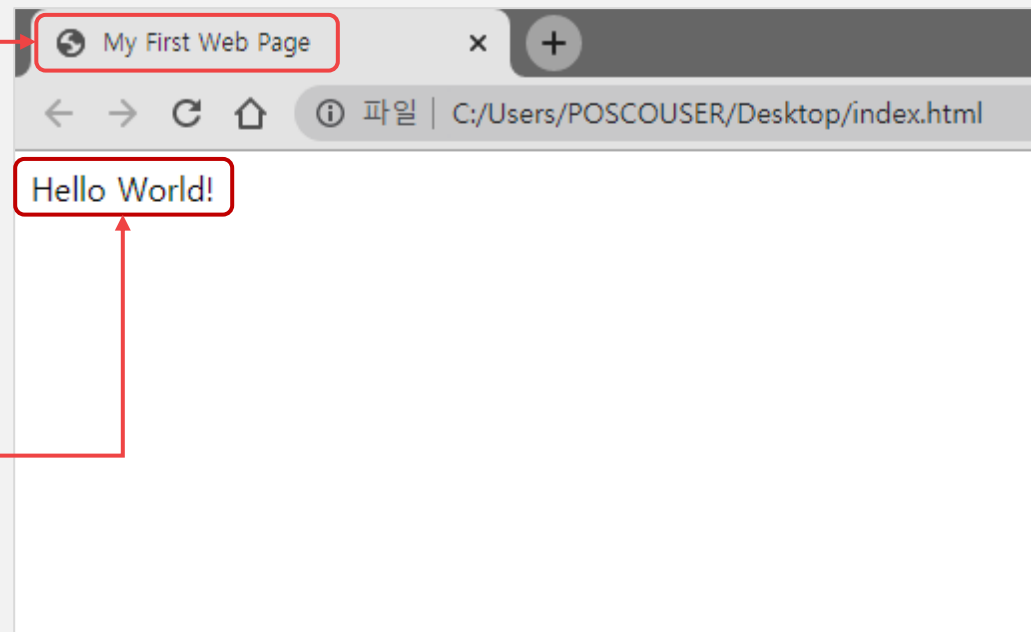
</head>

<body>

Hello World!

</body>

</html>



#<head></head> 부분은 본문에 나타나지 않으며, <body></body> 부분만 본문 화면에 나타남

네이버 HTML 확인

Naver 접속 → F12 클릭(개발자도구)

The screenshot shows the Naver homepage. At the top is a search bar with the text "검색어를 입력해 주세요.". Below it are icons for Mail, Cafe, Blog, Store, News, Stocks, Real Estate, Map, Webtoon, and Lounge. The main content area includes a featured advertisement for Japanese language learning, a section for "News Standards" with links to various news sources, and a "Shopping" section with product recommendations. There are also several advertisements, including one for a car and another for a home appliance.

The screenshot shows the Chrome DevTools 'Elements' panel. The HTML structure is displayed, starting with the DOCTYPE declaration and the root html element. The body contains several divs, including one with the id 'u_skip' and another with the id 'wrap'. The 'wrap' div contains a script tag, a script tag, and an iframe. The 'Elements' panel also shows the 'Styles' pane on the right, which displays the default styles for the selected element.

개발자 도구(DevTools)

- 웹페이지 내부 구조 확인 가능함
- HTML, CSS, JavaScript 분석 가능함
- 서버 통신 확인 가능함
- 웹 서비스 문제 분석 시 필수 도구임
- 실행방법: F12 클릭 / Ctrl + Shift + I / 마우스 우클릭 → 검사 클릭

구분	설명	실습
Elements	▪ HTML 구조 확인 ▪ CSS 스타일 확인 ▪ 화면 요소 수정	▪ 요소 선택 버튼 클릭 → 네이버 검색창 선택 → HTML 확인 (태그 종류 확인, id 확인, class 확인, style 확인)
Console	▪ JavaScript 실행 ▪ 오류 확인 가능 ▪ 변수 확인	▪ <code>document.querySelector("input").value</code> → 검색창 입력값 확인
Network	▪ 서버 통신 확인 ▪ API 호출 확인 ▪ 이미지 로딩 확인	▪ Network 탭 선택 → F5 새로고침 → 요청 목록 확인
Application	▪ 쿠키 등 확인	▪ 로그인 정보 확인, 저장 데이터 확인

CSS(Cascading Style Sheet)

내용을 '어떻게(HOW)' 보여줄 것인가? 를 정의

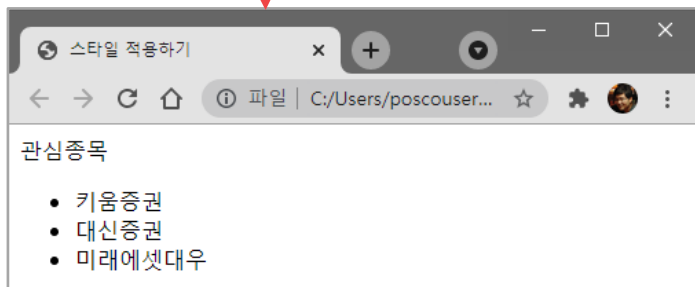
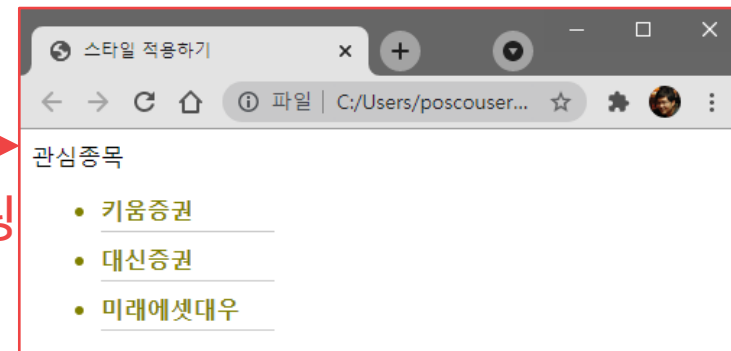
- HTML 등의 마크업 언어로 작성된 문서가 웹사이트에 표현되는 '서식'을 정해주는 언어
- HTML 문서 內 <style>태그 안에 HTML 각 요소의 배경색, 폰트 등의 내용을 정의하는 형태로 작성

```
관심종목<br>
<ul>
  <li>키움증권</li>
  <li>대신증권</li>
  <li>미래에셋대우
</li>
</ul>
```



```
<style>
li {
  width: 120px;
  margin: 8px;
  padding-bottom: 4px;
  font-weight: bold;
  color: olive;
  border-bottom: 1px solid #ccc;
}
</style>
```

렌더링



렌더링

네이버에 CSS를 제거하면...

NAVER

검색

검색어를 입력해 주세요.

Q 검색

입력도구 자동완성/최근검색어펼치기

- [메일](#)
- [카페](#)
- [블로그](#)
- [스토어](#)
- [뉴스](#)
- [증권](#)
- [부동산](#)
- [지도](#)
- [웹툰](#)
- [치지직추천](#)
- [바로가기 편집](#)

확장 영역

[페이지 바로가기](#)

알림

장바구니알림5

닥터피델 퓨어핏



아직도 변기청소 하는 분들 주목! 변기 청소 1년 동안 안 해요

타임 특가

- [뉴스탠드언론사편집](#)
- [엔터](#)
- [쇼핑투데이](#)
- [스포츠LIVE](#)

- [쇼핑투데이](#)
- [스포츠LIVE](#)
- [경제](#)

[분야별 뉴스](#)

[국민일보](#)

속보

[김정은 “우리 다치게 하면 韓완전봉고 배제 못해”](#)

[머니투데이](#)

속보

[北, 어제 김일성광장서 9차 당대회 기념 열병식 개최](#)

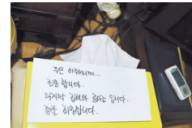
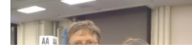
[뉴스홀](#)



[머니투데이](#) 구독

[\[르포\]"이게 진짜 아이폰" 갤럭시S26, 새 기능 발표하자...환호·탄성 쏟아졌다](#)

02월 26일 07:01



[서울경제](#) 구독

[“마지막 집세입니다, 죄송합니다”...세 모녀는 70만원 남기고 그렇게 떠났다.\[오늘의 그날\]](#)

02월 26일 07:30



[이전 페이지](#)

언론사 더보기1페이지전체/21

[다음 페이지](#)

[쇼핑](#) / [맨즈](#) / [브랜드샵](#) / [추천하달](#) / [MY추천](#) / [쇼핑라이브](#)

1/14 < >

32

JavaScript(JS)란?

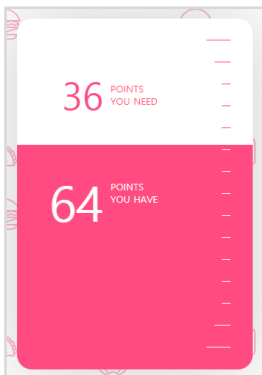
HTML 문서 안에 삽입되어 실행되는, 프로그래밍 가능한 인터프리트(Interpret) 언어

- 아주 먼 과거(20C)에는 JS없는 웹페이지도 많았으나, 이제 JS없이 웹페이지를 만든다는 것은 상상도 못할 일
- 따라서 모든 웹브라우저가 필수적으로 JS 코드 해석기(Interpreter)를 내장하여 JS 실행을 지원하고 있음



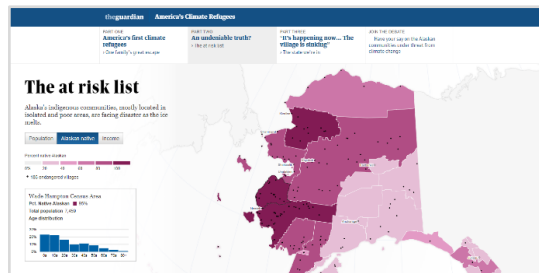
JS로 할 수 있는 것들

애니메이션*(anime.js)

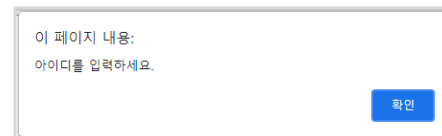


<https://codepen.io/lmgonzalves/pen/NyzKdr>

그래프 그리기(데이터 시각화)



경고창 출력



웹게임



<https://santatracker.google.com/intl/ko/>

*애니메이션은 CSS에서 지원하는 기능이나, 사용의 편리함 때문에 JS 라이브러리를 활용하는 것이 일반적

백엔드: 주방과 창고 비유

서버 (Server Application)



복잡한 연산을 처리하고
주문을 받는 주방

Python Node.js Java

데이터 요청 및 저장

데이터베이스 (DB: Data Base)



회원 정보와 게시물을
영구적으로 보관하는 창고

MySQL PostgreSQL

API(Application Programming Interface)

프로그램이 다른 프로그램의 기능이나 데이터를 사용할 수 있게 해 주는 연결 통로

API 미사용

모든 기술을 직접 구축하는 방식

- ❌ 지도/길찾기 직접 개발
전국 지도 데이터 수집 및 도로 데이터베이스 구축(수년 소요)
- ❌ 카드사별 직접 개별 연동
신한/삼성/현대 등 모든 카드사와 개별 계약 및 자체 보안 망 구축
- ❌ 개발 비용 & 시간 폭발
수십 명의 수석 엔지니어 필요, 앱 하나 만드는데 3년 이상 걸림



맛집 앱

직접 개별 연동
(복잡성/시간 폭발)



자체 지도 구축 필요



카드사개별망

API 사용

검증된 API를 조립하는 방식

- ✅ 카카오맵 / 네이버지도 API
코드 입력만으로 맛집 위치 표시 및 실시간 길찾기 완벽 연동
- ✅ 토스페이먼츠 / 아임포트 API
보안 걱정 없이 간편 결제(카카오페이, 제로페이) 탑재
- ✅ 압도적 경쟁력 확보
1인 개발자로 빠르게 앱 출시 후 핵심(맛집 추천)에 집중



맛집 앱

API연동



카카오맵
API



결제 API

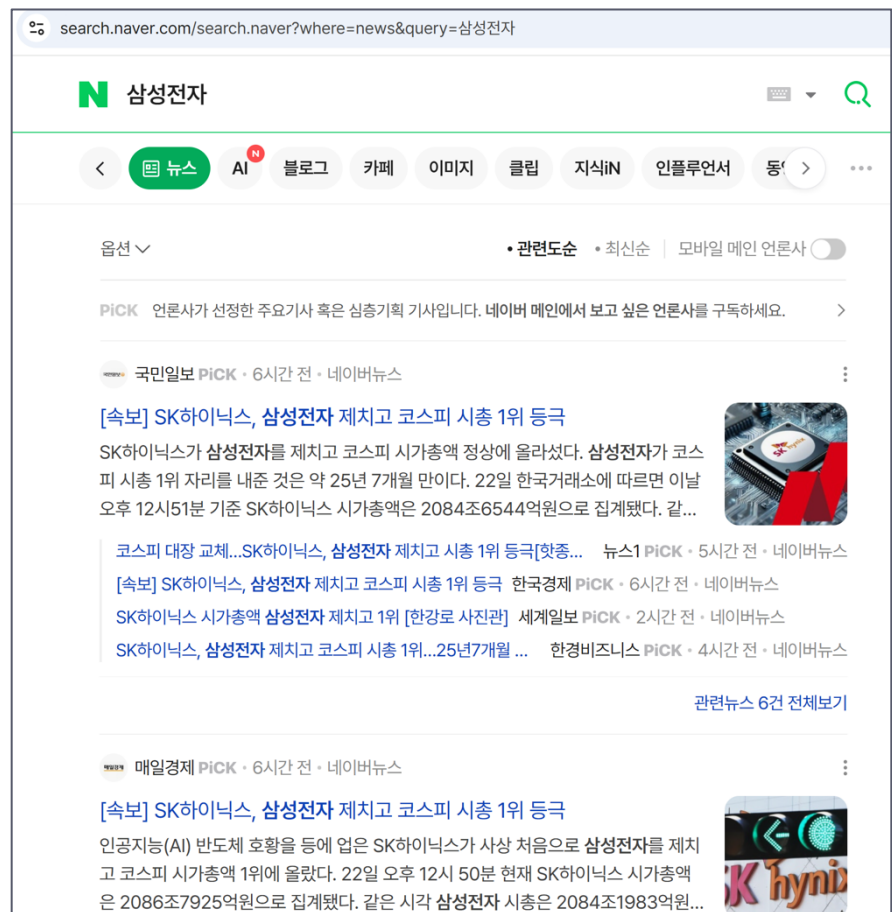


문자인증
API

웹페이지와 API 비교

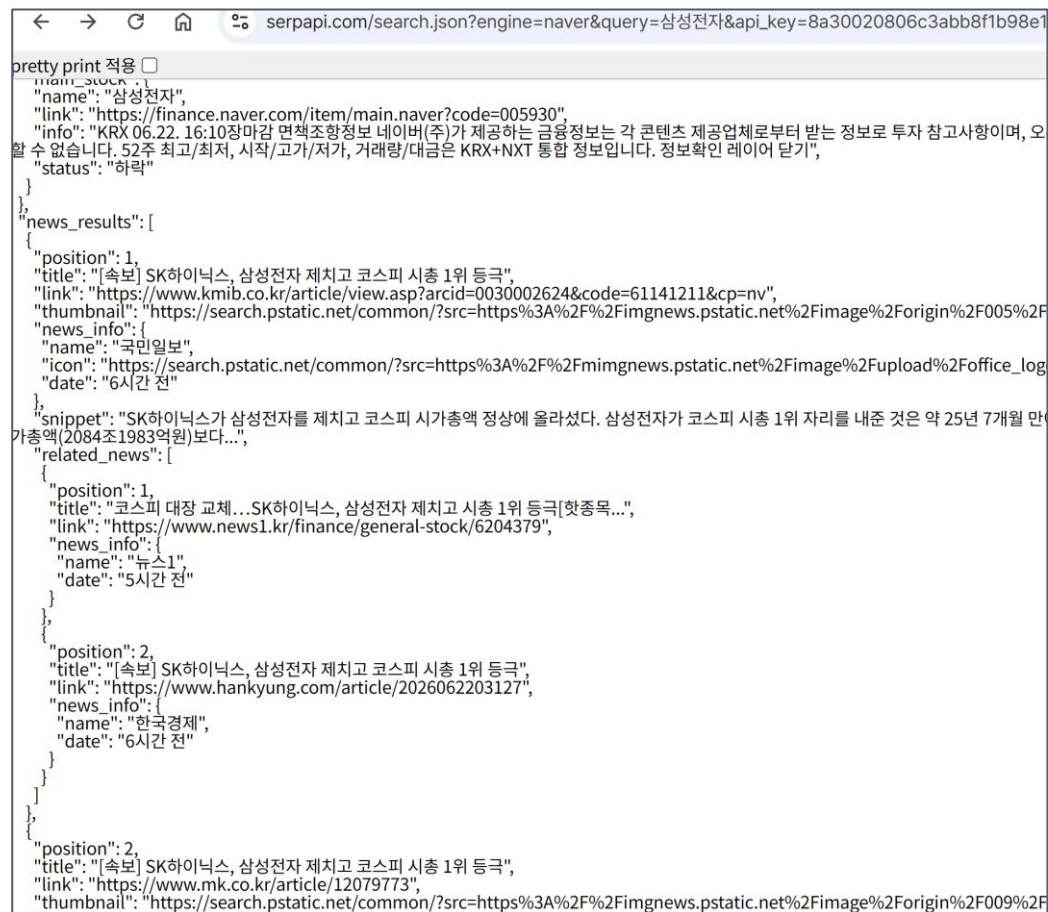
■ 웹페이지 요청

`https://search.naver.com/search.naver?where=news&query=삼성전자`



■ API 요청

`https://serpapi.com/search.json?engine=naver&query=삼성전자&api_key=키`



JSON: 데이터 형식

- 데이터를 저장하고 교환하기 위한 텍스트 형식
- 웹 API, 설정 파일, 로그, 데이터 저장에 많이 씀
- 사용규칙: 키(Key)와 문자열은 쌍따옴표, 값(Value) 사이에는 콤마, true/false/null은 소문자, 주석 지원 안 함

JSON 구조

{ Key : Value }

- 파이썬의 딕셔너리와 유사한 구조
- 직관적이고 가공이 편리한 텍스트 형식
- 화면에 필요한 데이터만 전송

JSON 예시

```
1 {  
2     "id": "user123",  
3     "status": "success",  
4     "message": "로그인 성공"  
5 }
```

* 예시 주소: <https://api.github.com/users/torvalds>

* 비교 주소: <https://github.com/torvalds>

M2. 바이트코딩 활용 앱/웹 개발

3. 바이트코딩 환경 구축

바이브코딩

복잡한 프로그래밍 문법 대신, 자연어로 의도(Vibe)를 설명하면 AI가 코드 작성



Andrej Karpathy
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

6:17 PM · Feb 2, 2025 · 4M Views



1.2K



3.9K



24K



12K



"바이브 코딩(vibe coding)"이라고 부르는 새로운 종류의 코딩이 있습니다. 여기서는 '바이브(vibe)'에 완전히 몸을 맡기고, 기하급수적인(AI의) 발전 속도를 받아들이며, 코드가 존재한다는 사실조차 잊어버립니다.

(중략)

저는 항상 '모두 수락(Accept All)'을 누르고, 이제 더 이상 코드 변경 사항을 읽지 않습니다. 에러 메시지가 뜨면 아무 설명 없이 그냥 복사해 붙여 넣는데, 보통 그러면 해결됩니다.

(중략)

저는 프로젝트나 웹 앱을 만들고 있지만, 이건 사실 코딩이 아닙니다. 그저 보고(see), 말하고(say), 실행하고(run), 복사해서 붙여 넣을(copy paste) 뿐인데, 대부분 잘 작동합니다."

- 2025년 2월, 안드레이 카파시 (Andrej Karpathy)의 언급

바이브코딩 Tools

도구 명칭		주요 특징 및 용도
커서(Cursor)		AI 기반 코드 편집기로, 기존 코드를 이해하고 자연어로 전체 파일 수정 및 생성이 가능
Google AI Studio		구글의 최신 모델(Gemini)을 테스트하고 프롬프트를 설계하여 빠르게 API 프로토타입 생성
Claude Code		앤트로픽(Anthropic)에서 제공하는 터미널 기반 도구로, 명령줄에서 직접 복잡한 코딩 작업을 수행
코덱스(Codex)		OpenAI의 모델로, 자연어를 코드로 변환하는 엔진 역할을 하며 다양한 서비스의 기반 마련
리플릿(Replit)		브라우저 기반 개발 환경으로, 'Replit Agent'를 통해 대화만으로 웹 서비스 전체를 구축
안티그래비티(Antigravity)		AI 에이전트 중심의 통합 개발 환경(IDE). '에이전트 우선(Agent-first)'방식의 플랫폼
제미나이(Gemini)		구글의 AI로, 코드 생성뿐만 아니라 논리적인 구조 설계 및 데이터 시각화 Prototyping에 강점
챗지피티(ChatGPT)		'Canvas'나 'GPTs' 기능을 통해 인터랙티브 한 앱 프로토타입과 문서, 코드를 즉각 생성
오팔(Opal AI)		시각적인 노드 기반이나 자연어 흐름을 통해 AI 앱의 로직을 설계하고 결과물을 확인

실습준비



시스템 및 프로그램 설정



① 파이썬 설치 : 3.11.9



② 패키지 설치 : requirements.txt 파일 작성 및 설치
`pip install -r requirements.txt`



③ Cursor AI 설치



④ Git 설치 * 2일차 실습, Skill 설치시 필요

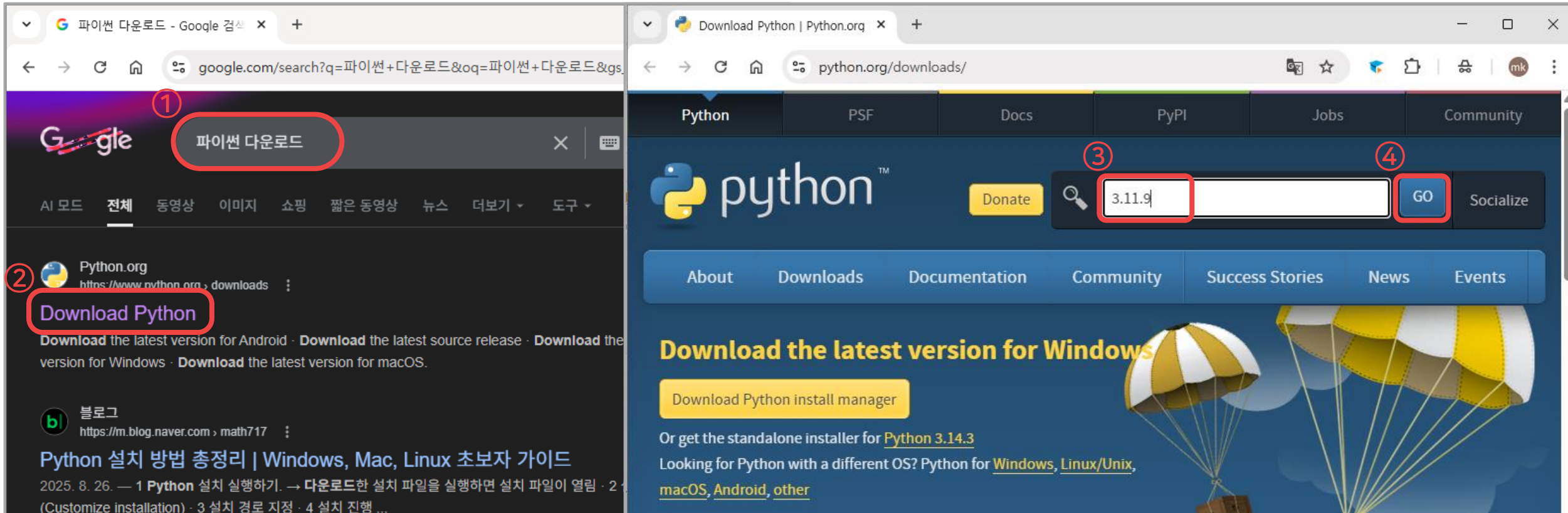


필수 패키지 목록(requirements.txt)

```
easyocr>=1.7.0
email-validator==2.1.0
fpdf2==2.7.9
Flask==3.0.2
Flask-SQLAlchemy==3.1.1
Flask-Login==0.6.3
Flask-WTF==1.2.1
matplotlib==3.8.2
numpy==1.26.2
openai
opencv-python>=4.8.0
openpyxl==3.1.2
pandas==2.1.4
pdf2image==1.17.0
pdfplumber==0.11.0
pytesseract==0.3.10
Pillow==10.2.0
PyMuPDF>=1.23.0
pypdf==4.0.1
PyQt5
python-pptx>=0.6.21
scikit-learn==1.3.2
scipy==1.11.4
seaborn==0.13.0
statsmodels==0.14.1
torch>=2.0.0
torchvision>=0.15.0
WTFORMS==3.1.1
Werkzeug==3.0.1
xlrd==2.0.1
yt-dlp==2024.12.23
.
```

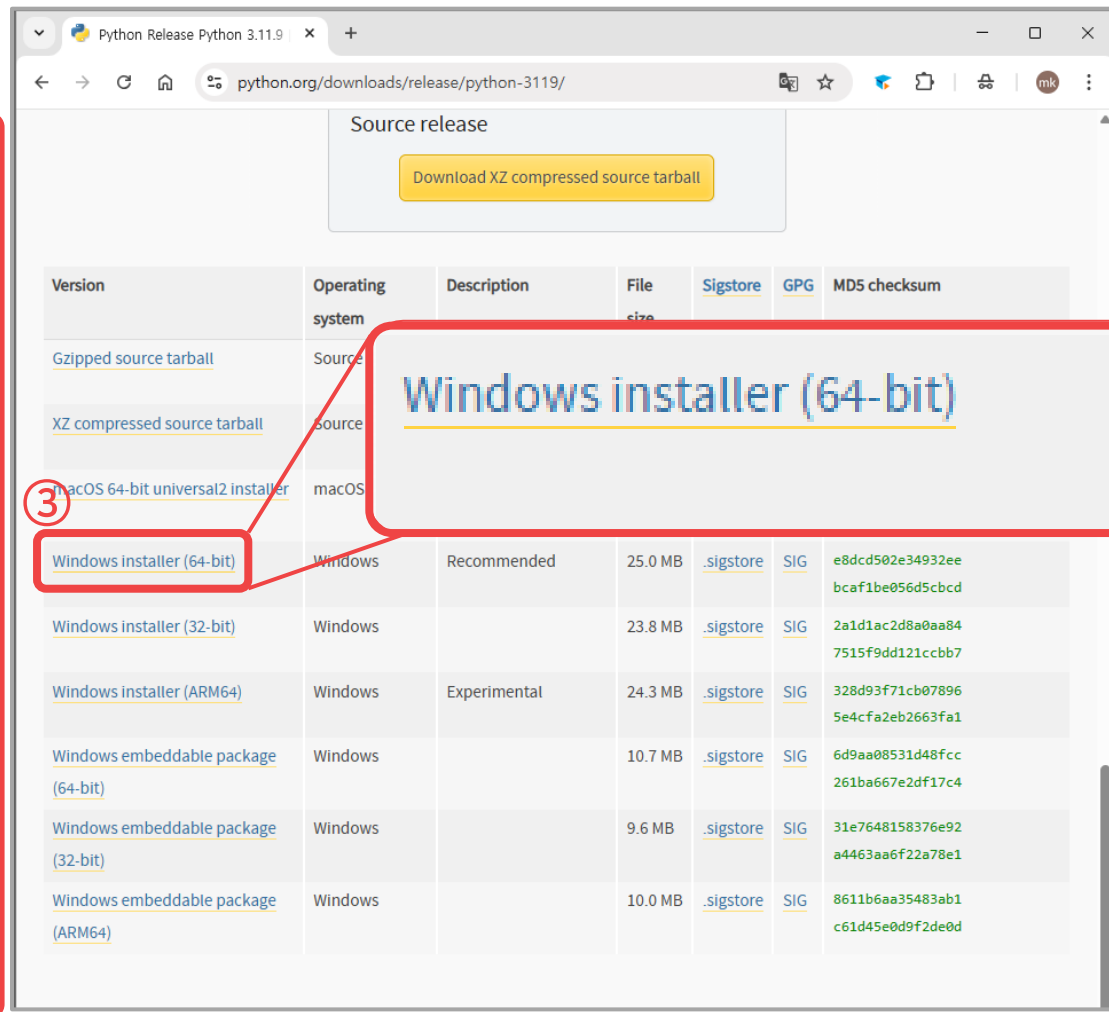
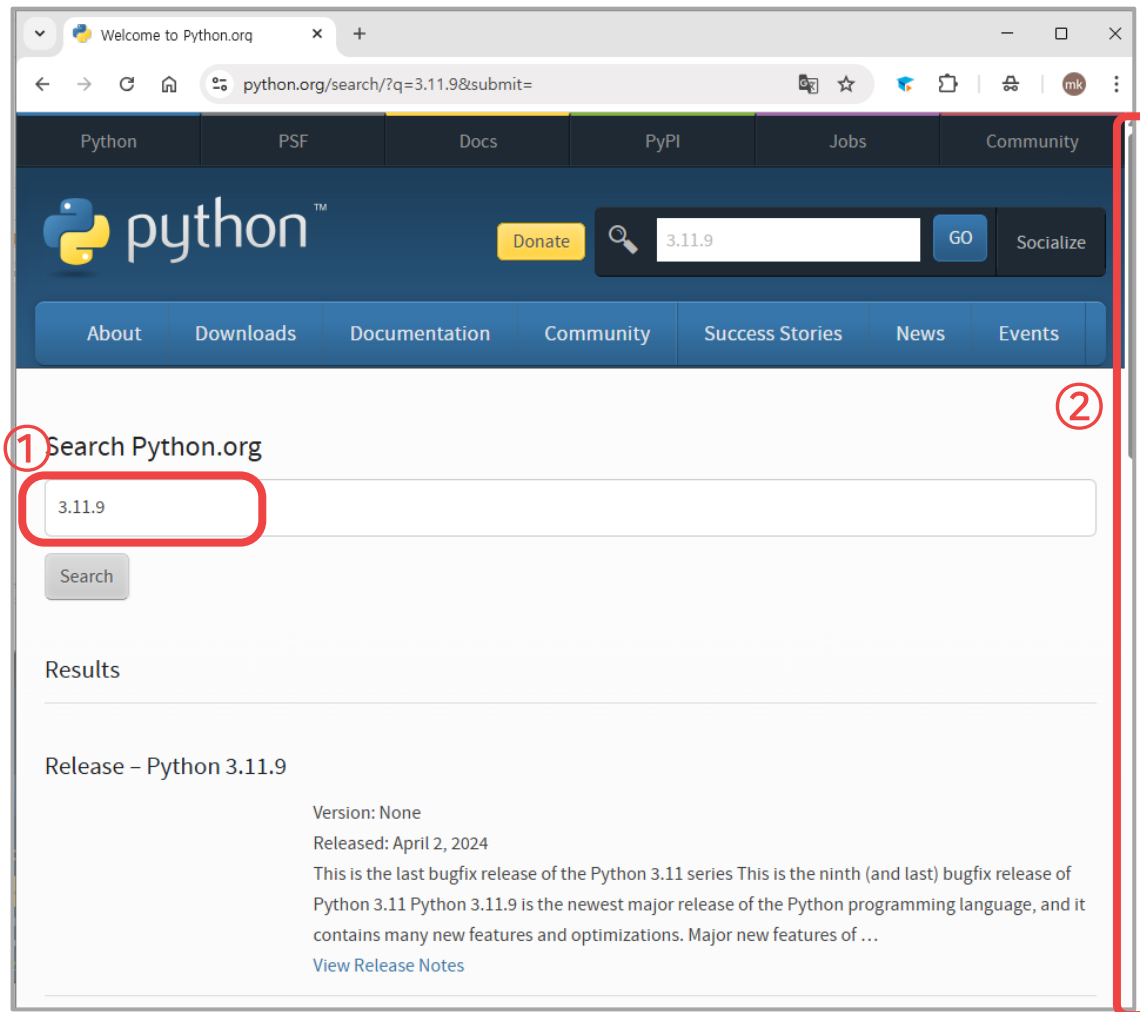
실습준비 _ ① 파이썬 설치

①크롬에서 파이썬 다운로드 검색 → ②Download Python 클릭 → ③3.11.9 검색 → ④클릭



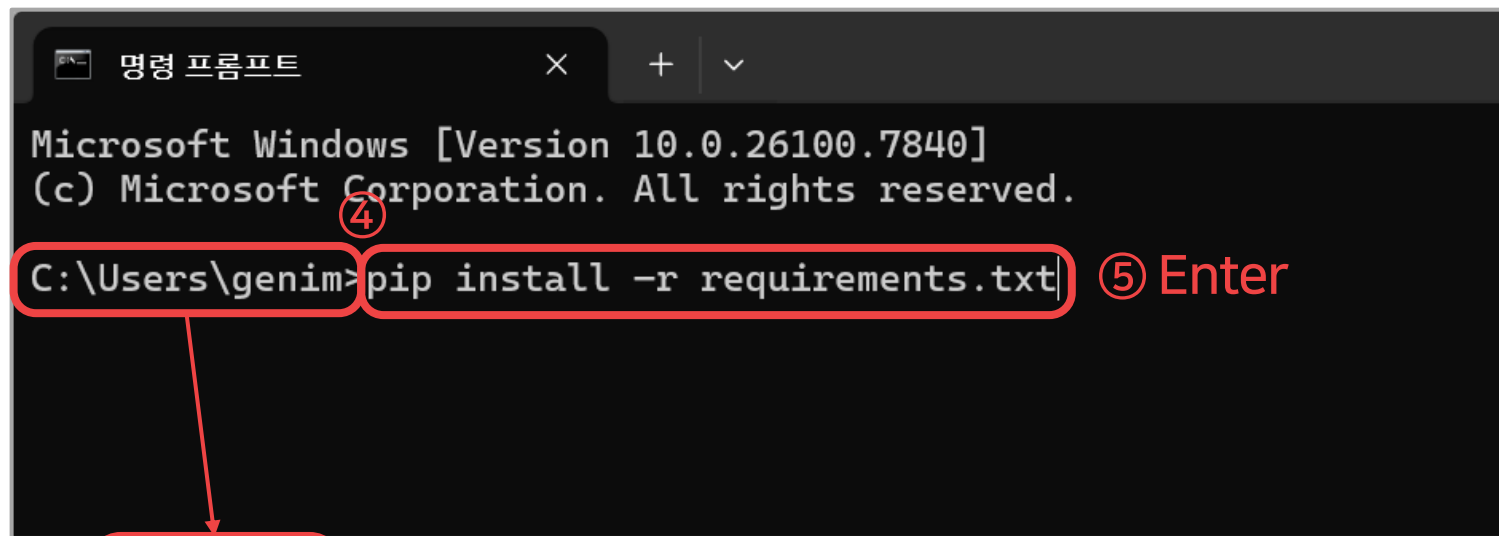
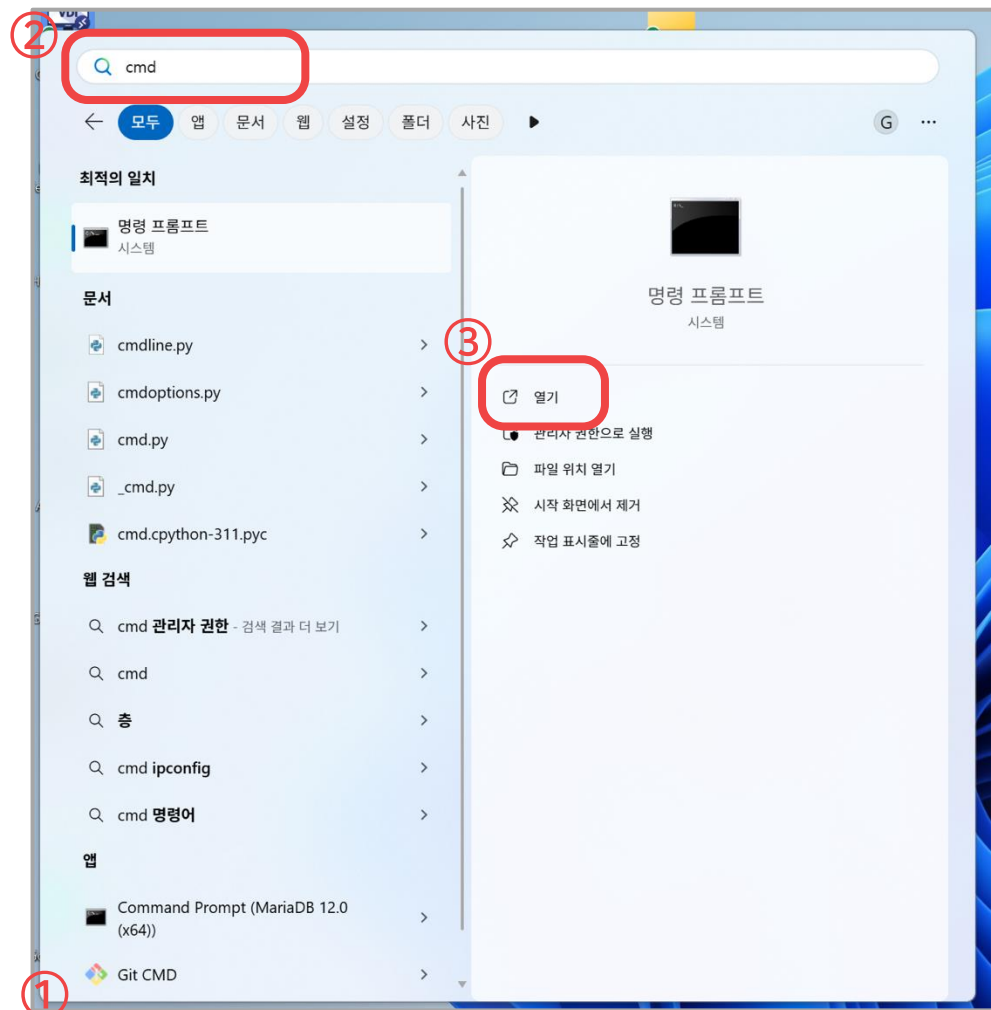
실습준비 _ ① 파이썬 설치

① Release - Python 3.11.9 클릭 → ② 스크롤 아래로 내리기 → ③ Windows Installer (64-bit) 클릭



실습준비 _ ② 패키지 설치

① 윈도우 클릭 → ② cmd 입력 → ③ 명령 프롬프트 선택 → ④ pip install -r requirements.txt 입력 → ⑤ Enter

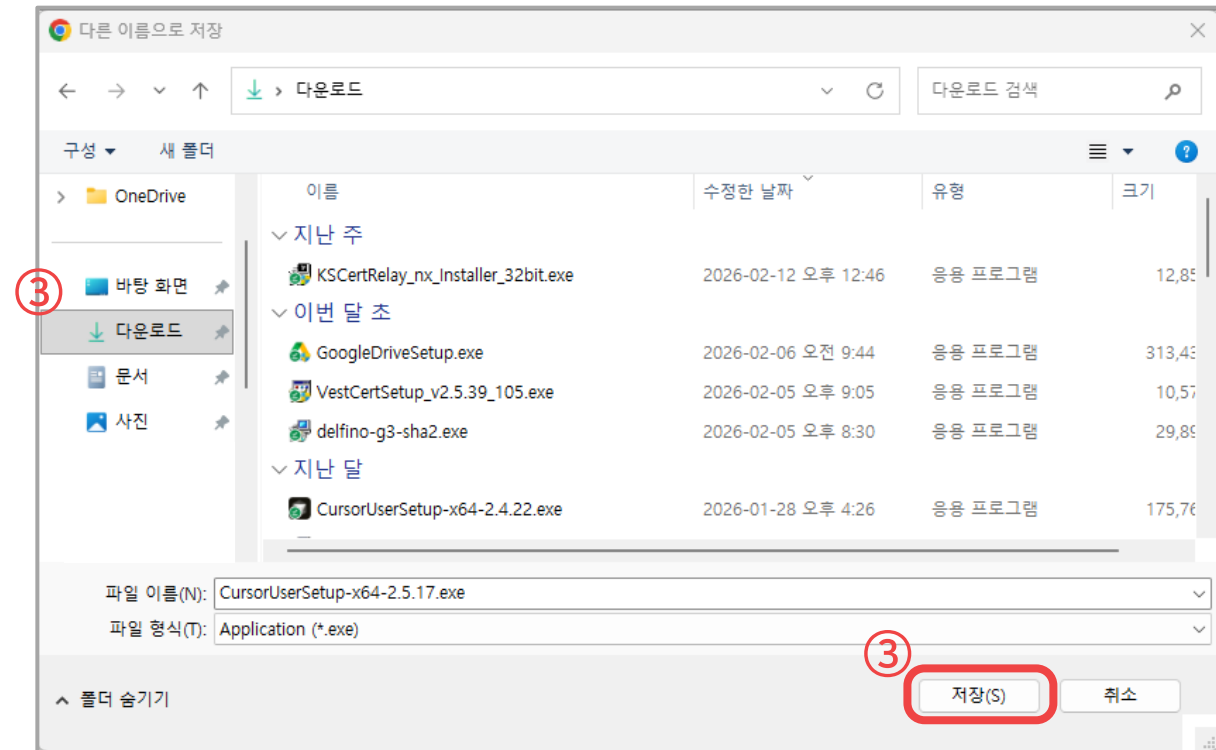
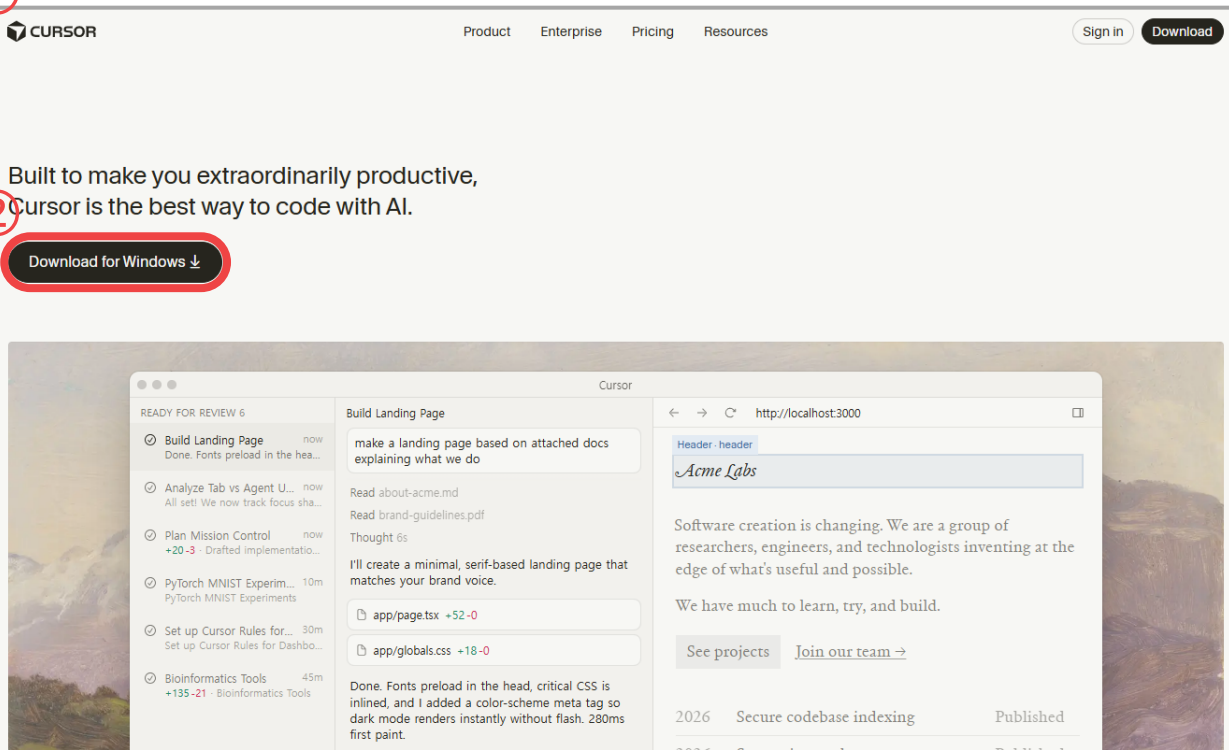


※ 해당 폴더에 requirements.txt 파일이 있어야 함

실습준비 _ ③ Cursor AI 설치

① cursor.com 접속 → ② Download for Windows 클릭 → ③ 다운로드 폴더 저장 → ④ 프로그램 설치

①



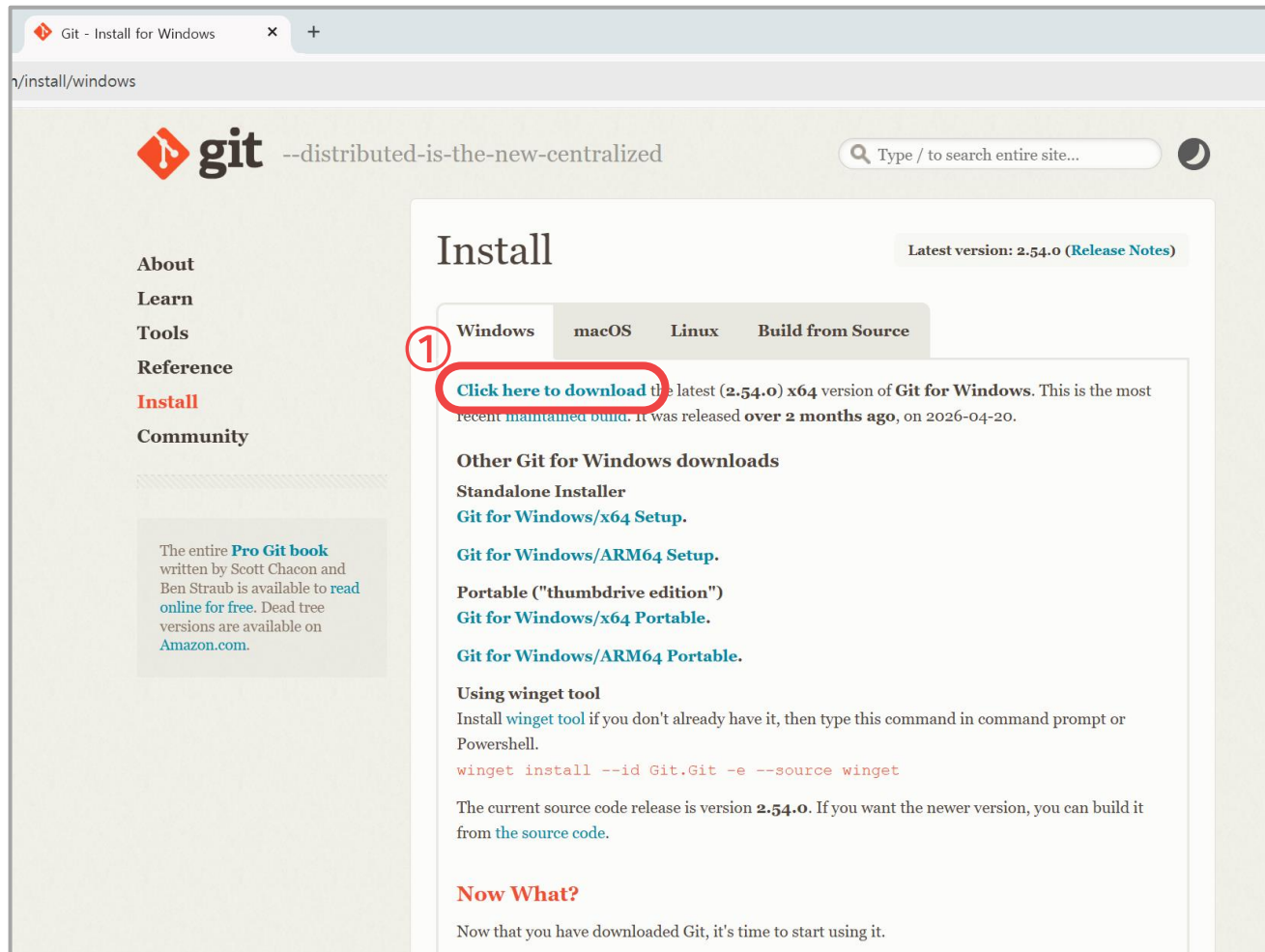
실습준비 _ ④ Git 설치

① 크롬에서 **Git 다운로드** 검색 → ② **Git - Install** 클릭 → ③ **Windows** 클릭 * 개인 PC 환경에 맞춰서 클릭

The image consists of two overlapping browser window screenshots. The left window shows a Google search for 'Git 다운로드' (Git download). The search bar contains the text 'Git 다운로드' which is circled in red with a red circle containing the number '1'. Below the search bar, the first search result is 'Git' from 'https://git-scm.com', with the link 'Git 설치' (Git installation) circled in red with a red circle containing the number '2'. The right window shows the 'Git - Install' page. The 'Install' section is circled in red with a red circle containing the number '3', and within it, the 'Windows' button is circled in red. The page also shows the 'Latest version: 2.54.0 (Release Notes)' and a 'Choose your operating system above.' prompt.

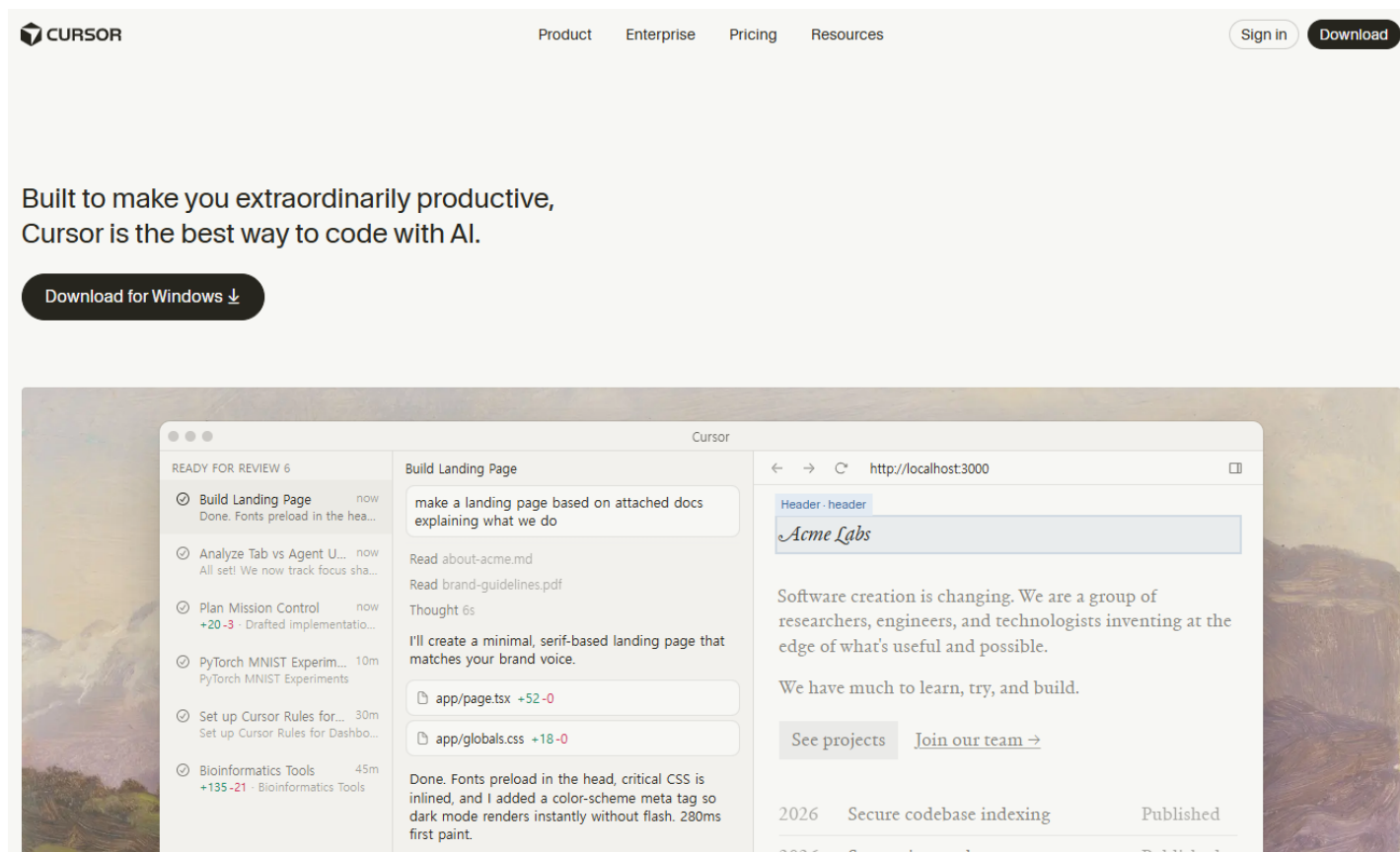
실습준비 _ ④ Git 설치

① Click here to download 클릭 → 다운로드 폴더에 저장 → 다운로드 폴더로 이동 → 설치파일 더블클릭



Cursor 개요

1. VS Code 기반의 'AI 네이티브' 에디터
2. 에디터 통합으로 프로젝트 전체 문맥(Context) 이해
3. 자연어 명령으로 멀티 파일 생성 및 수정
4. 개발자 의도를 예측하는 실시간 코드 완성
5. 최신 LLM 모델 선택 및 데이터 보안 지원



Cursor 주요 기능

1. 탭(Tab) : 사용자의 패턴을 학습하는 스마트 코드 자동 완성 기능

여러 줄 편집, 놓친 오류 수정, 다음 작업 예측 가능

2. 에이전트(Agent): 복잡한 코드 변경을 위한 AI 페어 프로그래머

정확한 컨텍스트 제어와 자동 수정으로 대규모 편집 가능

3. Cmd-K: 빠른 인라인 코드 편집 및 생성 기능

작업 흐름을 방해하지 않고 정확한 변경 가능

4. 채팅(Chat): 코드베이스를 이해하는 컨텍스트 인식 AI 어시스턴트

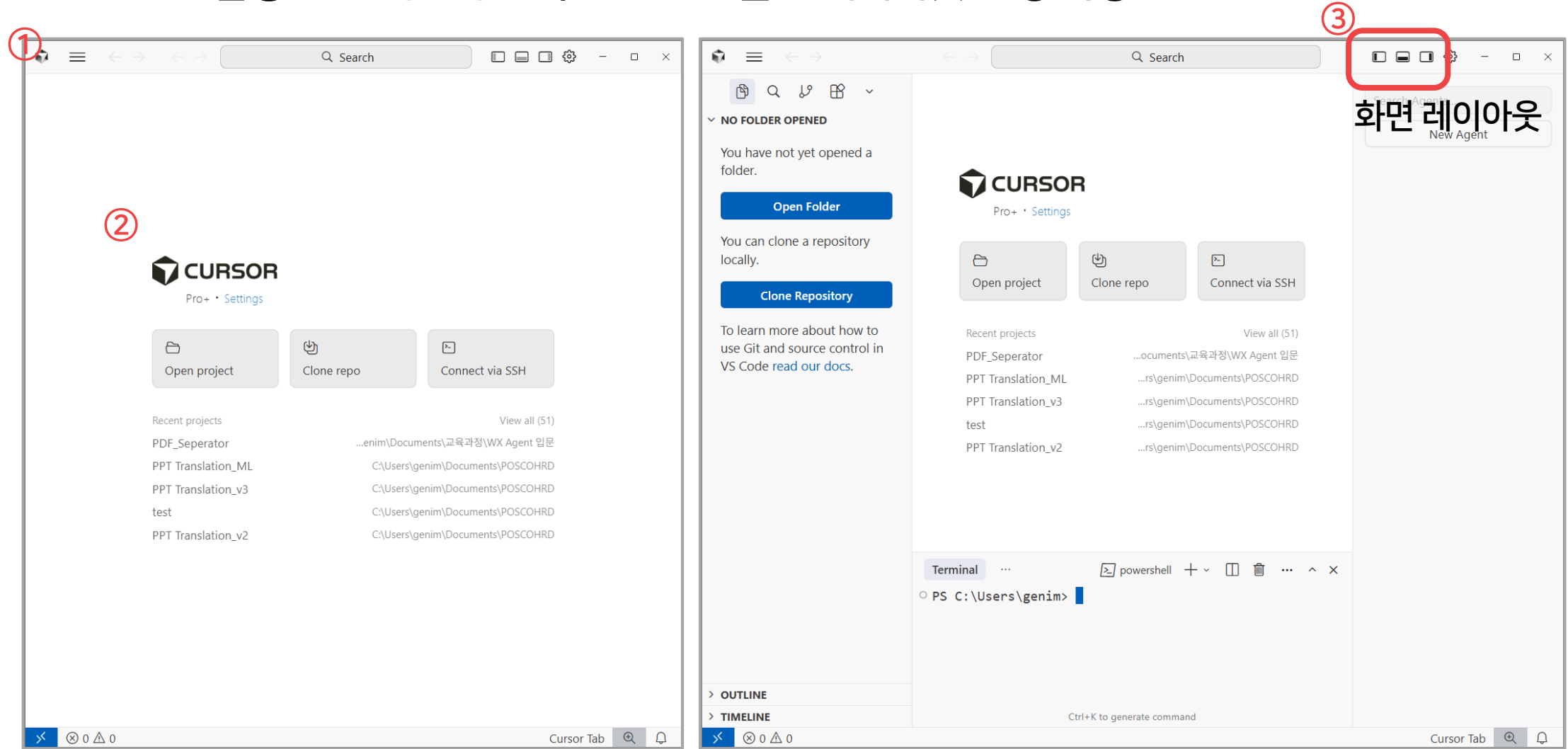
답변을 얻고 코드 변경사항을 에디터에 직접 적용

Cursor 주요 단축키

분류	기능	Windows	macOS	설명
AI	인라인 편집	Ctrl + K	Cmd(⌘) + K	현재 라인이나 선택한 범위에서 즉석으로 AI 편집 및 코드 생성
	통합 AI 인터페이스	Ctrl + I	Cmd(⌘) + I	사이드바 채팅창을 통해 코드 질문이나 로직 설명을 요청
	AI Chat	Ctrl + L	Cmd(⌘) + L	질문, 편집, 에이전트 모드가 포함된 Composer를 열어 프로젝트 전체 대상으로 작업
	코드 자동완성 수락	Tab	Tab	AI 기반의 코드 자동완성을 활성화하고 제안된 내용을 수락
설정	Cursor 설정 메뉴 열기	Ctrl + Shift + J	Cmd(⌘) + Shift(⌘) + J	사용할 AI 모델 선택 등 Cursor 전용 설정 메뉴 열기
	명령 팔레트/에디터 설정	Ctrl + Shift + P	Cmd(⌘) + Shift(⌘) + P	스타일 설정 인터페이스를 열거나 모든 명령어를 검색 및 실행
탐색	파일 검색	Ctrl + P	Cmd(⌘) + P	프로젝트 내의 파일을 이름으로 빠르게 찾아 열기
	코드 정의로 이동	F12	F12	선택한 함수나 변수가 정의된 실제 위치로 즉시 이동
	전체 프로젝트 텍스트 검색	Ctrl + Shift + F	Cmd(⌘) + Shift(⌘) + F	프로젝트 전체 파일 내에서 특정 텍스트를 검색
편집	코드 포매팅	Shift + Alt + F	Shift + Opt(⌥) + F	코드를 규칙에 맞게 자동으로 정렬
	주석 처리 / 해제	Ctrl + /	Cmd(⌘) + /	현재 줄이나 선택 영역을 주석으로 처리하거나 해제
	멀티 커서	Ctrl + D	Cmd(⌘) + D	동일한 단어를 찾아 다중 선택하여 한 번에 편집
화면	사이드바 제어	Ctrl + B	Cmd(⌘) + B	왼쪽 사이드바 탐색창을 보이거나 숨기기
	하단 패널 제어	Ctrl + J	Cmd(⌘) + J	터미널이나 출력 창이 있는 하단 패널을 열거나 닫기

Cursor 실행

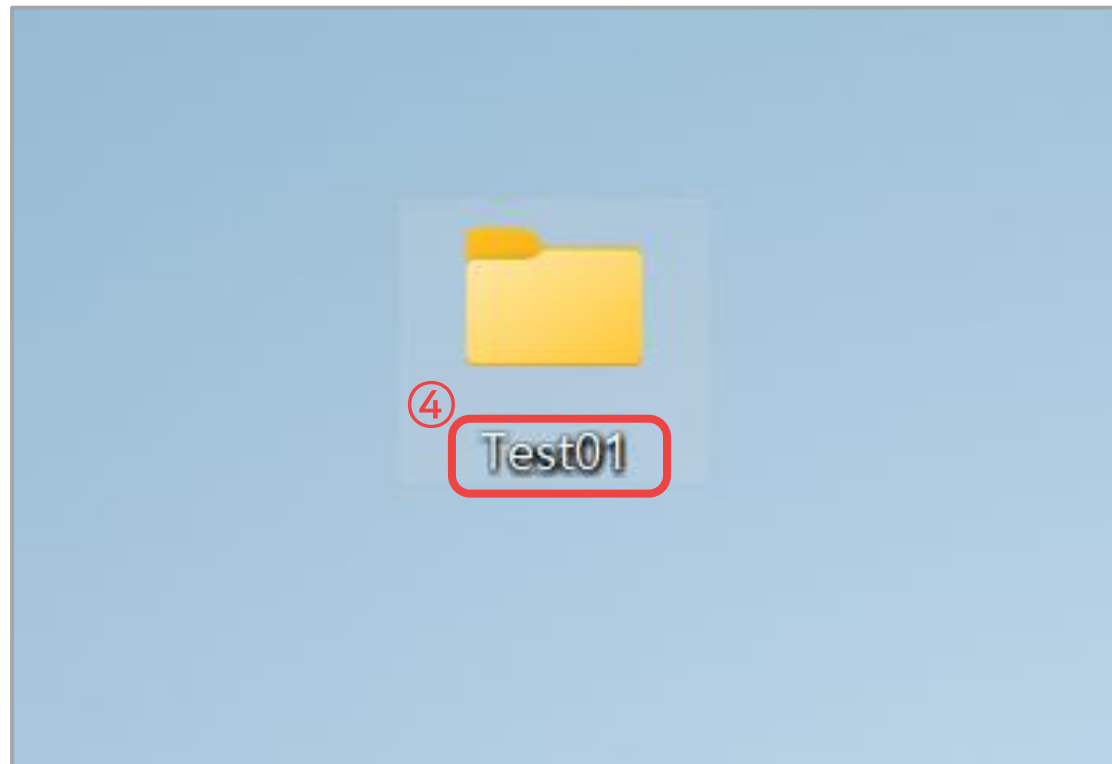
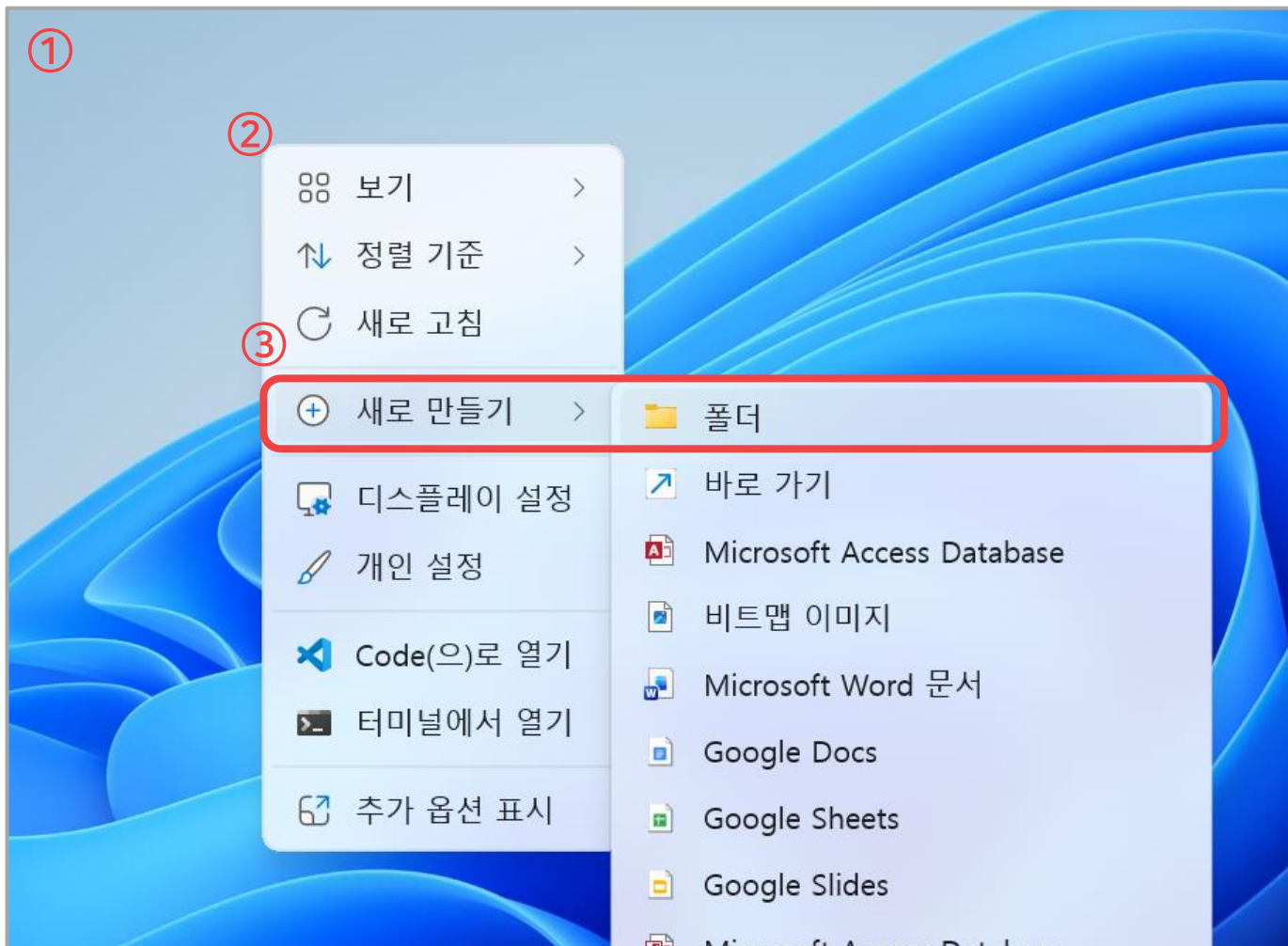
①Cursor 실행 → ②메인화면 확인 → ③토글로 레이아웃 변경 가능





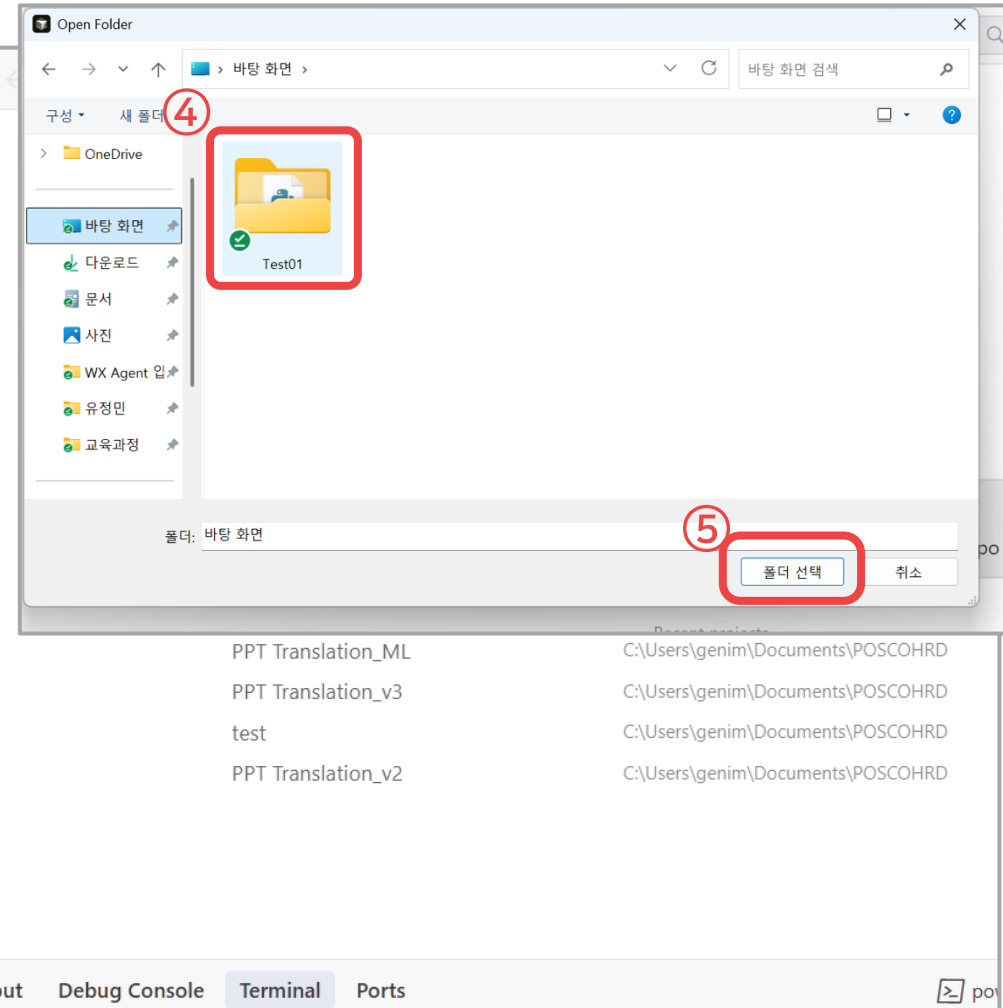
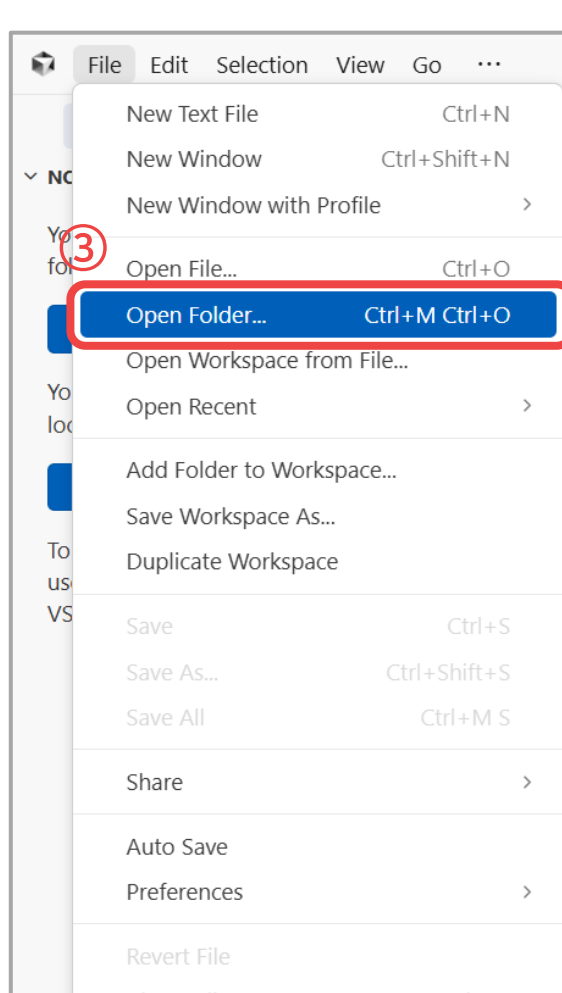
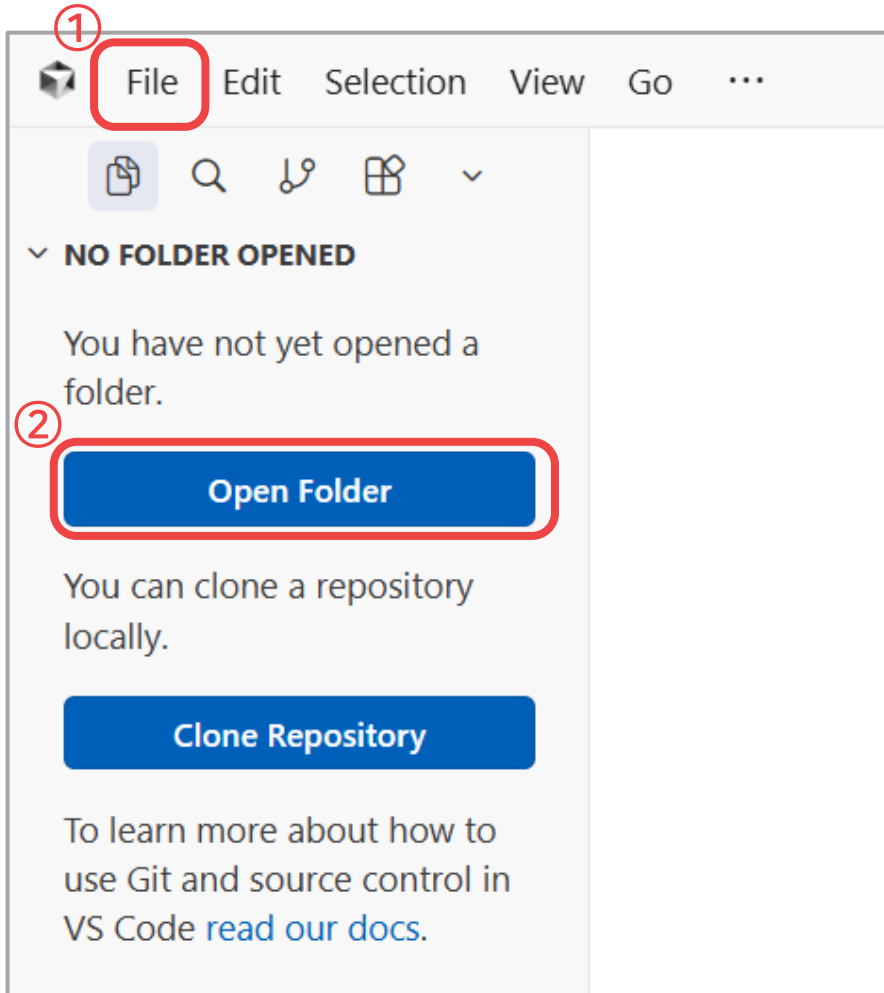
Cursor 시작 _ 바탕화면 실습 폴더 만들기

①바탕화면 접속 → ②마우스 오른쪽 클릭 → ③새로만들기 + 폴더 클릭 → ④파일명(Test01) 작성 → ⑤Enter



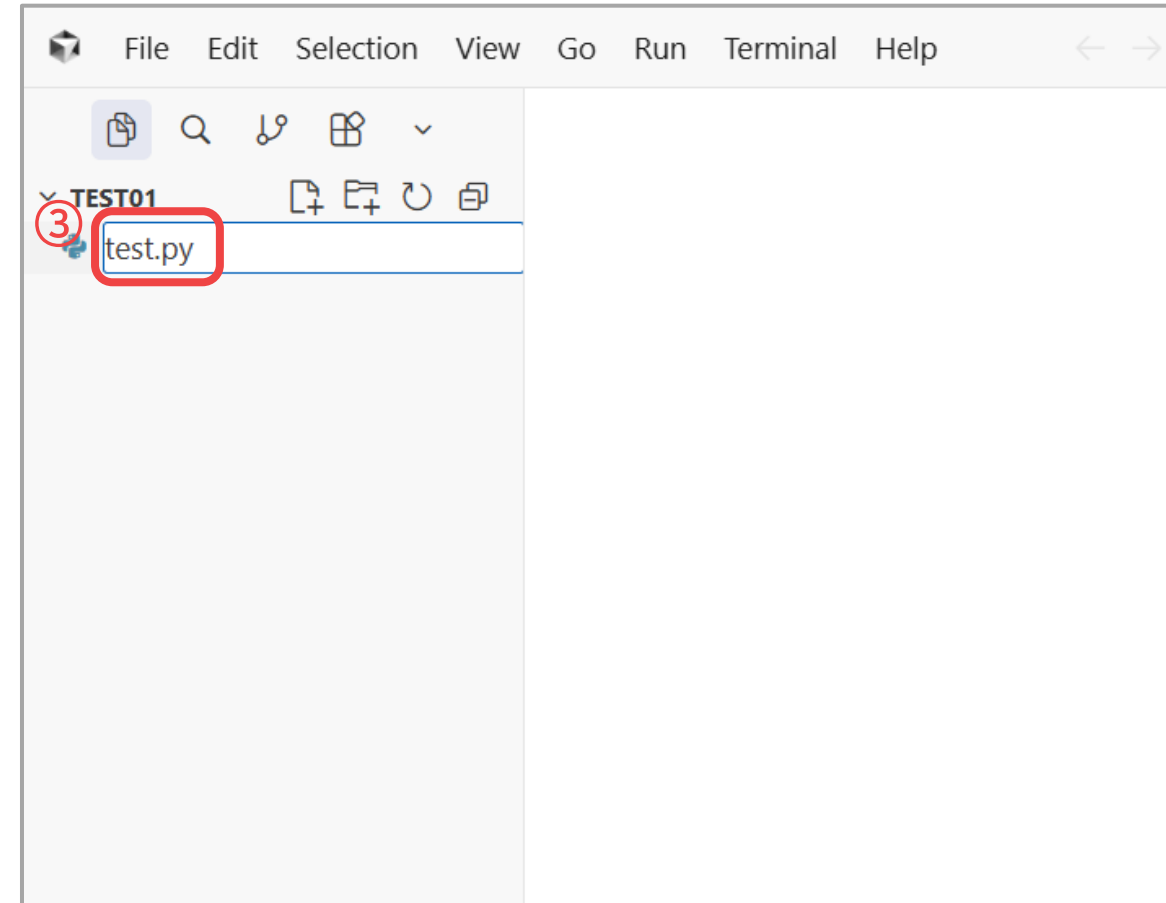
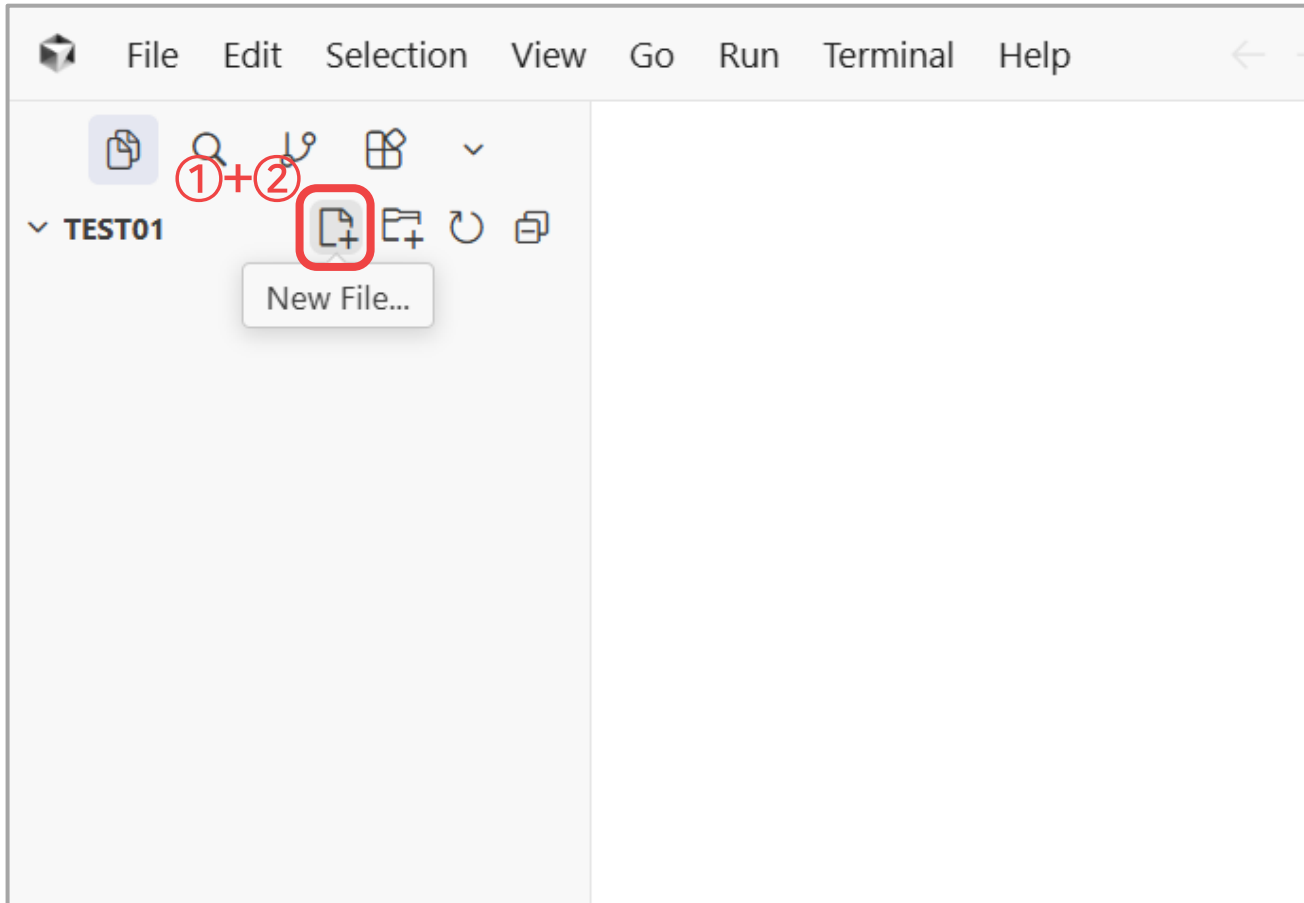
Cursor 실행 _ 폴더 열기

①File 클릭 → ②Open Folder 클릭 → ③ Open Folder... 클릭 → ④바탕화면-test01선택→ ⑤폴더 선택 클릭



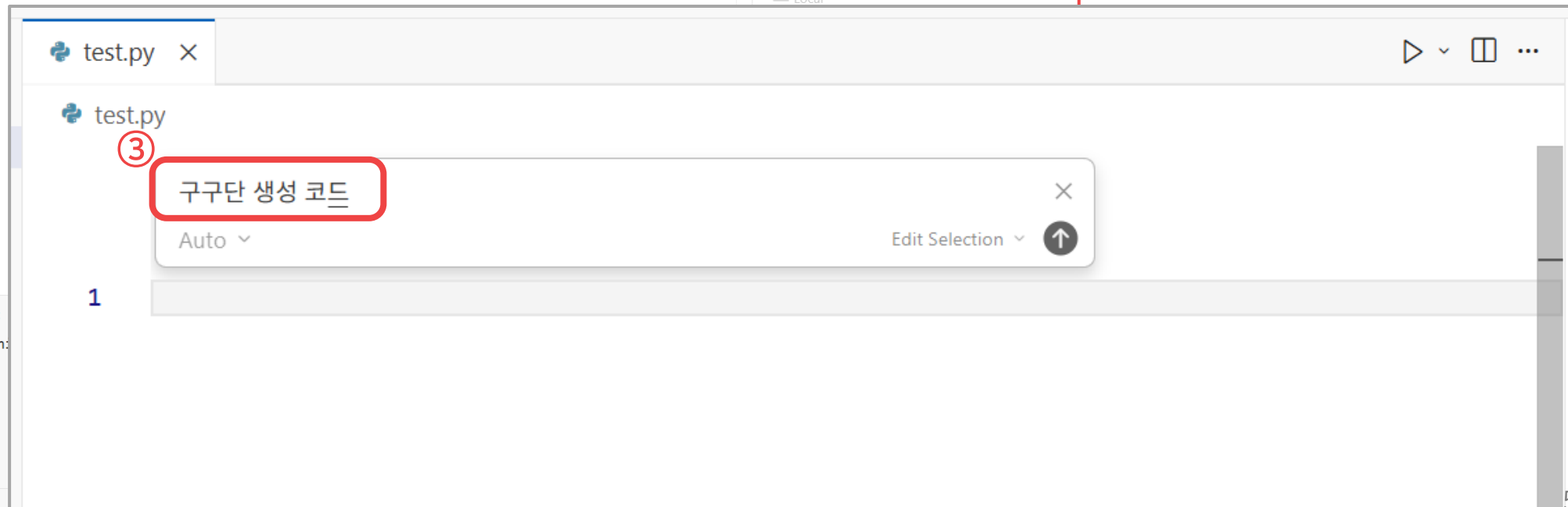
Cursor 실습 _ 실습파일 생성

①폴더명 옆에 **마우스 커서** 이동 → ②**문서+ icon** 클릭 → ③파일명 **test.py** 생성



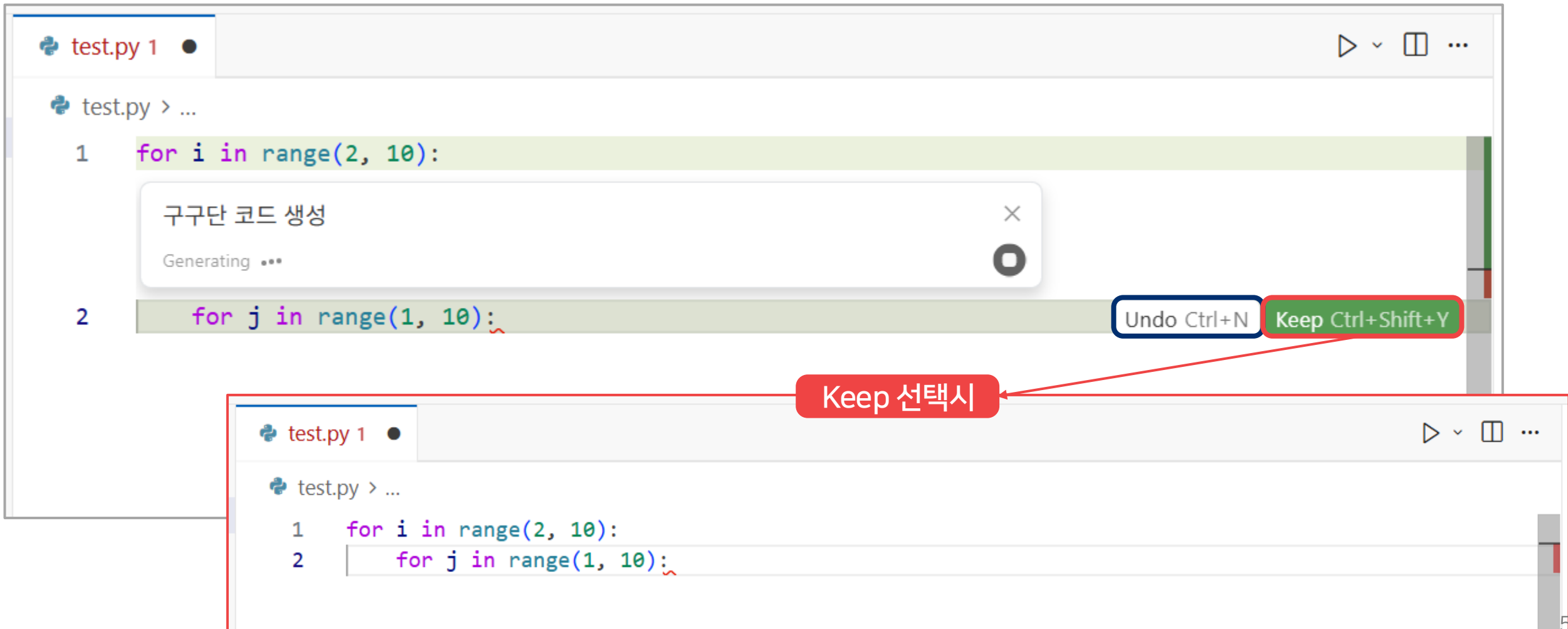
Cursor 실습 _ Ctrl+K

① 편집창 클릭 → ② Ctrl+K (인라인 편집 or 코드 생성 실행) → ③ 구구단 생성 코드 입력 → ④ Enter



Cursor 실습 _ Ctrl+K

Undo or Keep 클릭



Cursor 실습 _ AI Pane 활용

① 구구단 출력하는 코드 만들어줘 입력 → ② Enter → ③ Undo or Keep 클릭

The screenshot illustrates the workflow for generating code using the AI Pane in the Cursor IDE. It shows three overlapping windows:

- Top Left Window:** A 'New Chat' dialog with a text input field containing 'Plan, @ for context, / for commands' and a dropdown menu set to 'Sonnet 4.5'.
- Top Right Window:** A 'New Chat' dialog with a text input field containing '구구단 출력하는 코드 만들어줘' (Generate code for printing multiplication tables) and a red circle with the number '1' next to it. A red box highlights the input field and the text '구구단 출력하는 코드 만들어줘 ②Enter'.
- Bottom Window:** The main Cursor IDE interface. The 'test.py' file is open, showing a Python script for printing multiplication tables. A red box highlights the code, and a red arrow points from the 'Keep 선택시' (When selecting Keep) label to the 'Keep Ctrl+Shift+Y' button in the IDE's toolbar.

The AI Pane on the right side of the IDE shows the generated code and a thought process:

```
구구단 출력 코드
```

Now · +2 · Sonnet 4.5

구구단 출력하는 코드 만들어줘

```
for i in range(2, 10):
    print(f"\n[{i}단]")
    for j in range(1, 10):
        print(f"{i} x {j} = {i*j}")
```

Thought for 1s

구구단 출력 코드를 완성했습니다!

이 코드는 2단부터 9단까지 구구단을 출력합니다. 각 단의 제목을 표시하고, 곱셈 결과를 보기 좋게 출력합니다.

실행하려면 터미널에서 `python test.py`를 입력하시면 됩니다.

M2. 바이트코딩 활용 앱/웹 개발

4. 바이트코딩 기반 앱 개발

기획과 설계 프롬프트 작성

바이트 코딩 프롬프트는 Why와 What을 명확하게 하는 것, How는 LLM이 수행



【필요 역량】

도메인 지식 + Process분석 역량 + IT 지식



기획과 설계 프롬프트 작성

웹/앱 개발 프로세스: 웹/앱의 Why, What 생각 정리에서 프롬프트 설계까지



Why & What 생각정리

- 웹 서비스의 목적은 무엇인가?
- 보여줄 콘텐츠나 기능은 무엇인가?
- 메뉴는 어떻게 구성할 것인가?
- 메뉴별 콘텐츠 구성은 어떻게 할까?
- 데이터베이스 항목은?
- 개발 환경은 어떻게 할 것인가?

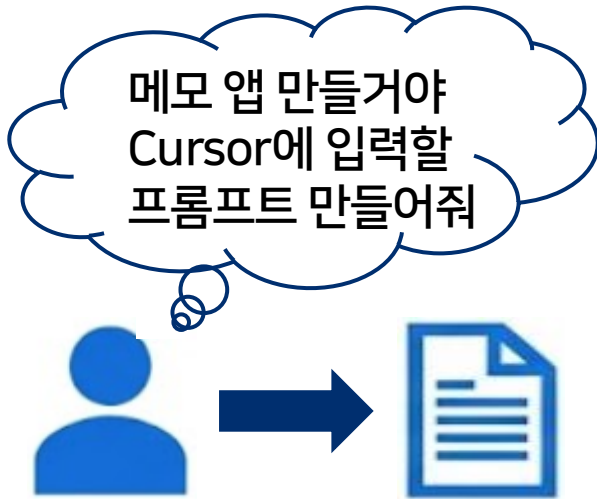


프롬프트 설계

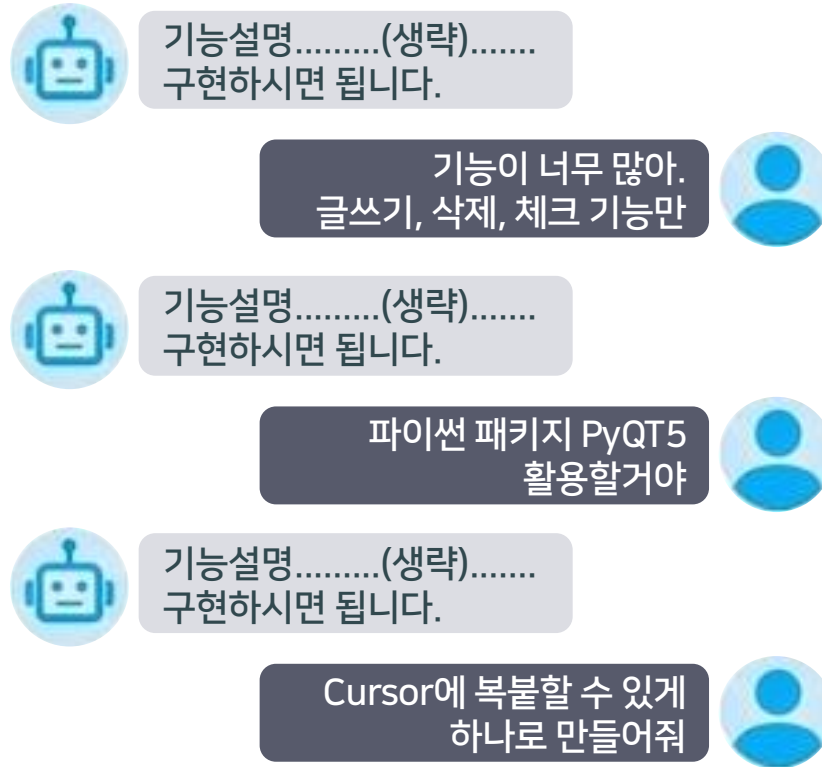
- 웹 서비스 목적 정의
- 개발 환경 및 패키지 개발환경
- 데이터 베이스 및 백엔드 프로세스 구성
- 메뉴별 콘텐츠, 백엔드 역할 노드설명
- 웹 페이지의 컬러, 폰트 구성 UI설명
- 초기 화면 구성, 메뉴 위치 및 기능

LLM활용 프롬프트 작성(Interactive Prompting)

1. 목표물만 제시 (Goal Input)



2. LLM과 대화하며 구체화 (Co-Creation with LLM)



3. 완성된 프롬프트 (Final Prompt)

Python과 PyQt5를 사용하여 세련된 디자인의 데스크톱 메모 앱을 만들어줘.

1. 기술 스택 및 구조:
GUI: PyQt5(직관적이고 모던한 UI)
데이터베이스: SQLite3
(메모 저장 및 불러오기)
스타일링: QSS(Qt Style Sheets)를 사용하여 다크 모드 테마 적용
(배경 #2e2e2e, 텍스트 #ffffff, 포인트 컬러 #0078d4)

2. 화면 레이아웃:
왼쪽 사이드바: 메모 목록(QListWidget)
....(생략)

Cursor에 붙여넣기

웹/앱/웹앱/하이브리드 앱

구분	특징	활용	설치여부
웹 (web)	- 네이버, 블로그, 뉴스 사이트 - URL로 접근하고, 콘텐츠를 읽고 탐색하는 "페이지" 중심	- 네이버, 블로그, 뉴스 사이트 - URL로 접근하고, 콘텐츠를 읽고 탐색하는 "페이지" 중심	❌ (URL만 있으면 됨)
앱 (App, Native App)	기기 안에 들어온 프로그램	- 카카오톡, 인스타그램, 게임 앱 - 설치 후 OS에서 직접 실행하고, 알림·카메라·GPS 등 기기 기능 활용	⊙ (앱스토어/플레이스토어)
웹앱 (Web App)	설치 안 해도 앱처럼 쓰는 서비스	- Gmail, Notion, Google Docs, EP - 로그인 후 작성·저장·수정하며, 새로고침 없이 화면이 계속 변함	❌ (기본은 브라우저) - 로그인 필요 ⊙
하이브리드 앱 (Hybrid App)	웹앱을 앱 껍데기 안에 넣은 것	- 쿠팡 일부 화면 등 - 앱스토어에 올라가지만 내부 일부 화면은 HTML/CSS/JS 웹앱으로 구성	

■ 구분 기준: “설치 여부 → 실행 위치 → 기능과 데이터 처리 → 기기 기능 활용” 순서로 판단

1. 설치해야 하면? → 앱

2. 설치 안 하는데 기능이 많고
로그인/데이터 처리? → 웹앱

3. 그냥 정보 보기 위주? → 웹

앱 개발을 위한 GUI 이해(PyQt5)

Qt5 어플리케이션 프레임워크에 대한 파이썬 버전, 플랫폼에 관계없이 다양한 기능을 포함하는 개발툴

특징	
주요 특징	내용
크로스 플랫폼	Windows, macOS, Linux 등 다양한 운영체제에서 동일한 코드 실행 가능
다양한 위젯	버튼, 입력창, 표, 트리 등 어플리케이션 구성에 필요한 UI 요소 기본 포함
시그널 & 슬롯	이벤트 발생(시그널)시 특정 함수(슬롯)가 호출 되도록 연결하는 PyQt의 핵심 메커니즘
Qt Designer 지원	GUI 디자인 도구로 U를 시각적으로 디자인하고, .ui 파일로 저장
강력한 멀티미디어	오디오, 비디오, 2D/3D 그래픽 기능까지 포함하여 다양한 기능의 어플리케이션 개발이 가능

구조



앱 개발 실습 절차

① 프롬프트 만들기

- ChatGPT or Gemini에서 Cursor AI에 요청할 프롬프트 만들기

② 실습 폴더 확인

- 배포된 실습 폴더 확인 (예: M2_04-1_pdf_split_merge)

③ Cursor 실행

- Cursor 실행, 실습 폴더 열기, 파이썬 패키지 설치(기존 설치 버전 유지)

④ New Chat 클릭

- 오른쪽 사이드바에서 New Chat 클릭, 프롬프트 입력

⑤ 진행사항 체크

- 모델 선택, 진행 여부(폴더 생성, 패키지 설치 등) 승인(Run) 클릭

Cursor

⑥ 앱 테스트

- 실습파일 업로드 후 앱 테스트, 이상 여부 확인

⑦ 오류 보완

- 오류 발생 화면 캡처, 오류발생 프롬프트 작성, 재실행

Cursor

⑧ 실행파일 생성

- 에러 해결 후, 실행파일로 만들기

앱 개발 실습1. PDF 분할/병합

▶ 실습목표: PDF 분할/병합 앱 만들기

PDF 분할 및 병합 도구

PDF 분할 PDF 병합

1. PDF 파일 선택

선택된 파일이 없습니다

파일 선택

2. 페이지 미리보기

3. 분할 옵션 선택

☒ 범위 입력 (예: 1-5, 10-15)

예: 1-5, 10-15

☐ 특정 페이지 입력 (예: 1, 3, 5)

예: 1, 3, 5, 7

☐ 고정 페이지 분할 (균등 분할)

페이지 수: 2 페이지씩 분할

분할 실행

1. 기능 : PDF 분할/병합
2. 메뉴 : 분할(범위 입력, 페이지 입력, 고정페이지 분할)
병합(파일 선택, 병합버튼)
3. 데이터베이스 : 없음
4. 개발환경 : PyQt5, PyPDF2

앱 개발 실습1. PDF 분할/병합

▶ 프롬프트 초안 : 카테고리 요청



아래 내용을 참고해서 개발환경, 노드(기능), UI로 구분해줘

=====

Python과 PyQt5를 사용해 간단한 PDF 분할/병합하는 앱을 만들거야

- PDF 업로드는 'pypdf'(또는 PyPDF2) 패키지 사용
- UI는 심플하고 직관적인 기본 수준의 PyQt5 인터페이스로 구현
- pdf 업로드 및 미리보기 기능(썸네일)
- 분할 기능 : 범위 입력, 페이지 입력, 고정페이지 분할
- 병합 기능 : 파일 선택, 병합버튼

앱 개발 실습1. PDF 분할/병합

① 프롬프트 만들기



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어 PDF 분할/병합하는 앱을 만들거야

1. 개발환경

- PDF 업로드는 'pypdf'(또는 PyPDF2) 패키지 사용

2. 노드설명

- pdf 업로드 기능
- 분할 기능 : 범위 입력, 페이지 입력, 고정페이지 분할
- 병합 기능 : 파일 선택, 병합버튼

3. UI설명

- PyQt5 인터페이스, 심플하고 직관적인 기본 수준
- pdf 썸네일 미리보기 UI
- 분할 : 범위 입력창, 페이지 입력창, 고정페이지 입력창
- 병합 : 파일 선택 버튼, 병합 버튼

앱 개발 실습1. PDF 분할/병합

① 프롬프트 만들기 : 완성_LLM이 만들어준 프롬프트

Prompt

너는 Python 데스크톱 애플리케이션 개발 전문가야. 아래 요구사항을 바탕으로 Python과 PyQt5를 활용한 'PDF 분할 및 병합 애플리케이션'의 전체 코드를 작성해 줘.

기술 스택

- GUI: PyQt5 (심플하고 직관적인 기본 수준의 UI 구성, 탭 방식 활용 권장)
- PDF 처리: 'pypdf' (또는 PyPDF2)
- PDF 미리보기(썸네일) 렌더링: 'pypdf'로는 썸네일 이미지 추출이 어려우므로 'PyMuPDF(fitz)' 또는 'pdf2image' 중 가볍고 구현이 쉬운 것을 선택해서 적용해 줘.

핵심 기능 요구사항

1. 분할 (Split) 기능

- PDF 파일 업로드: 단일 PDF 파일을 선택하여 업로드할 수 있어야 함.
- 썸네일 미리보기: 업로드된 PDF의 각 페이지를 썸네일 형태로 미리 볼 수 있는 UI를 제공해.
- 분할 옵션 선택:
 - 1) 범위 입력 (예: 1-5)
 - 2) 특정 페이지 입력 (예: 1, 3, 5)
 - 3) 고정 페이지 분할 (예: 2페이지씩 균등 분할)
- 중요 분할 조건: 사용자가 지정한 분할 범위에 포함되지 않은 '나머지 페이지'들도 버려지지 않고 따로 분할되어 저장되어야 해.
- 저장 위치: 원본 PDF가 있는 경로에 새로운 폴더를 생성하여 저장해.
 - * 새 폴더명: '원제목_{분할된 총 파일 개수}' (예: document_3)
- 분할된 파일명 형식: '원제목_{분할된 페이지 정보}.pdf' (예: document_1-5.pdf, document_6-10.pdf 등)

2. 병합 (Merge) 기능

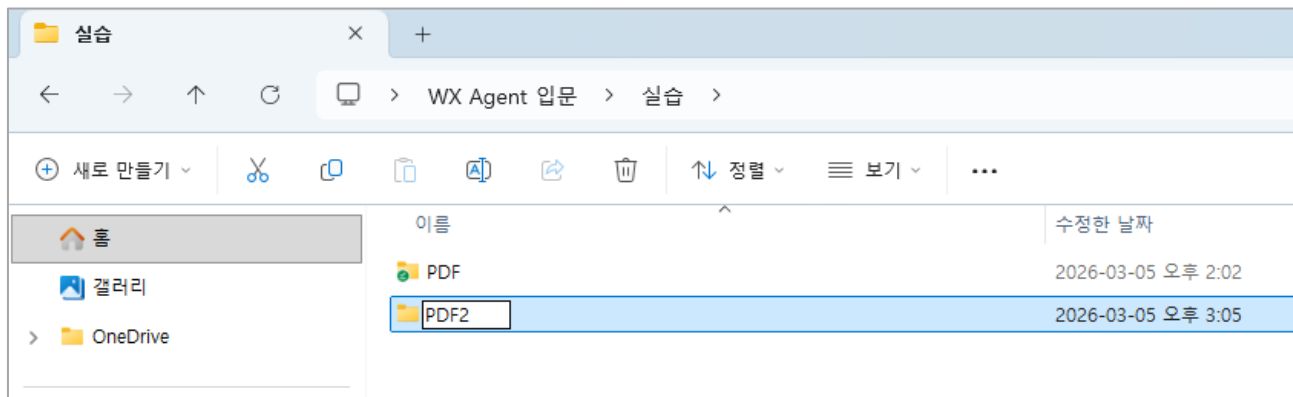
- PDF 다중 업로드: 여러 개의 PDF 파일을 한꺼번에 선택하고 업로드할 수 있어야 해 (목록 형태로 표시).
- 병합 실행: 업로드된 파일들을 하나의 PDF로 병합해.
- 병합 완료 미리보기: 병합이 완료되면, 완성된 PDF의 첫 페이지나 전체 썸네일을 미리보기로 보여주는 기능을 제공해.

추가 지시사항

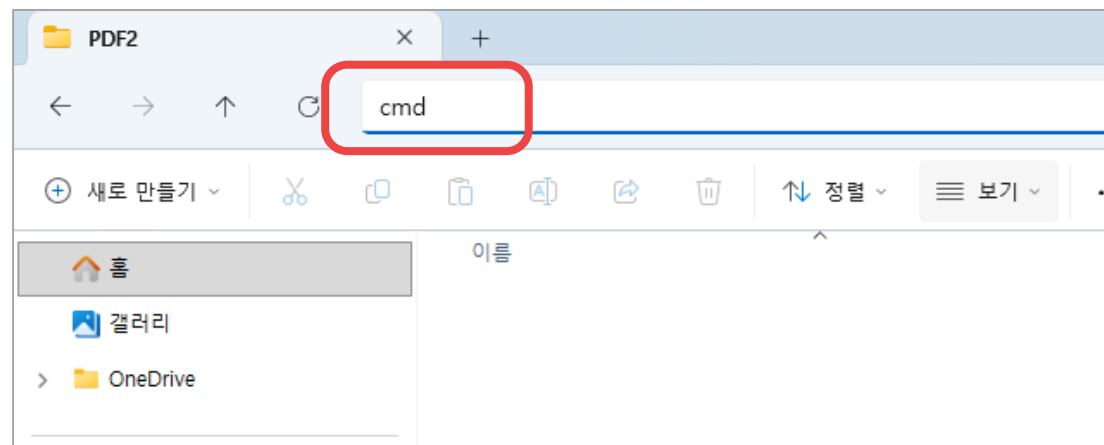
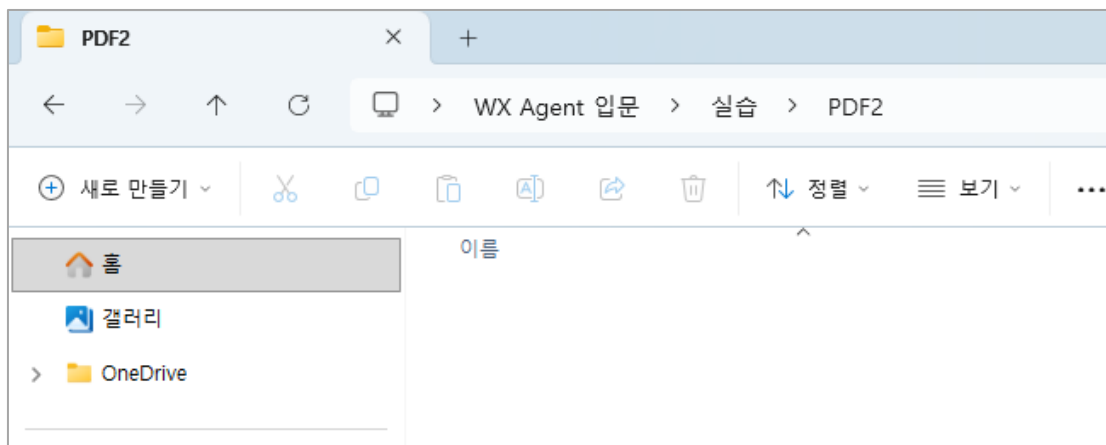
- 코드는 모듈화하여 가독성 있게 작성하고, 각 주요 클래스나 함수에 한국어로 주석을 달아줘.
- requirements.txt에 필요한 패키지 목록을 함께 출력해 줘. 이미 설치된 패키지는 설치된 버전 그대로 적고 업데이트하지 마.
- 에러 처리(예: 잘못된 페이지 범위 입력, 권한 부족 등)를 꼼꼼하게 적용해서 앱이 튕기지 않도록 해.

앱 개발 실습1. PDF 분할/병합

② 실습 폴더 확인 : 실습 - 「M2_04-1_pdf_split_merge」 폴더

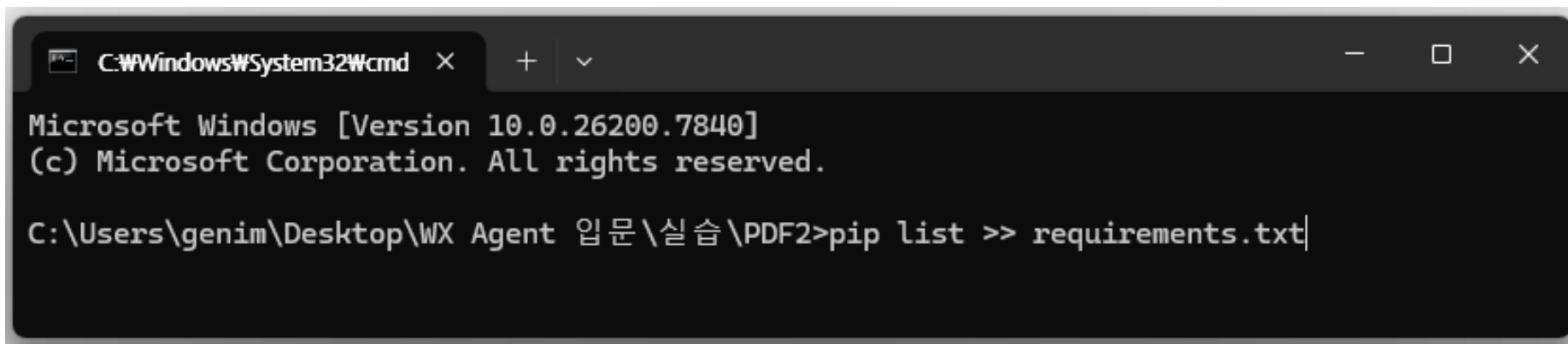


③ 개발 환경 설정 : 「M2_04-1_pdf_split_merge」 이동 → 주소창 cmd → Enter



앱 개발 실습1. PDF 분할/병합

③ 개발 환경 설정 : `pip list >> requirements.txt`



```
C:\Windows\System32\cmd
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\genim\Desktop\WX Agent 입문\실습\PDF2>pip list >> requirements.txt
```

③ 개발 환경 설정 : Cursor AI 실행 → File-Open Folder-실습-M2_04-1_pdf_split_merge 폴더 클릭



앱 개발 실습1. PDF 분할/병합

④ New Chat 클릭 : 프롬프트 입력 → 실행

Prompt

너는 Python 데스크톱 애플리케이션 개발 전문가야. 아래 요구사항을 바탕으로 Python과 PyQt5를 활용한 'PDF 분할 및 병합 애플리케이션'의 전체 코드를 작성해 줘.

기술 스택

- GUI: PyQt5 (심플하고 직관적인 기본 수준의 UI 구성, 탭 방식 활용 권장)
- PDF 처리: `pypdf` (또는 PyPDF2)
- PDF 미리보기(썸네일) 렌더링: `pypdf`로는 썸네일 이미지 추출이 어려우므로 `PyMuPDF(fitz)` 또는 `pdf2image` 중 가볍고 구현이 쉬운 것을 선택해서 적용해 줘.

핵심 기능 요구사항

1. 분할 (Split) 기능

- PDF 파일 업로드: 단일 PDF 파일을 선택하여 업로드할 수 있어야 함.
- 썸네일 미리보기: 업로드된 PDF의 각 페이지를 썸네일 형태로 미리 볼 수 있는 UI를 제공해.
- 분할 옵션 선택:
 - 1) 범위 입력 (예: 1-5)
 - 2) 특정 페이지 입력 (예: 1, 3, 5)
 - 3) 고정 페이지 분할 (예: 2페이지씩 균등 분할)
- **중요 분할 조건** : 사용자가 지정한 분할 범위에 포함되지 않은 '나머지 페이지'들도 버려지지 않고 따로 분할되어 저장되어야 해.
- 저장 위치: 원본 PDF가 있는 경로에 새로운 폴더를 생성하여 저장해.
- * 새 폴더명: `원제목\_{분할된 총 파일 갯수}` (예: document_3)
- 분할된 파일명 형식: `원제목\_{분할된 페이지 정보}.pdf` (예: document_1-5.pdf, document_6-10.pdf 등)

2. 병합 (Merge) 기능

- PDF 다중 업로드: 여러 개의 PDF 파일을 한꺼번에 선택하고 업로드할 수 있어야 해 (목록 형태로 표시).
- 병합 실행: 업로드된 파일들을 하나의 PDF로 병합해.
- 병합 완료 미리보기: 병합이 완료되면, 완성된 PDF의 첫 페이지나 전체 썸네일을 미리보기로 보여주는 기능을 제공해.

추가 지시사항

- 코드는 모듈화하여 가독성 있게 작성하고, 각 주요 클래스나 함수에 한국어로 주석을 달아줘.
- requirements.txt에 필요한 패키지 목록을 함께 출력해 줘.
- 에러 처리(예: 잘못된 페이지 범위 입력, 권한 부족 등)를 꼼꼼하게 적용해서 앱이 튼튼하게 작동하도록 해.

New Chat

Plan, @ for context, / for commands

Agent

Sonnet 4.5

- New Agent Ctrl + Shift + L
- Show Terminal Ctrl + J
- Hide Files Ctrl + B
- Search Files Ctrl + P
- Open Browser Ctrl + Shift + B
- Maximize Chat Ctrl + Alt + E

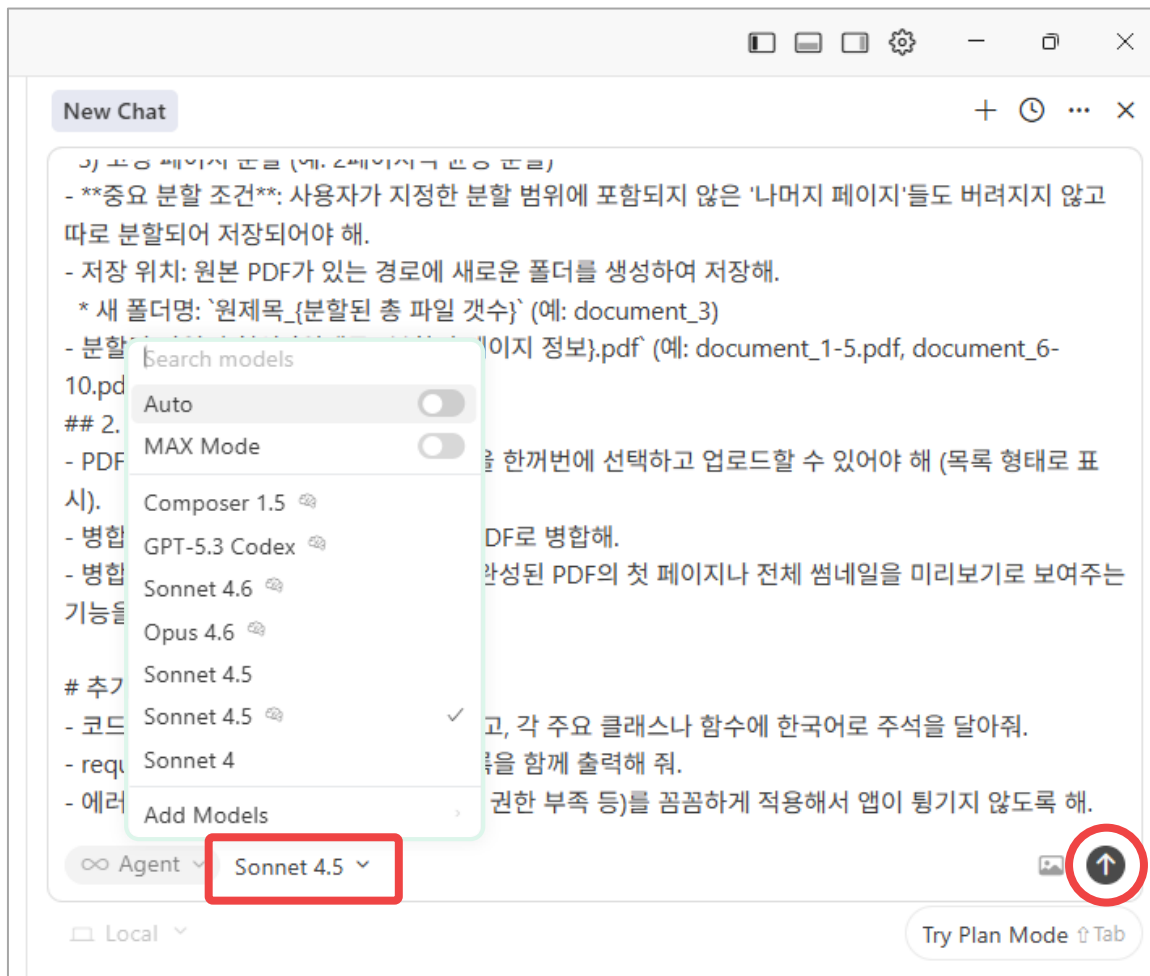
Past Chats

New Chat

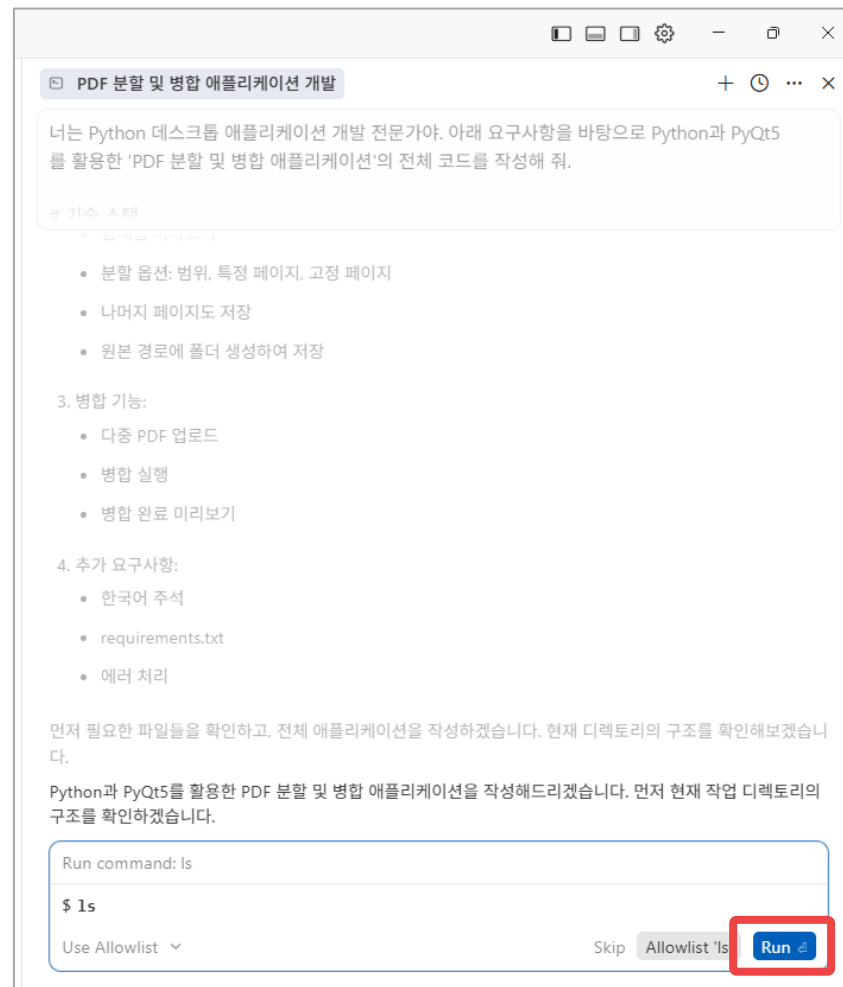
Now

앱 개발 실습1. PDF 분할/병합

④ New Chat 클릭 : 모델 선택 → 실행 클릭



⑤ 진행사항 승인 체크 : Run 클릭



앱 개발 실습1. PDF 분할/병합

⑥ 앱테스트 : 터미널 창 열기 → python pdf_tool_app.py 입력 → Enter

The image shows the PDF2 application interface with a terminal window open. The terminal displays the command `python pdf_tool_app.py` being executed. The application window has a sidebar with a file explorer showing `pdf_tool_app.py`, `README.md`, and `requirements.txt`. The main area shows the application's features and instructions. A red box highlights the terminal window, and a red dashed line connects it to the command input field in the terminal.

PDF 분할 및 병합 애플리케이션 개발

너는 Python 데스크톱 애플리케이션 개발 전문가야. 아래 요구사항을 바탕으로 Python과 PyQt5를 활용한 'PDF 분할 및 병합 애플리케이션'의 전체 코드를 작성해 줘.

기술 스택

- 파일명: 원제목_{페이지범위}.pdf 형식

병합 (Merge) 기능

- 다중 PDF 파일 선택
- 파일 순서 조정 (위/아래 이동, 제거)
- 병합 완료 후 썸네일 미리보기 (최대 5페이지)
- 사용자 지정 경로에 저장

에러 처리

- 잘못된 페이지 범위/번호 입력 검증
- PDF 로드 실패 처리
- 파일 저장 권한 오류 처리
- 사용자 친화적인 오류 메시지

실행 방법

패키지가 이미 설치되어 있으므로 바로 실행 가능

```
python pdf_tool_app.py
```

애플리케이션은 탭 방식으로 구성되어 있어 분할과 병합 기능을 쉽게 전환할 수 있습니다. 모든 주요 함수에 한국어 주석이 포함되어 있으며, PyMuPDF(fitz)를 사용하여 가볍고 빠른 썸네일 렌더링을 지원합니다.

PDF2

File Edit Selection View Go Run Terminal Help

PDF2

pdf_tool_app.py
README.md
requirements.txt

New Agent Ctrl + Shift + L
Hide Terminal Ctrl + J
Hide Files Ctrl + B
Search Files Ctrl + P
Open Browser Ctrl + Shift + B
Maximize Chat Ctrl + Alt + E

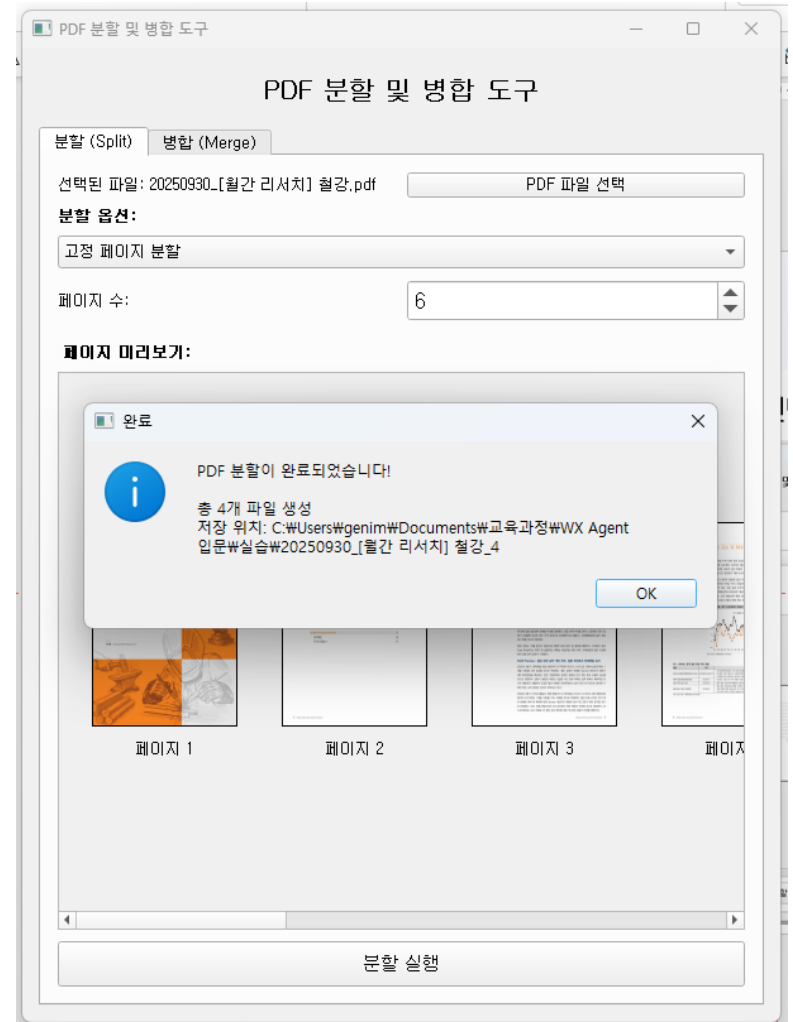
Problems Output Debug Console **Terminal** Ports

PS C:\Users\genim\Desktop\WX Agent 입문\실습\PDF2> python pdf_tool_app.py

python pdf_tool_app.py

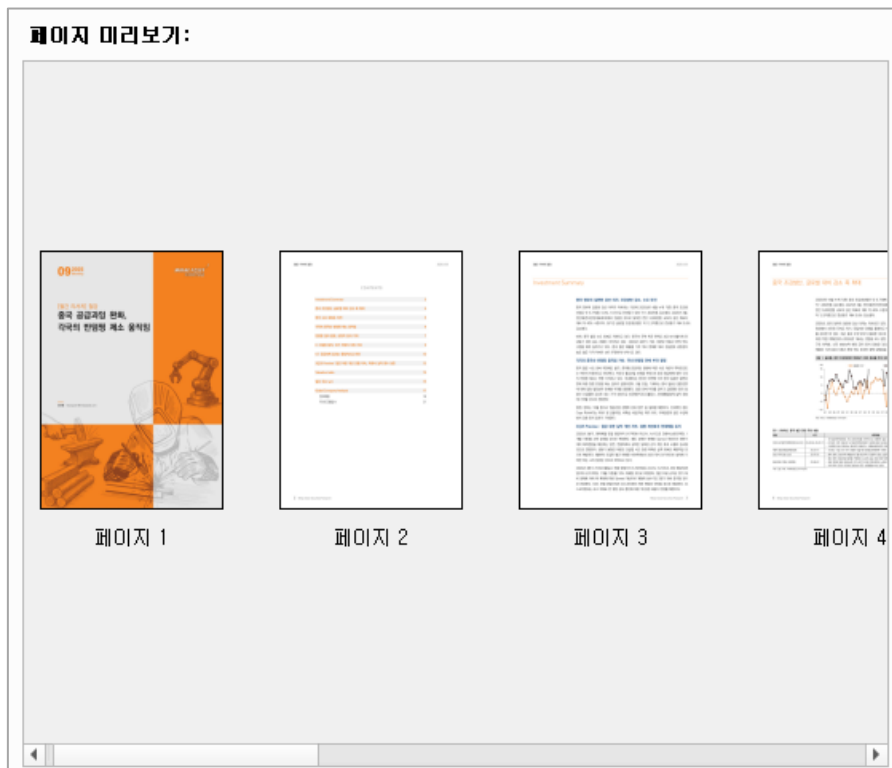
앱 개발 실습1. PDF 분할/병합

⑥ 앱테스트 : '20250930_[월간 리서치] 철강'파일 선택 → 분할 옵션 선택 → 분할 실행 클릭

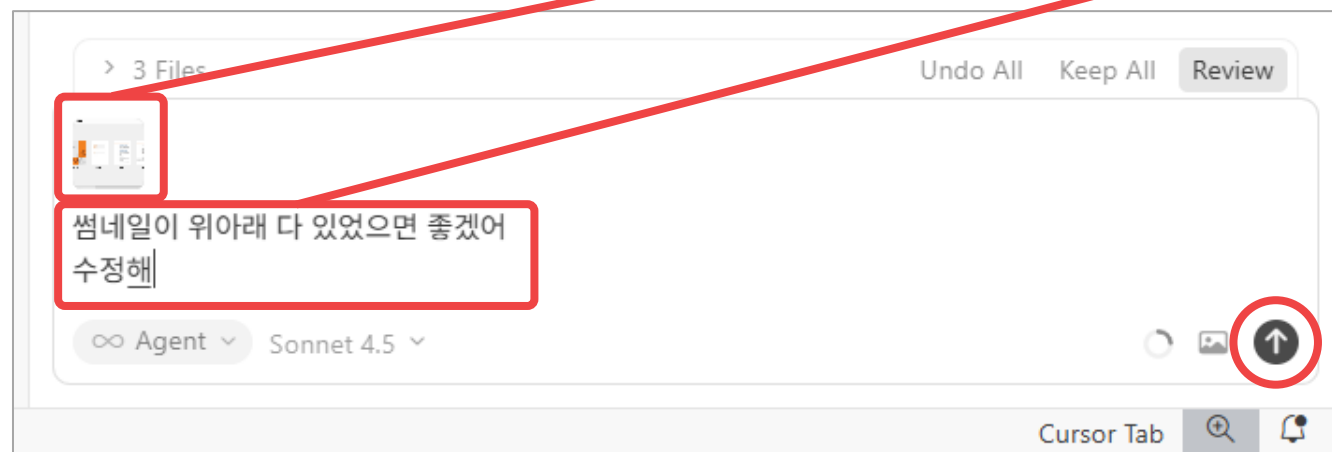


앱 개발 실습1. PDF 분할/병합

⑦ 오류 보완 : 오류 발생한 화면 또는 수정하고 싶은 화면 캡처(window + shift + s) → 앱 종료



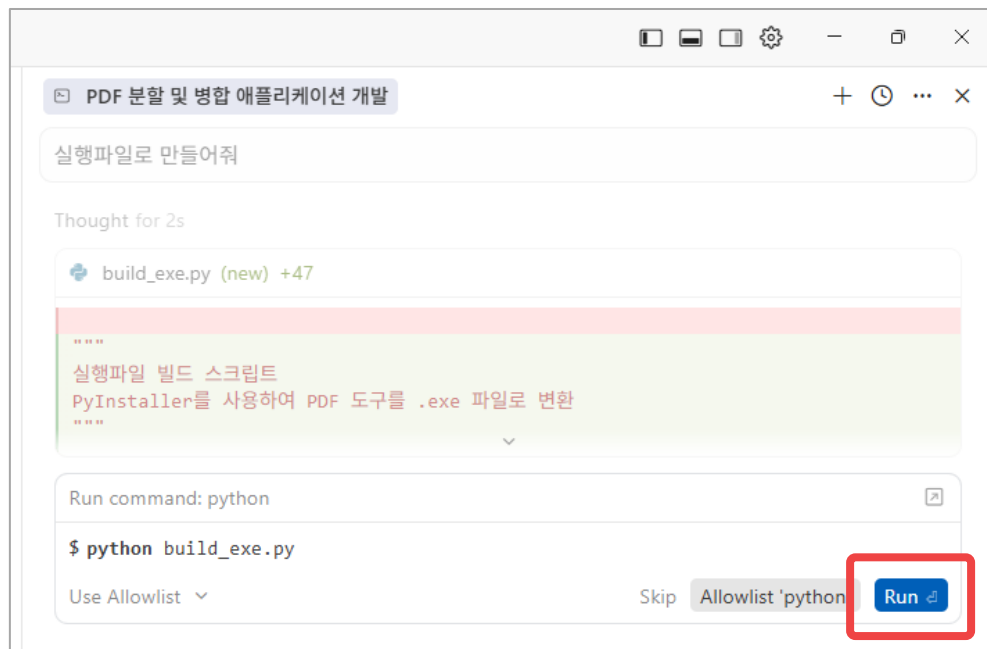
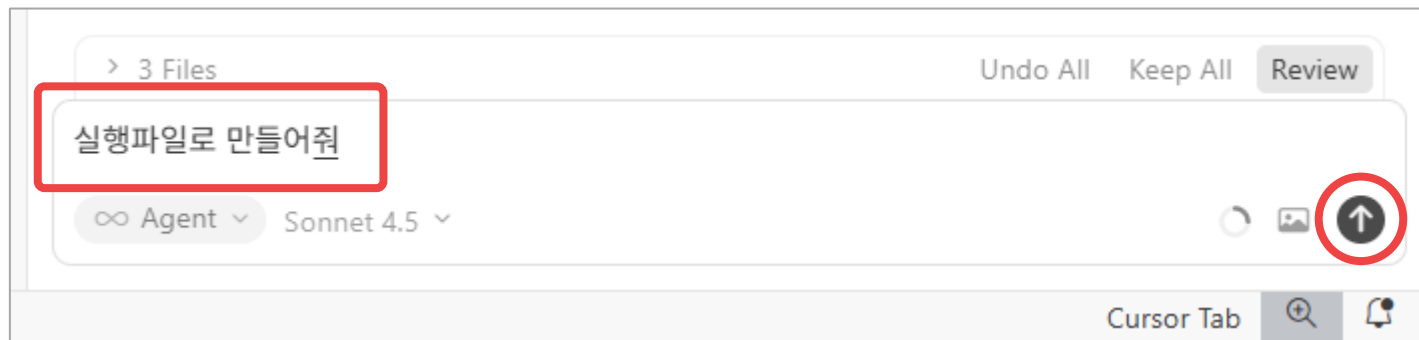
→ cursor AI 대화창에 캡처 이미지 붙여 넣고 보완 사항 입력



※ 수정 완료 후 앱 다시 실행(⑥앱테스트 슬라이드 참고)

앱 개발 실습1. PDF 분할/병합

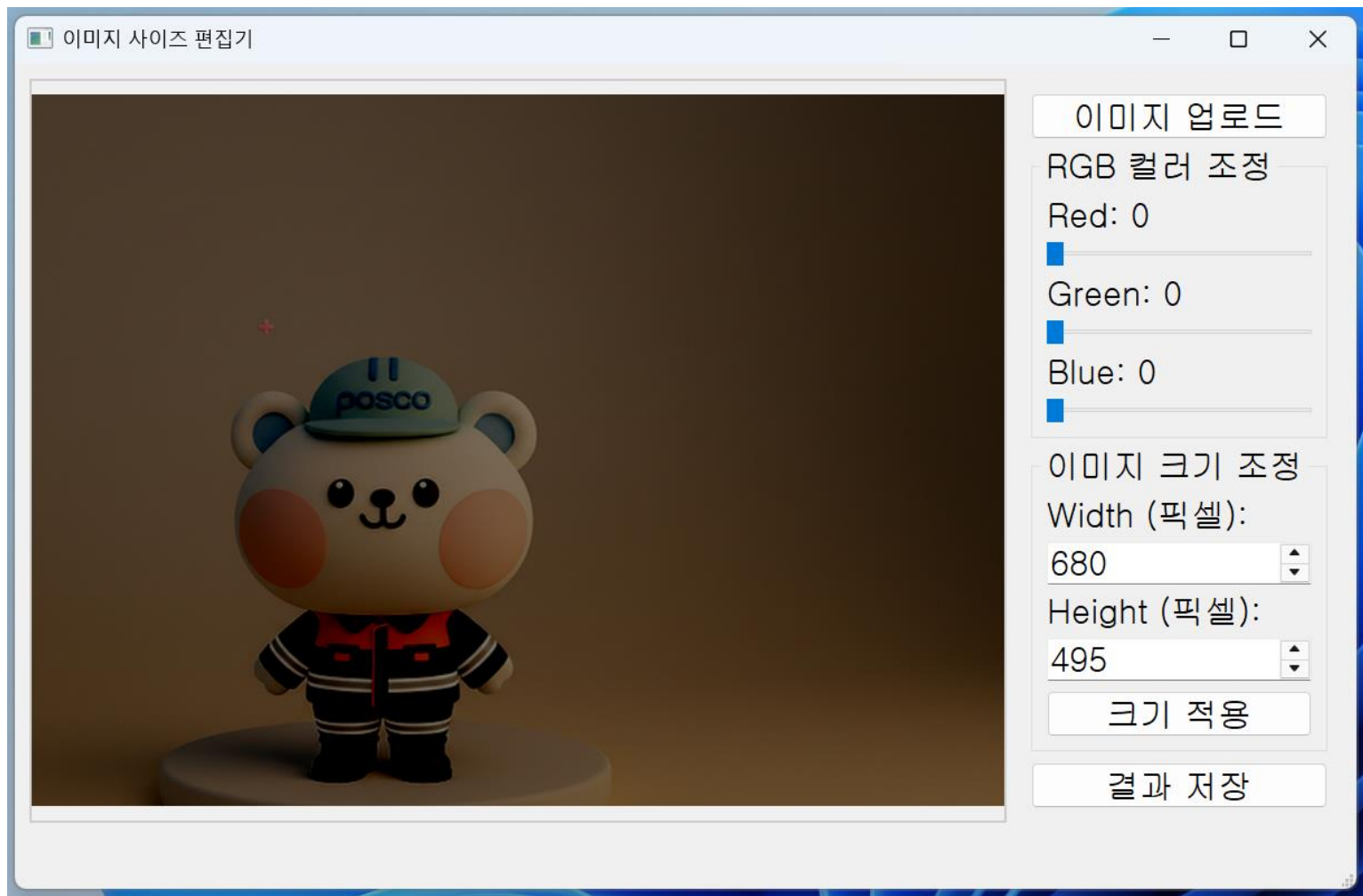
⑧ 실행파일 생성 : cursor AI 대화창에 '실행파일로 만들어줘' 입력 후 실행 → Run 클릭



⑨ dist 폴더에 실행파일(*.exe) 확인

앱 개발 실습2. 이미지 사이즈 및 컬러 조정하기

▶ 실습목표: 이미지 사이즈 및 컬러 조정하기



1. 기능 : 이미지업로드, 색상 조정, 사이즈 조정
2. 메뉴 : 이미지 업로드, RGB Bar, 저장
3. 데이터베이스 : 필요없음
4. 개발환경 : PyQt5

앱 개발 실습2. 이미지 사이즈 및 컬러 조정하기

▶ 프롬프트 초안 : 카테고리 만들기



아래 내용을 참고해서 개발환경, 노드(기능), UI로 구분해줘

=====

이미지를 업로드하면 사이즈 및 컬러 조정하는 앱을 만들거야

- pyqt5 활용
- 여러 이미지 업로드
- 컬러조정 : RGB 바를 이용해 실시간으로 이미지 컬러 조정, 버튼으로 흑백 변경
- 크기 조정 : 이미지 사이즈를 x,y 픽셀을 입력하여 크기 조정,
가로 또는 세로 사이즈 조정시 다른 사이즈 자동 설정,
하나의 이미지 사이즈 설정시 다른 이미지도 동일한 사이즈로 일괄 적용
- 결과 저장 : 수정된 결과를 jpg, png, gif 등으로 저장

앱 개발 실습2. 이미지 사이즈 및 컬러 조정하기

▶ 프롬프트 만들기



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.
이미지 사이즈 및 컬러 조정하는 앱을 만들거야

1. 개발환경

- PyQt5 활용

2. 노드설명

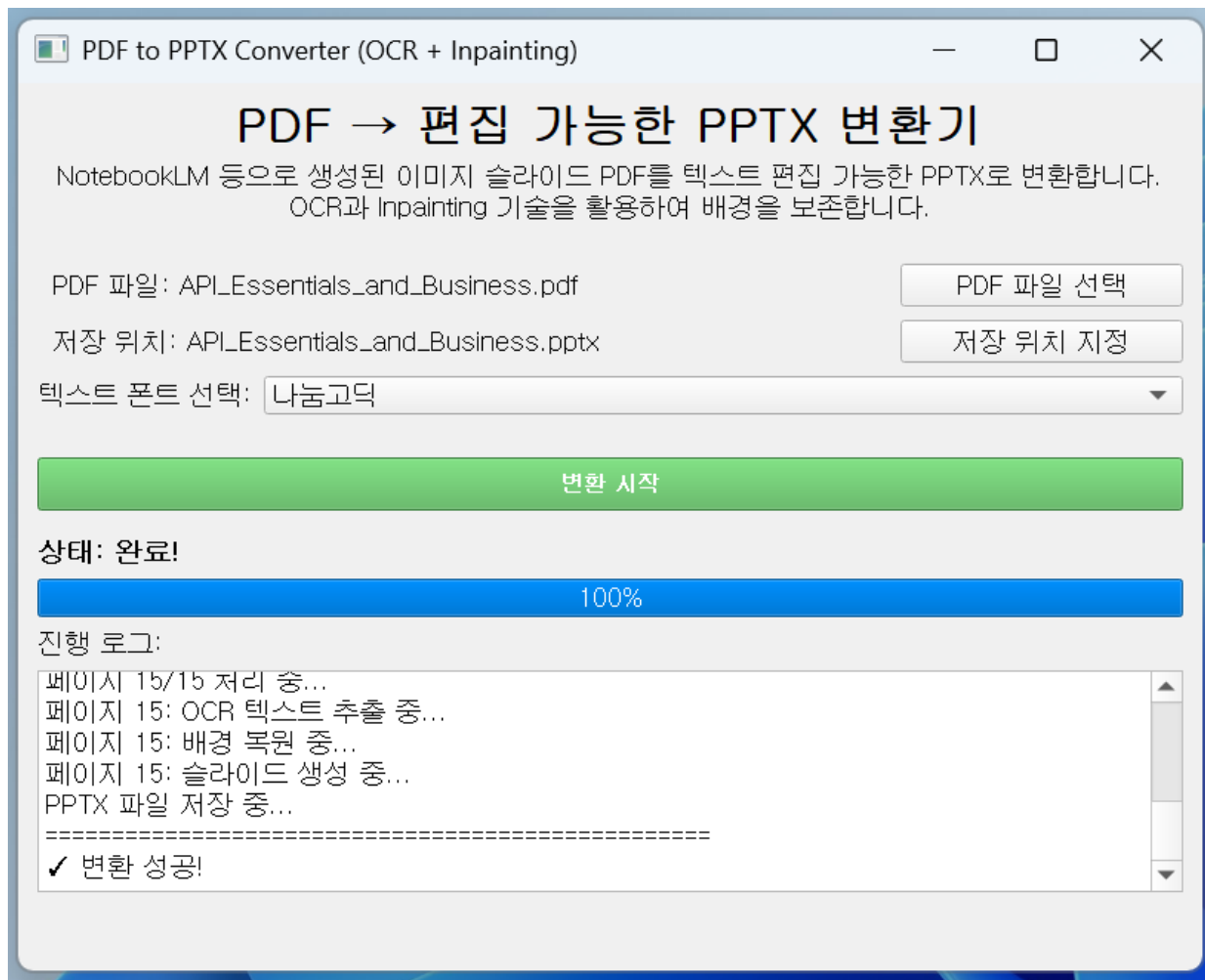
- 이미지 업로드 (다중 업로드 지원)
- 하나의 이미지를 수정하면 동일한 기준으로 다른 이미지에도 일괄 적용
- 컬러 조정: RGB 바를 이용해 실시간으로 이미지 컬러 조정
- 크기 조정: x사이즈 입력 시 비율에 맞춰 y사이즈 자동 조정
- 흑백 변환 기능
- 결과 저장: 수정된 결과를 jpg, png, gif 등으로 저장

3. UI설명

- 다중 이미지 업로드 UI
- RGB 슬라이더 바 (실시간 컬러 조정)
- x 픽셀 입력창 / y사이즈 자동 계산 표시
- 흑백 변환 버튼
- 저장 형식 선택 (jpg, png, gif 등)

앱 개발 실습3. 노트북lm 슬라이드 PPTX로 변환

▶ 실습목표: 노트북lm 슬라이드 PPTX로 변환



1. 기능 : PDF 파일 등록, PPTX 파일 저장
2. 메뉴 : 파일 업로드, 변환버튼, 진행률, 저장
3. 데이터베이스 : 없음
4. 개발환경 : PyQt5, EasyOCR

앱 개발 실습3. 노트북lm 슬라이드 PPTX로 변환

▶ 프롬프트 초안 : 카테고리 만들기



아래 내용을 참고해서 개발환경, 노드(기능), UI로 구분해줘

=====

구글 노트북 LM으로 작성한 이미지 슬라이드를 편집가능한 PPTX로 변환하는 앱을 만들거야

- 파이썬 PyQt5 패키지를 이용해
- 나에게 없는 패키지는 니가 설치해줘
- ocr 패키지는 EasyOCR을 이용해
- 원본도 16:9 크기고, 만들려고 하는 pptx도 16:9 크기야
- 우선 PDF파일을 읽어오는 메뉴만들고, 저장 파일명 및 경로를 지정하는 메뉴도 만들어줘
- pptx 파일명은 업로드한 pdf파일명과 동일하게 자동 설정해.
- 글꼴은 맑은 고딕과 나눔고딕이 있던데, 나눔스퀘어 Bold체도 추가해.
- 변환을 누르면 슬라이드를 하나씩 읽어서 슬라이드에 있는 텍스트를 OCR로 읽어줘, 텍스트는 한글, 영어 등으로 되어 있어
- 기존 텍스트가 있던 위치의 이미지 글자를 배경과 동일하게 만들어줘(텍스트를 삭제)
- 그 다음 그위치에 추출한 텍스트를 원래 크기 그대로 삽입해줘
- 이미지 등 다른 레이아웃은 절대로 변경하지마,
- 좌표 정밀도: 이미지의 해상도(px)와 PPT의 물리적 크기(inch)는 다르기 때문에 비율 계산(Ratio)이 중요해:
 . PPT_Width = 10 inches, PPT_Height = 5.625 inches (16:9 기준)
 . 기본 폰트로 나눔고딕, 맑은 고딕, 나눔스퀘어 Bold를 선택하게 해줘

앱 개발 실습3. 노트북lm 슬라이드 PPTX로 변환

▶ 프롬프트 만들기



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.

구글 노트북 LM으로 작성한 이미지 슬라이드를 편집가능한 PPTX로 변환하는 앱을 만들어줘

1. 개발환경

- 파이썬 PyQt5 패키지를 이용
- 나에게 없는 패키지는 설치
- OCR 패키지: EasyOCR
- 원본 16:9, 출력 PPTX도 16:9
- PPT_Width = 10 inches, PPT_Height = 5.625 inches (16:9 기준)
- 좌표 정밀도: 이미지 해상도(px)와 PPT 물리적 크기(inch) 비율 계산(Ratio) 적용

2. 노드설명

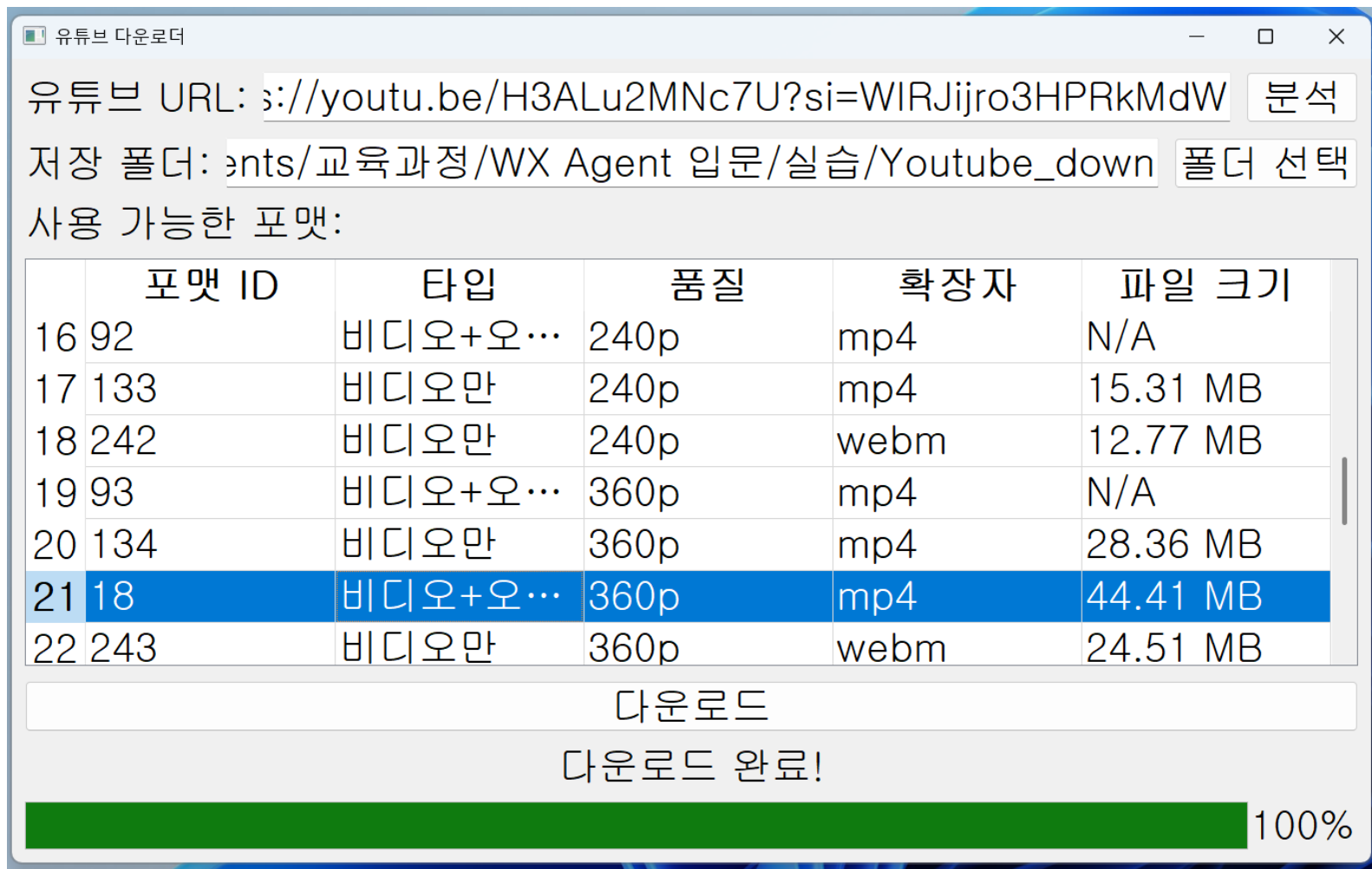
- PDF 파일 읽기
- 저장 파일명 및 경로 지정
- PPTX 파일명은 업로드한 PDF 파일명과 동일하게 자동 설정
- 변환 실행 시 슬라이드를 하나씩 읽어서 OCR로 텍스트 추출 (한글, 영어 등)
- 기존 텍스트가 있던 위치의 이미지 글자를 배경과 동일하게 처리 (텍스트 삭제)
- 추출한 텍스트를 원래 위치 및 크기 그대로 삽입
- 이미지 등 다른 레이아웃은 변경하지 않음

3. UI설명

- PDF 파일 불러오기 메뉴
- 저장 경로 및 파일명 지정 메뉴
- 기본 폰트 선택: 나눔고딕, 맑은 고딕, 나눔스퀘어 Bold
- 변환 실행 버튼

앱 개발 추가 예제. 유튜브 영상 다운로드

▶ 실습목표: 유튜브 영상 링크를 입력하면 해당 영상을 다운로드



1. 기능 : 유튜브 다운로드
2. 메뉴 : 링크주소 입력, 항목 선택
3. 데이터베이스 : 없음
4. 개발환경 : PyQt5, yt-dlp

앱 개발 추가 예제. 유튜브 영상 다운로드

▶ 프롬프트 초안 : 카테고리 만들기



아래 내용을 참고해서 개발환경, 노드(기능), UI로 구분해줘

=====

유튜브 영상 링크를 붙여넣기 하면 해당 영상을 다운로드 하는 앱을 만들거야

- Python으로 YouTube 링크를 입력하면 영상 또는 음원을 다운로드할 수 있는 GUI 앱을 만들어줘
- GUI 프레임워크는 PyQt5 사용
- 영상/음원 정보 분석 및 다운로드는 yt-dlp 패키지 사용
- 사용자가 유튜브 링크 입력, 다운로드 폴더 선택
- "분석" 버튼 클릭 → yt-dlp로 해당 링크 분석 후 가능한 다운로드 포맷 리스트 출력
- 사용자가 포맷 선택
- "다운로드" 버튼 클릭 → 선택한 포맷으로 영상 또는 음원 다운로드
- UI는 심플하게 구성

앱 개발 추가 예제. 유튜브 영상 다운로드

▶ 프롬프트 만들기



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.
유튜브 영상 링크를 붙여넣기 하면 해당 영상을 다운로드 하는 앱을 만들거야

1. 개발환경

- Python 사용
- GUI 프레임워크: PyQt5
- 영상/음원 분석 및 다운로드: yt-dlp 패키지

2. 노드설명

- 유튜브 링크 입력
- 다운로드 폴더 선택
- "분석" 버튼 클릭 시 yt-dlp로 링크 분석 후 가능한 다운로드 포맷 리스트 출력
- 사용자가 포맷 선택
- "다운로드" 버튼 클릭 시 선택한 포맷으로 영상 또는 음원 다운로드

3. UI설명

- 심플한 UI 구성
- 유튜브 링크 입력창
- 다운로드 폴더 선택 메뉴
- 분석 버튼
- 포맷 리스트 출력 영역
- 다운로드 버튼

M2. 바이트코딩 활용 앱/웹 개발

5. 바이트코딩 기반 웹 개발

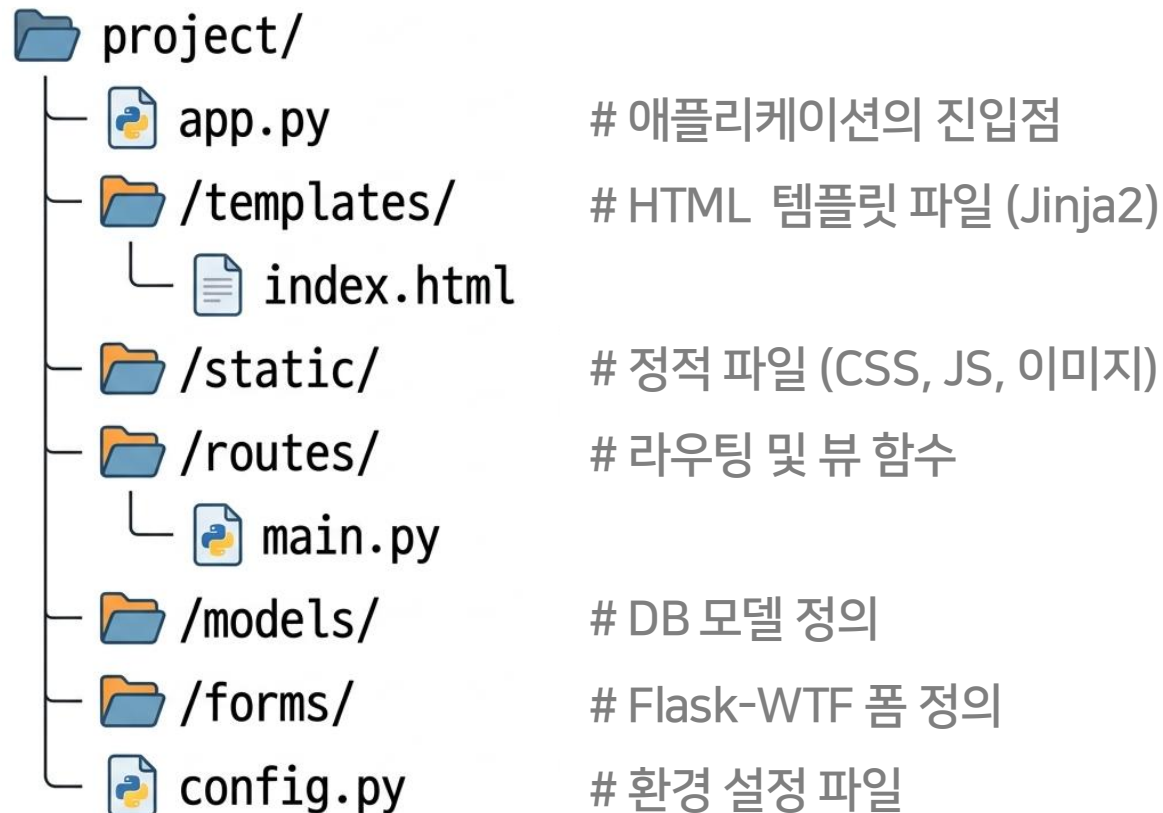
웹 개발을 위한 웹 프레임워크(Flask)

파이썬(Python) 기반 경량 웹 프레임워크(Micro Web Framework)

특징

주요 특징	내용
경량성 & 유연성	기본 기능만 포함되어 가볍고 빠르며, 플러그인 및 확장 패키지로 기능 추가 가능
모듈화된 구조	Blueprint를 사용하여 코드 구조를 모듈화하고 대규모 프로젝트에 유리
템플릿 렌더링	Jinja2 템플릿 엔진을 통해 HTML과 데이터를 쉽게 연결
디버깅 및 테스트	내장 디버거와 유닛 테스트 도구로 개발 편의성 향상
풍부한 확장 기능	Flask-Login, SQLAlchemy, Migrate 등 다양한 확장 모듈 제공

구조



웹 개발 실습 절차

① 프롬프트 만들기

- ChatGPT or Gemini에서 Cursor AI에 요청할 프롬프트 만들기

② 실습 폴더 확인

- 배포된 실습 폴더 확인 (예: M2_05-3_pdf_translate)

③ Cursor 실행

- Cursor 실행, 실습 폴더 열기, 파이썬 패키지 설치(기존 설치 버전 유지)

④ New Chat 클릭

- 오른쪽 사이드바에서 New Chat 클릭, 프롬프트 입력

⑤ 진행사항 체크

- 모델 선택, 진행 여부(폴더 생성, 패키지 설치 등) 승인(Run) 클릭

Cursor

⑥ 앱 테스트

- 실습파일 업로드 후 앱 테스트, 이상 여부 확인

⑦ 오류 보완

- 오류 발생 화면 캡처, 오류발생 프롬프트 작성, 재실행

Cursor

웹 개발 실습1. PDF 번역

▶ 실습목표: PDF 문서의 텍스트 추출 및 다국어 번역

PDF 번역기

1. PDF 파일 업로드

파일 선택

20250930_[월간 리서치] 철강_part_2.pdf

2. 번역 방향 선택

영어 → 한국어

한국어 → 영어

번역 시작

번역 진행률

100%

추출된 원본 텍스트

[월간 리서치] 철강

Mirae Asset Securities Research 3 2025.9.30

Investment Summary

중국 정부의 일관된 강산 의지, 조강생산 감소, 수요 부진

중국 정부의 일관된 강산 의지가 지속되는 가운데 2025년 (1~8월 누계 기준) 중국 조강생

산량은 약 6.7억톤 (-2.8%, YoY) 으로 전년동기 대비 약 1,950 만톤 감소했다. 2025 년 3월,

전인대 (전국인민대표회의) 에서 언급된 것으로 알려진 연간 5,000만톤 규모의 강산 목표치

대비 약 40% 수준이다. 동기간 글로벌 조강생산량은 약 12.3억톤으로 전년 동기 대비 0.6%

감소했다.

반면, 중국 철강 수요 회복은 지연되고 있다. 중국의 주택 착공 면적은 2021년 8월이래 전

년 동기 대비 감소 흐름이 이어지고 있다. 2025 년 상반기 기준, 미분양 부동산 면적 역시

고점을 점차 넘어가고 있다. 중국 철강 제품별 가격 역시 판매량 대비 건설업과 관련성이

높은 철근 가격 약세가 보다 두드러지게 나타나고 있다.

각국의 중국산 반덤핑 움직임 지속. 국내 반덤핑 관세 부과 결정

번역된 텍스트

[Monthly Research] Steel

Mirae Asset Securities Research 3 September 2025

Investment Summary

Consistent reduction efforts by the Chinese government. Decrease in steel production, weak demand

Amidst the Chinese government's consistent reduction efforts, as of 2025 (cumulative basis from January to August), China's steel

production volume decreased by approximately 67 million tons (-2.8%, YoY) compared to the previous year, amounting to a reduction of

about 19.5 million tons. In March 2025, the annual reduction target of 50 million tons, known to have been mentioned at the National

People's Congress, accounts for approximately 40% of the target. During the same period, global steel production decreased by 0.6% to

approximately 1.23 billion tons compared to the previous year.

On the other hand, the recovery of steel demand in China is being delayed. The area of housing construction started in China has been

following a decreasing trend compared to the previous year since August 2021. In the first half of 2025, the area of unsold real

estate has also reached a new high. Prices of Chinese steel products, particularly reinforcing bars that are highly related to the

construction industry, are showing a more distinct decline compared to raw materials.

Continued anti-dumping movements against Chinese products. Decision on imposing domestic anti-dumping tariffs

번역 파일 다운로드

1. 기능 : PDF 파일 업로드, 한/영 및 영/한 번역, 번역 진행률 표시, 원본 및 번역 텍스트 대조, 결과물 다운로드
2. 메뉴 : PDF 파일 업로드, 번역 방향 선택, 번역시작, 추출된 원본 텍스트 확인, 번역된 텍스트 확인
3. 데이터베이스 : 필요없음
4. 개발환경 : 파이썬(Python), 웹 프레임워크(Flask), PDF 추출 라이브러리, LLM API(OpenAI)

웹 개발 실습1. PDF 번역

▶ 프롬프트 초안 : 카테고리 만들기



아래 내용을 참고해서 개발환경, 노드(기능), UI로 구분해줘

=====

PDF파일을 업로드하면 번역해주는 웹을 만들거야

- flask패키지와 pypdf(또는 pypdf2)를 이용해서 단순하게 만들고 싶어
- 번역은 open api key를 사용할거야
- open api key는 나의 시스템 환경 변수에 저장되어 있어
- 영어를 한국어로, 한국어를 영어로 선택할 수 있게 해
- 번역후에 저장 할 수 있게 만들어
- 번역 진도율을 볼 수 있게 해
- pdf에서 텍스트만 추출해서 번역하고 번역된 내용을 텍스트 파일에 저장해
- 저장하는 파일명은 업로드한 pdf파일명이랑 동일하게 설정해
- pdf에서 추출한 텍스트와 번역한 내용을 미리보기로 볼 수 있도록 해

웹 개발 실습1. PDF 번역

① 프롬프트 만들기



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어
PDF파일을 업로드하면 번역해주는 앱을 만들고 싶어

1. 개발환경

- Flask 패키지 사용
- pypdf (또는 pypdf2) 사용
- 번역: OpenAI API 사용
- OpenAI API key는 시스템 환경 변수에 저장

2. 노드설명

- PDF 파일 업로드 시 텍스트 추출
- 영어→한국어, 한국어→영어 번역 방향 선택에 따라 OpenAI API 호출
- 번역 진도율 실시간 전달
- 번역 완료 후 업로드한 PDF 파일명과 동일한 이름으로 텍스트 파일 저장
- 추출한 텍스트와 번역 결과 미리보기 반환

3. UI설명

- PDF 업로드 메뉴
- 번역 방향 선택 (영어→한국어 / 한국어→영어)
- 번역 진도율 표시 UI
- 원문 텍스트 미리보기 영역
- 번역 결과 미리보기 영역
- 저장 버튼

웹 개발 실습1. PDF 번역

① 프롬프트 만들기 : 완성_LLM이 만들어준 프롬프트

Prompt

너는 파이썬 및 Flask 웹 개발 전문가야. 아래의 요구사항을 바탕으로 PDF 파일의 텍스트를 추출해 번역하고, 결과를 텍스트 파일로 저장할 수 있는 웹 애플리케이션 코드를 작성해 줘.

기술 스택

- Backend: Python, Flask
- PDF 텍스트 추출: pypdf (또는 PyPDF2)
- 번역 API: OpenAI API (openai 패키지)

UI/UX 및 화면 레이아웃 요구사항 (ChatGPT 스타일의 2단 분할 화면)
화면을 좌/우 두 개의 영역으로 분할하여 데스크톱에서 보기 좋게 구성해.

1. 왼쪽 패널 (컨트롤 및 진행 상태):

- PDF 파일 업로드 폼
- 번역 방향 설정 (영어 -> 한국어, 한국어 -> 영어 선택 라디오 버튼 또는 드롭다운)
- '번역 시작' 버튼
- 번역 진행률(%)을 실시간으로 보여주는 프로그레스 바 또는 텍스트

2. 오른쪽 패널 (미리보기 및 다운로드):

- 'PDF에서 추출한 텍스트'를 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)
- '번역된 내용'을 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)
- 번역 완료 시 활성화되는 '다운로드' 버튼

3. 스타일링:

- 배경이 어두운 색이면 글자 색깔을 흰색으로, 배경이 밝은 색이면 글자 색깔을 검은색으로 자동 적용되도록 CSS를 작성해.
- 좌우 패널이 깔끔하게 나뉘도록 Flexbox나 Grid를 활용해 직관적으로 디자인해.

핵심 기능 요구사항

1. PDF 텍스트 추출 및 번역 연동:

- pypdf를 이용해 업로드된 PDF 파일에서 '텍스트'만 추출해.
- OpenAI API를 사용하여 추출된 텍스트를 선택한 언어로 번역해.
- API Key는 절대 하드코딩하지 말고, 시스템 환경 변수에 저장된 'OPENAI_API_KEY'를 'os.environ'으로 불러와서 사용해.

2. 번역 진행률 표시 및 실시간 렌더링 (SSE):

- 텍스트를 페이지 단위나 적절한 청크 단위로 나누어 번역을 진행해.
- 번역이 진행되는 동안 왼쪽 패널의 진도율이 업데이트되고, 동시에 오른쪽 패널의 '번역된 내용' 영역에 텍스트가 실시간으로 추가되도록 SSE(Server-Sent Events)나 AJAX 폴링을 구현해.

3. 텍스트 파일(.txt) 저장 및 다운로드:

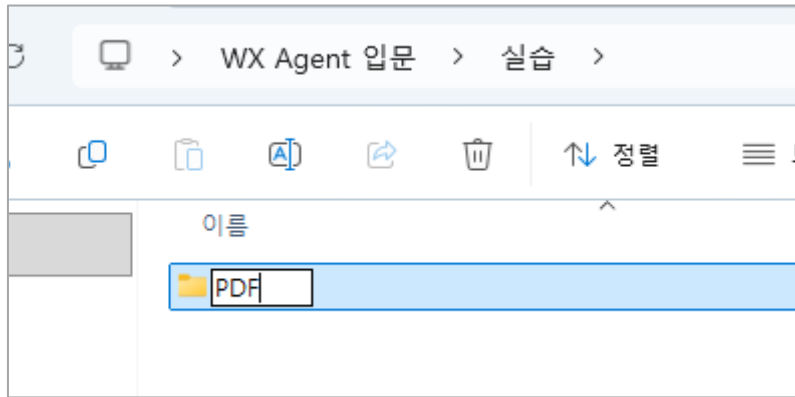
- 번역이 완료되면 오른쪽 패널의 다운로드 버튼을 통해 번역된 텍스트 파일(.txt)을 받을 수 있게 해.
- 다운로드되는 텍스트 파일의 이름은 업로드한 원본 PDF 파일의 이름과 동일하게 설정해 (예: 'report.pdf' 업로드 시 'report.txt'로 다운로드).

출력 지침

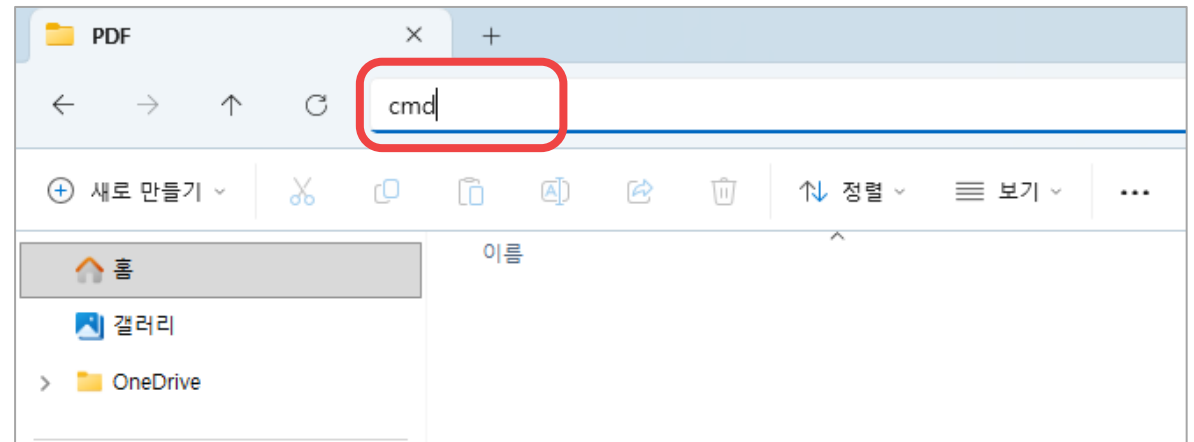
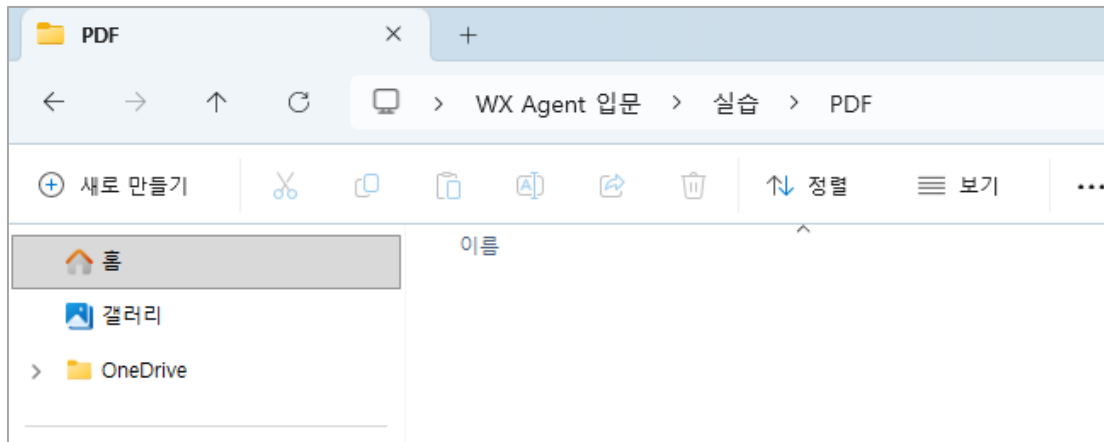
- 'app.py', 'templates/index.html' 코드와 필요한 폴더 구조를 명확히 나누어 작성해 줘.
- 코드를 실행하기 위해 설치해야 하는 'requirements.txt' 내용과 터미널 실행 명령어도 함께 안내해 줘. 이미 설치된 패키지는 설치된 버전 그대로 적고 업데이트하지 마.

웹 개발 실습1. PDF 번역

② 실습 폴더 확인 : 실습 - 「M2_05-3_pdf_translate」 폴더

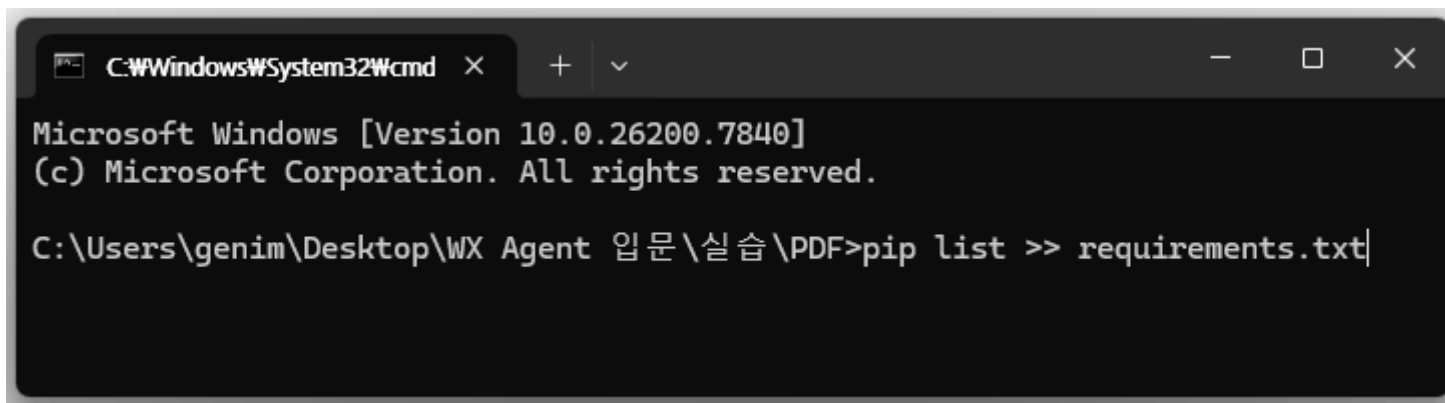


③ 개발 환경 설정 : 「M2_05-3_pdf_translate」 이동 → 주소창 cmd → Enter



웹 개발 실습1. PDF 번역

③ 개발 환경 설정 : `pip list >> requirements.txt`



```
C:\Windows\System32\cmd
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\genim\Desktop\WX Agent 입문\실습\PDF>pip list >> requirements.txt
```

③ 개발 환경 설정 : Cursor AI 실행 → File-Open Folder-실습-M2_05-3_pdf_translate 폴더 클릭



웹 개발 실습1. PDF 번역

④ New Chat 클릭 : 프롬프트 입력 → 모델 선택 → 실행 클릭

The screenshot shows a web application interface for PDF translation. The main area is a code editor with a file named 'requirements.txt'. To the right is a 'New Chat' window with a text input field and a dropdown menu for selecting a model (currently set to 'Sonnet 4.5'). A red box highlights the 'New Chat' window, and a red arrow points from the instructions on the left to it.

Prompt

너는 파이썬 및 Flask 웹 개발 전문가야. 아래의 요구사항을 바탕으로 PDF 파일의 텍스트를 추출해 번역하고, 결과를 텍스트 파일로 저장할 수 있는 웹 애플리케이션 코드를 작성해 줘.

기술 스택

- Backend: Python, Flask
- PDF 텍스트 추출: pypdf (또는 PyPDF2)
- 번역 API: OpenAI API (openai 패키지)

UI/UX 및 화면 레이아웃 요구사항 (ChatGPT 스타일의 2단 분할 화면)
화면을 좌/우 두 개의 영역으로 분할하여 데스크톱에서 보기 좋게 구성해.

- 왼쪽 패널 (컨트롤 및 진행 상태):
 - PDF 파일 업로드 폼
 - 번역 방향 설정 (영어->한국어, 한국어->영어 선택 라디오 버튼 또는 드롭다운)
 - '번역 시작' 버튼
 - 번역 진행률(%)을 실시간으로 보여주는 프로그레스 바 또는 텍스트
- 오른쪽 패널 (미리보기 및 다운로드):
 - 'PDF에서 추출한 텍스트'를 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)
 - '번역된 내용'을 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)
 - 번역 완료 시 활성화되는 '다운로드' 버튼
- 스타일링:
 - 배경이 어두운 색이면 글자 색깔을 흰색으로, 배경이 밝은 색이면 글자 색깔을 검은색으로 자동 적용되도록 CSS를 작성해.
 - 좌우 패널이 깔끔하게 나뉘도록 Flexbox나 Grid를 활용해 직관적으로 디자인해.

핵심 기능 요구사항

- PDF 텍스트 추출 및 번역 연동:
 - pypdf를 이용해 업로드된 PDF 파일에서 '텍스트'만 추출해.
 - OpenAI API를 사용하여 추출된 텍스트를 선택한 언어로 번역해.
 - API Key는 절대 하드코딩하지 말고, 시스템 환경 변수에 저장된 'OPENAI_API_KEY'를 'os.environ'으로 불러와서 사용해.
- 번역 진행률 표시 및 실시간 렌더링 (SSE):
 - 텍스트를 페이지 단위나 적절한 청크 단위로 나누어 번역을 진행해.
 - 번역이 진행되는 동안 왼쪽 패널의 진도율이 업데이트되고, 동시에 오른쪽 패널의 '번역된 내용' 영역에 텍스트가 실시간으로 추가되도록 SSE(Server-Sent Events)나 AJAX 폴링을 구현해.
- 텍스트 파일(.txt) 저장 및 다운로드:
 - 번역이 완료되면 오른쪽 패널의 다운로드 버튼을 통해 번역된 텍스트 파일(.txt)을 받을 수 있게 해.
 - 다운로드되는 텍스트 파일의 이름은 업로드한 원본 PDF 파일의 이름과 동일하게 설정해 (예: 'report.pdf' 업로드 시 'report.txt'로 다운로드).

출력 지침

- 'app.py', 'templates/index.html' 코드와 필요한 폴더 구조를 명확히 나누어 작성해 줘.
- 코드를 실행하기 위해 설치해야 하는 'requirements.txt' 내용과 터미널 실행 명령어도 함께 안내해 줘.

OUTLINE

TIMELINE

PDF 0 0 0

New Chat

Plan, @ for context, / for commands

Agent Sonnet 4.5

Past Chats

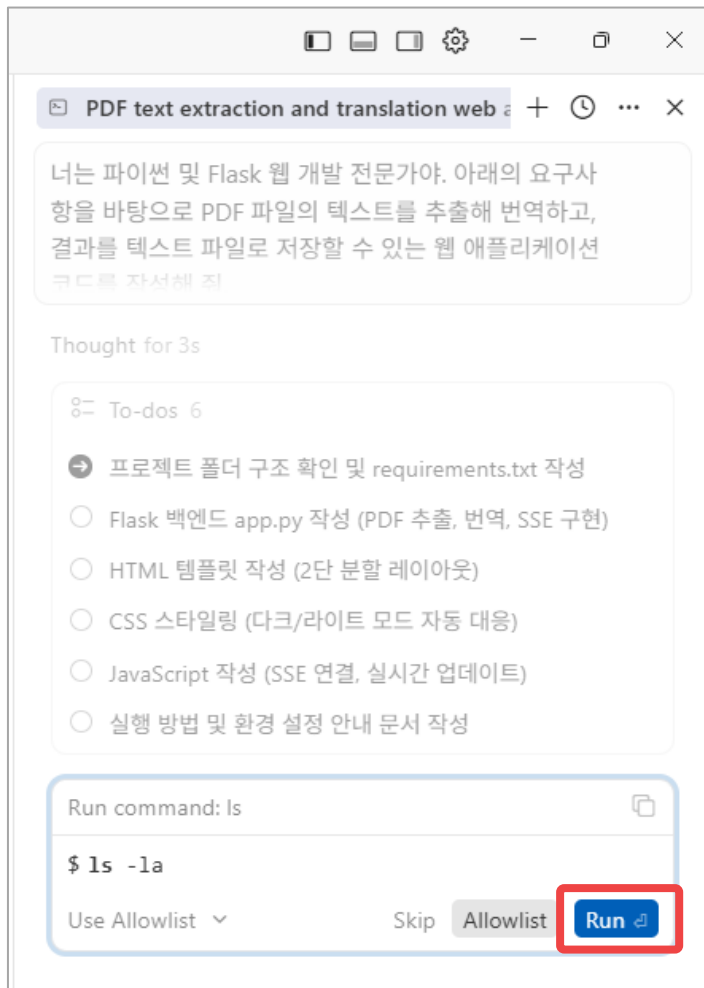
New Chat

Now

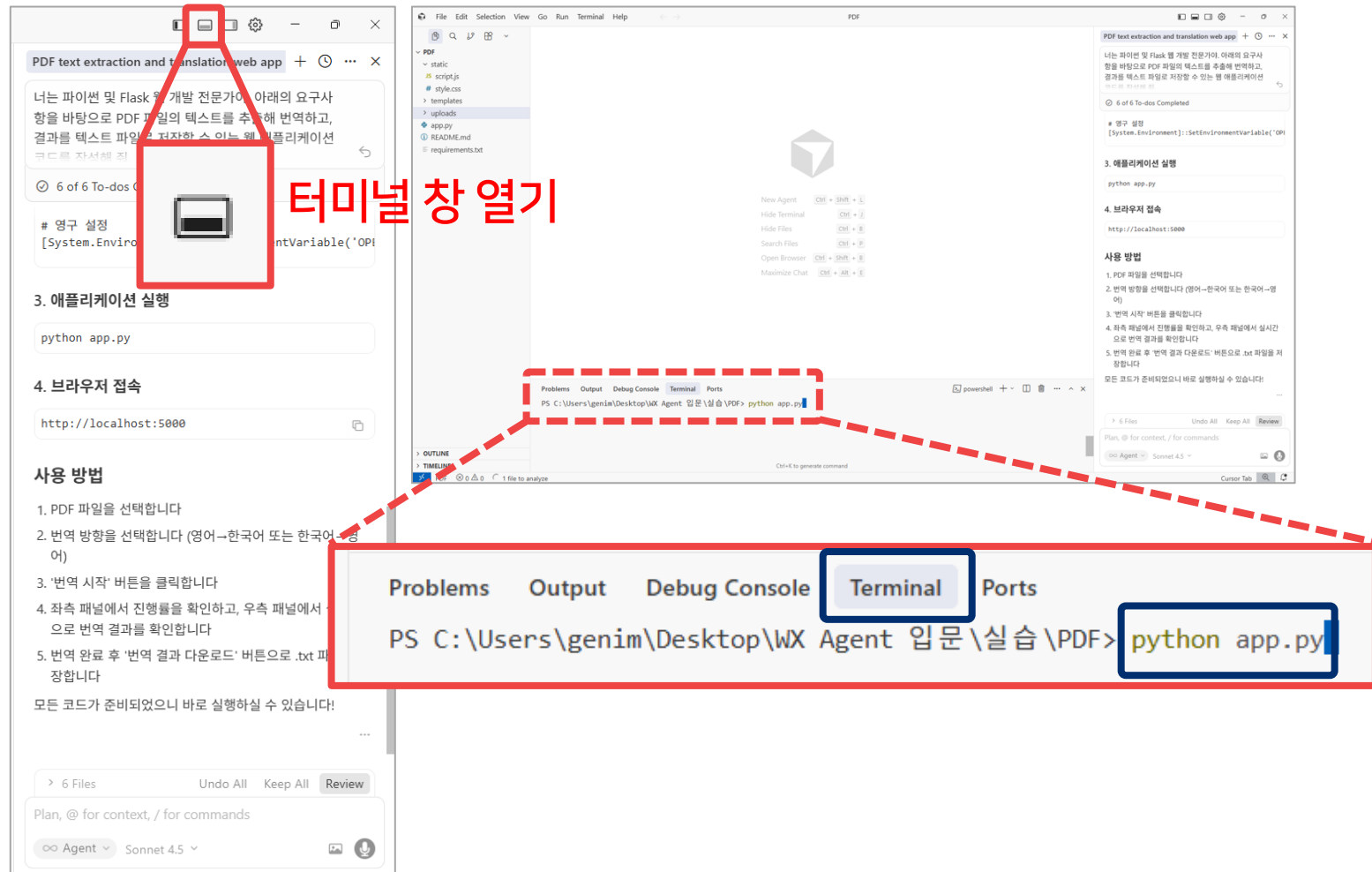
Cursor Tab

웹 개발 실습1. PDF 번역

⑤ 진행사항 승인 체크 : Run 클릭

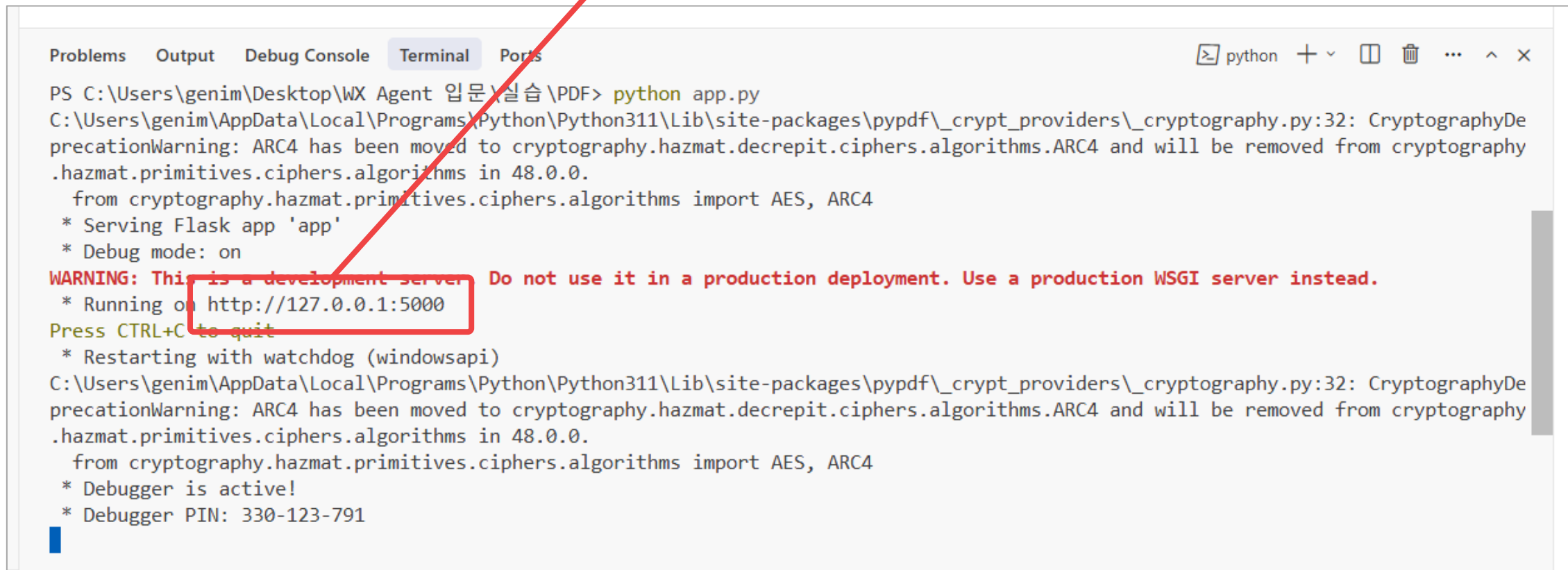


⑥ 웹 테스트 : 터미널 창 열기 → python app.py 입력 → Enter



웹 개발 실습1. PDF 번역

⑥ 웹 테스트 : ctrl 키 누른 상태에서 주소 클릭

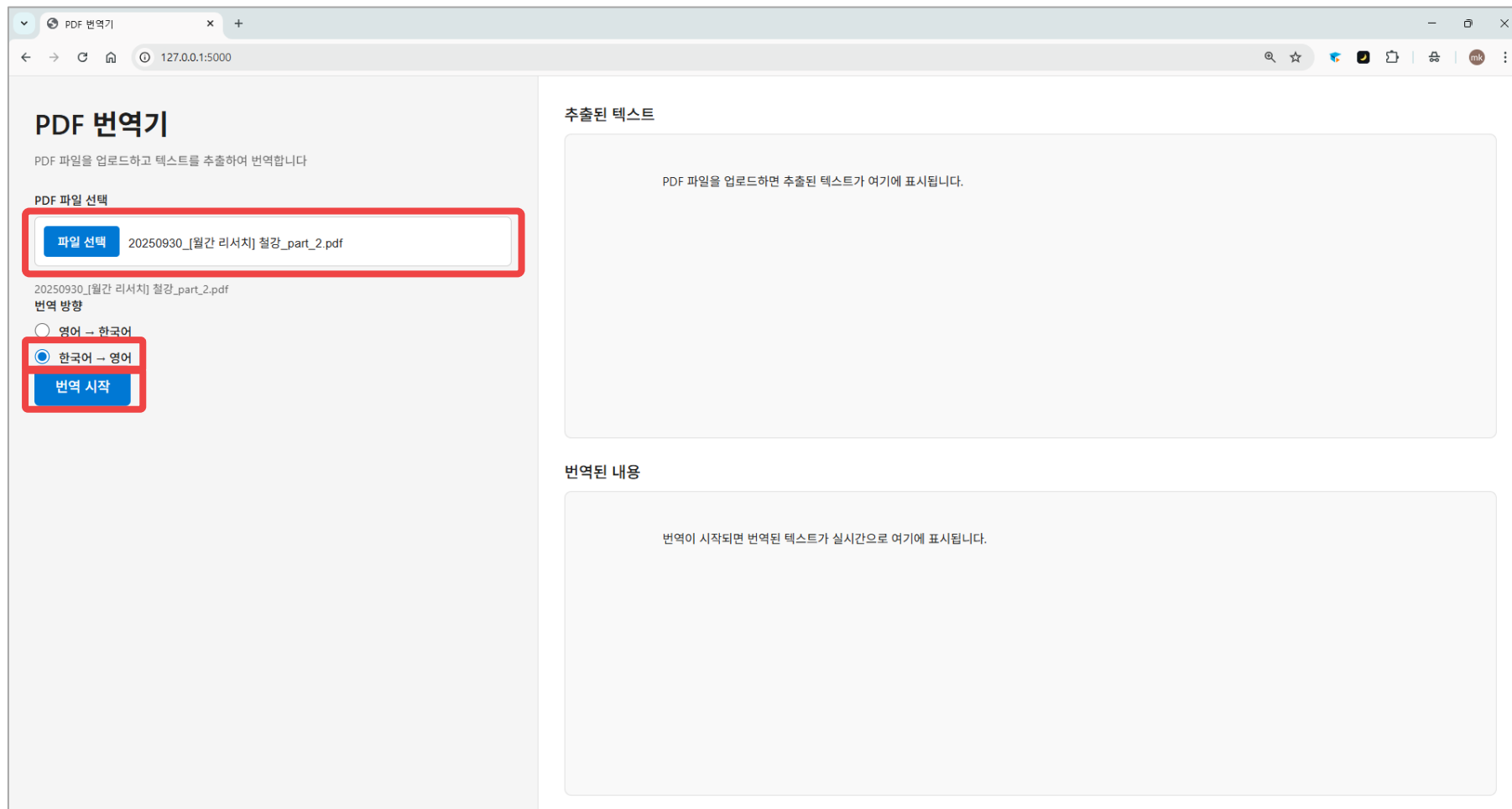


```
Problems Output Debug Console Terminal Ports
PS C:\Users\genim\Desktop\WX Agent 입문\실습\PDF> python app.py
C:\Users\genim\AppData\Local\Programs\Python\Python311\Lib\site-packages\pypdf\_crypt_providers\_cryptography.py:32: CryptographyDeprecationWarning: ARC4 has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  from cryptography.hazmat.primitives.ciphers.algorithms import AES, ARC4
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
C:\Users\genim\AppData\Local\Programs\Python\Python311\Lib\site-packages\pypdf\_crypt_providers\_cryptography.py:32: CryptographyDeprecationWarning: ARC4 has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  from cryptography.hazmat.primitives.ciphers.algorithms import AES, ARC4
* Debugger is active!
* Debugger PIN: 330-123-791
```

※ Cursor 안에서 웹페이지 실행될 경우, 크롬 창 열어서 크롬 주소창에 `http://127.0.0.1:5000` 입력 후 Enter

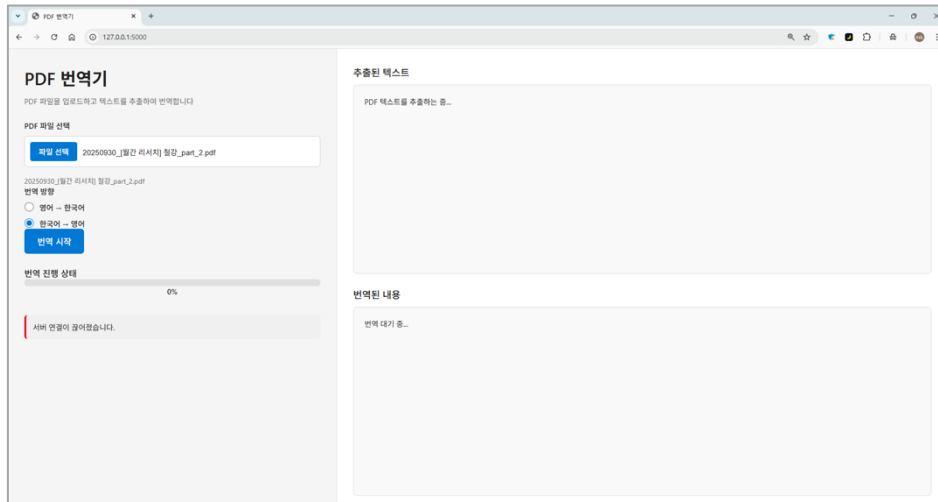
웹 개발 실습1. PDF 번역

⑥ 웹 테스트 : '20250930_[월간 리서치] 철강_part_2'파일 업로드 → 번역 방법 선택 → 번역 시작 클릭

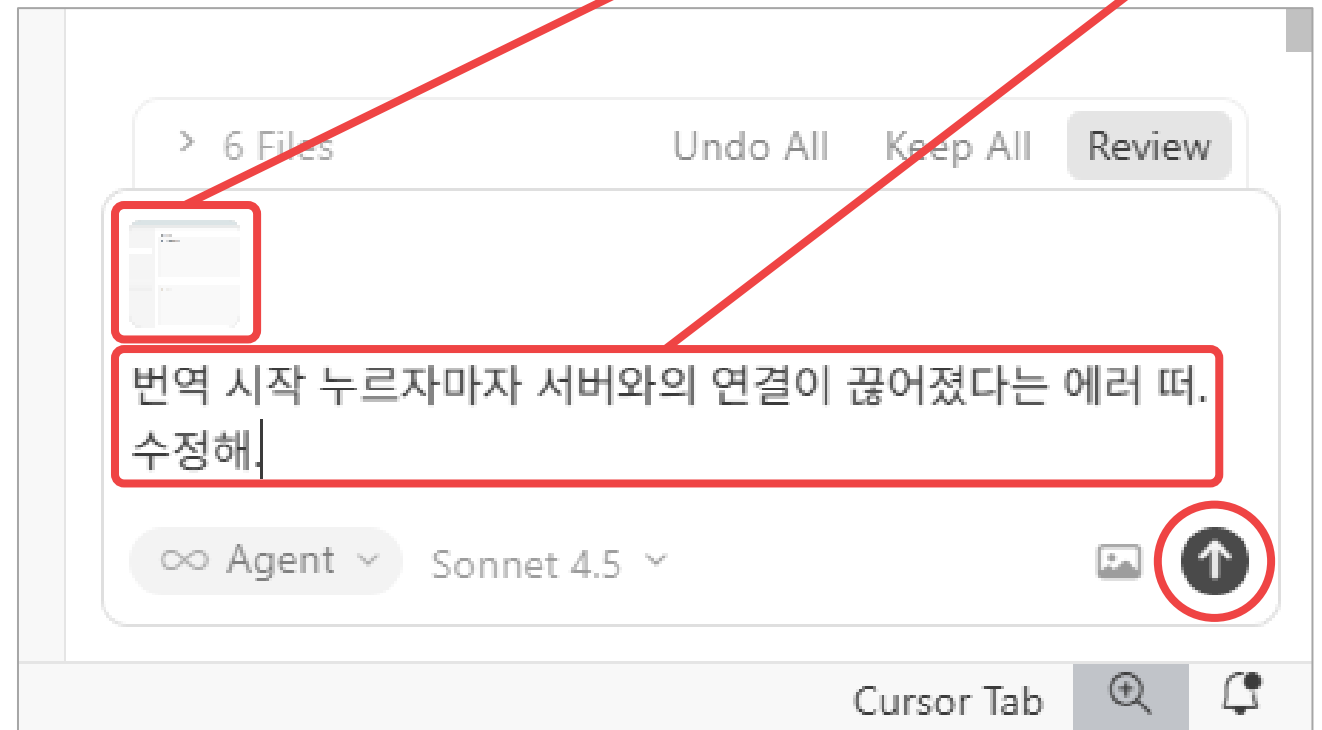


웹 개발 실습1. PDF 번역

⑦ 오류 보완 : 오류 발생한 화면 캡처(window + shift + s)



→ cursor AI 대화창에 캡처 이미지 붙여 넣고 오류 상황 입력



※ 수정 완료 후 웹 페이지 새로고침하여 수정 여부 확인

오류 수정 후, 결과

PDF 파일 선택

파일 선택 20250930_[월간 리서치] 철강_part_2.pdf

20250930_[월간 리서치] 철강_part_2.pdf

번역 방향

☐ 영어 → 한국어

☒ 한국어 → 영어

번역 시작

번역 진행 상태

100%

번역이 완료되었습니다!

PDF 번역기

PDF 파일을 업로드하고 텍스트를 추출하여 번역합니다

PDF 파일 선택

파일 선택 20250930_[월간 리서치] 철강_part_2.pdf

20250930_[월간 리서치] 철강_part_2.pdf

번역 방향

☐ 영어 → 한국어

☒ 한국어 → 영어

번역 시작

번역 진행 상태

100%

번역이 완료되었습니다!

PDF 번역기

PDF 파일을 업로드하고 텍스트를 추출하여 번역합니다

PDF 파일 선택

파일 선택 20250930_[월간 리서치] 철강_part_2.pdf

20250930_[월간 리서치] 철강_part_2.pdf

번역 방향

☐ 영어 → 한국어

☒ 한국어 → 영어

번역 시작

번역 진행 상태

100%

번역이 완료되었습니다!

자료: WSA, 미래에셋증권 리서치센터

표 1. 2025 년, 중국 철강 관련 주요 내용

내용 시기 세부내용

양회 (국가발전개혁위원회 보고서) 25.03.04~25.03.11 - 국가발전개혁위원회, “ 조강 생산관리를 지속적으로 시행하여 철강 산업의 규모 축소 및 구조조정 촉진” 제시

- 2019년 이후 처음으로 국가발전개혁위원회가 양회에 철강 감산을 제안한것

- 구체적인 감산 목표치는 제시하지 않았으나, 언론을 통해 연간 5천만톤 감산 계획 (추정) 보도

제6차 중앙재정경제위원회 25.07.01 - 시진핑, 기업 간의 자가 경쟁과 무질서한 경쟁을 통제하며 낙후된 생산능력의 질서있는 퇴출을 강조

당산 지역 감산 소문 25.07.02 - 중국 정부, 당산지역 제철소에 7월 4일 부터 15일까지 당산 소결기 생산을 30% 감축한다고 통지

철강산업 2개년 성장계획 25.09.22 - 중국 정부, 철강산업 발전을 저해하는 요소로 공급 과잉 등에 따른 수급불균형 문제를 지목

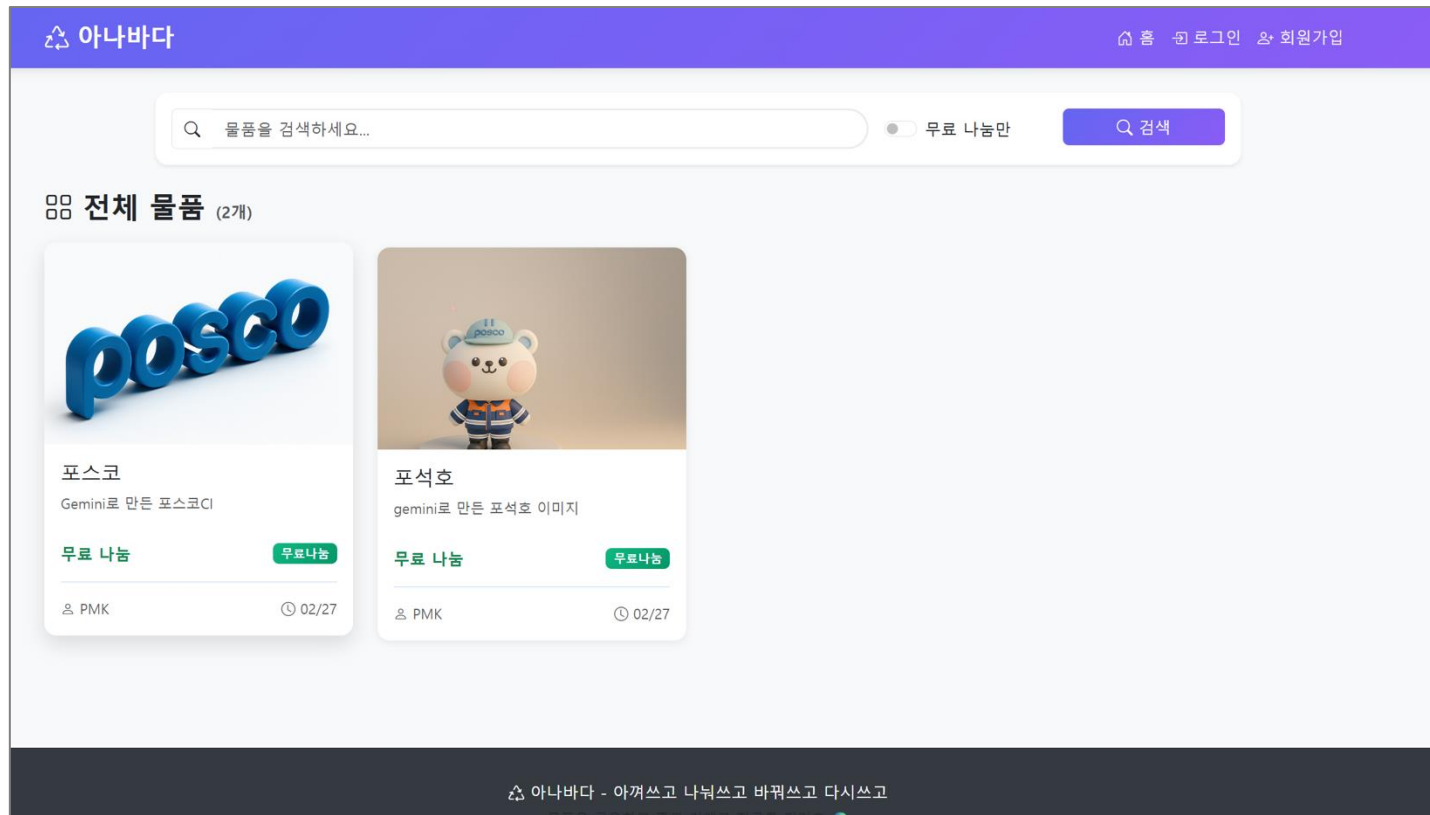
- 해당 계획은 철강 생산능력 추가 금지 및 생산 정밀 통제, 노후화비효를 설비 폐쇄 등의 방침 제시

- 이에 따라 2025~2026년 철강산업 연간 성장률을 4%로 설정

results due to improved spreads in raw materials, the consolidated company's performance is forecasted to decrease due to the disappearance of one-on-one prompt improvement effects. Performance in the special steel sector in the current quarter is expected to be limited due to the aftermath of construction accidents. The effects of the pre-order system for rebar distribution introduced in September are believed to have had a limited impact on short-term performance. In the third quarter of 2025, POSCO Holdings' consolidated operating profit is estimated to be KRW 570.2 billion (-23.3% YoY), which is expected to fall 10% below market expectations (operating profit of KRW 631 billion, on a one-month basis). The steel sector's profitability is forecasted to increase compared to the second quarter, a seasonal peak, due to improved spreads from the decline in raw material prices. However, the consolidated operating profit is expected to be weak due to the expanded losses at POSCO E&C, reflecting direct and indirect costs from the suspension of on-site construction due to accidents. [Mirae Asset Securities Research] Steel: Chinese Steel Production, Wider Decrease Compared to Global. In 2025 (accumulated basis from January to August), China's steel production decreased by approximately 670 million tons (-2.8% YoY), a reduction of about 19.5 million tons compared to the previous year. In March 2025, during the National People's Congress, an annual reduction target of 50 million tons was mentioned, which is around 40%. During the same period, global steel production decreased by about 1.23 billion tons, a 0.6% decrease compared to the previous year. In 2025, the Chinese government's consistent reduction efforts continue. In July, during the 6th Central Economic Work Conference, President Xi Jinping emphasized controlling low-cost and disorderly competition among enterprises and orderly withdrawal of outdated production capacity. A recent 'Work Plan for Stable Growth of the Steel Industry (2025-2026)' announced by five Chinese ministries includes an annual growth target of 4%, as well as quality improvement, structural optimization, and prohibition of expanding new production capacity. The 15th Five-Year Plan to be announced in 2025 is expected to confirm a similar policy direction. [Graph 1. Global, YoY Change in Chinese Steel Production Compared to Previous Year] [Data: WSA, Mirae Asset Securities Research Center] [Table 1. Key Contents Related to Chinese Steel in 2025] [Data: Media, Mirae Asset Securities Research Center]

웹 개발 실습2. 물품 공유

▶ 실습목표: 물품 공유 웹 개발(flask사용, Cursor로 코드 생성)



1. 기능 : 물품 사진, 위치 등
2. 메뉴 : 등록, 수정, 삭제
3. 데이터베이스 : 등록자, 등록일, 수정일, 제품정보
4. 개발환경 : flask

웹 개발 실습2. 물품 공유

▶ 프롬프트 초안 : 카테고리 만들기



아래 내용을 참고해서 개발환경, 노드(기능), UI로 구분해줘

=====

아나바다처럼 물품 공유 웹을 만들고 싶어

- flask패키지를 이용해.
- 사용자가 물품 사진을 등록, 수정, 삭제할 수 있어야 돼
- 물품에 대한 설명을 넣을 수 있어야돼
- 가격을 넣을 수도 있어야 하고, 무료도 있어야 돼
- 데이터베이스: Database(등록자, 등록일, 수정일, 제품정보 등)가 있어야 돼,
- 이 외에도 추가 기능이 필요하면 추가해줘

웹 개발 실습2. 물품 공유

▶ 프롬프트 만들기



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.
아나바다처럼 물품 공유 웹을 만들고 싶어

1. 개발환경

- Flask 패키지 사용
- 데이터베이스 구성: 등록자, 등록일, 수정일, 제품정보
- 물품 사진 파일 업로드 저장소 구성
- 사용자 인증 세션 관리

2. 노드설명

- 물품 사진 등록, 수정, 삭제
- 물품 설명 입력
- 가격 입력 (무료 옵션 포함)
- 물품 목록 조회 및 검색
- 사용자 회원가입 및 로그인/로그아웃
- 관심 물품 찜하기 기능
- 등록자 간 댓글 또는 문의 기능
- 거래 완료 상태 표시

3. UI설명

- 메인 페이지: 물품 목록 카드형 표시
- 물품 등록 폼 (사진 업로드, 설명, 가격/무료 선택)
- 물품 수정/삭제 버튼
- 검색창
- 회원가입 / 로그인 메뉴
- 찜하기 버튼
- 댓글/문의 입력 영역
- 거래 완료 배지 표시

웹 개발 실습2. 물품 공유

▶ 프롬프트 최종

Prompt

Flask와 Python을 사용하여 '아나바다(물품 공유 및 중고 거래)' 웹 애플리케이션을 개발해 줘. 아래의 요구사항과 데이터베이스 구조를 바탕으로 필요한 모든 코드(app.py, models.py, HTML 템플릿 등)를 작성해 주길 바라.

1. 기술 스택

- Backend: Python, Flask
- Database: SQLite 및 Flask-SQLAlchemy (ORM)
- Frontend: HTML, CSS (Bootstrap 5를 사용하여 깔끔하고 반응형으로 디자인), Jinja2 템플릿
- Image Handling: Flask-Uploads 또는 werkzeug.utils.secure_filename을 이용한 로컬 저장

2. 핵심 기능 요구사항

- 회원 관리: 사용자가 물품의 '등록자'가 되어야 하므로 간단한 회원가입 및 로그인/로그아웃 기능이 필요해 (Flask-Login 사용).
- 물품 CRUD: 사용자는 물품을 등록(사진 업로드 포함), 조회, 수정, 삭제할 수 있어야 해. 수정과 삭제는 해당 물품을 등록한 작성자만 가능해야 해.
- 물품 상세 정보: 제목, 사진(이미지), 상세 설명, 가격 정보가 들어가야 해.
- 가격 및 무료 나눔 처리: 가격을 입력받되, 가격이 '0'이거나 체크박스를 선택하면 '무료 나눔'으로 표시해.
- 물품 상태 관리 (추가 기능): 물품이 현재 '거래 중'인지 '나눔/거래 완료'인지 상태를 변경할 수 있는 버튼을 추가해.
- 검색 및 필터 (추가 기능): 메인 페이지에서 제목이나 내용으로 물품을 검색할 수 있고, '무료 나눔만 보기' 필터가 있으면 좋겠어.

3. 데이터베이스 모델 (Database Schema)

- User 모델:
 - id (Primary Key)
 - username (사용자 이름, Unique)
 - password_hash (비밀번호)
- Item 모델:
 - id (Primary Key)
 - title (물품 제목, String)
 - description (물품 설명, Text)
 - price (가격, Integer, 0이면 무료)
 - image_filename (업로드된 이미지 파일명, String)
 - status (상태: '거래가능', '거래완료' 등, String)
 - author_id (등록자, User 모델의 id를 참조하는 Foreign Key)
 - created_at (등록일, DateTime, 기본값 현재시간)
 - updated_at (수정일, DateTime, 수정 시 업데이트)

4. 디렉토리 구조 및 단계별 작업

- 프로젝트의 기본 디렉토리 구조를 먼저 제시해 줘.
- 그 다음, app.py 세팅, 데이터베이스 모델 구성, 라우팅, 그리고 프론트엔드 HTML(base.html, index.html, item_detail.html, item_form.html 등) 코드를 순서대로 제공해 줘.

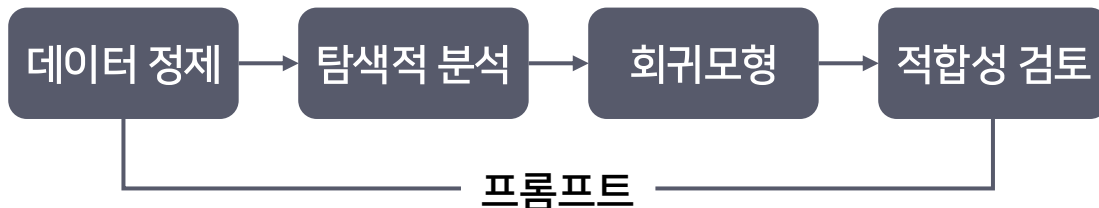
웹 개발 실습 3. 발전기 복합출력 예측

▶ 실습목표: 발전기 복합출력과 연관성 있는 변수들과의 회귀분석을 통해 복합출력량 예측 모델 개발

데이터셋

- output.csv * 변수: 7개, 데이터: 414개
- 종속변수 : 복합출력
- 독립변수 : 대기온도, 대기압력, 대기상대습도, 전체필터차압, 공기압축기날개개도, 공기압축기흡입유량

절차



회귀분석

독립변수와 종속변수간의 관계를 분석하여 변수간의 인과 관계를 파악하거나 미래의 값을 예측하는데 사용

회귀분석의 종류

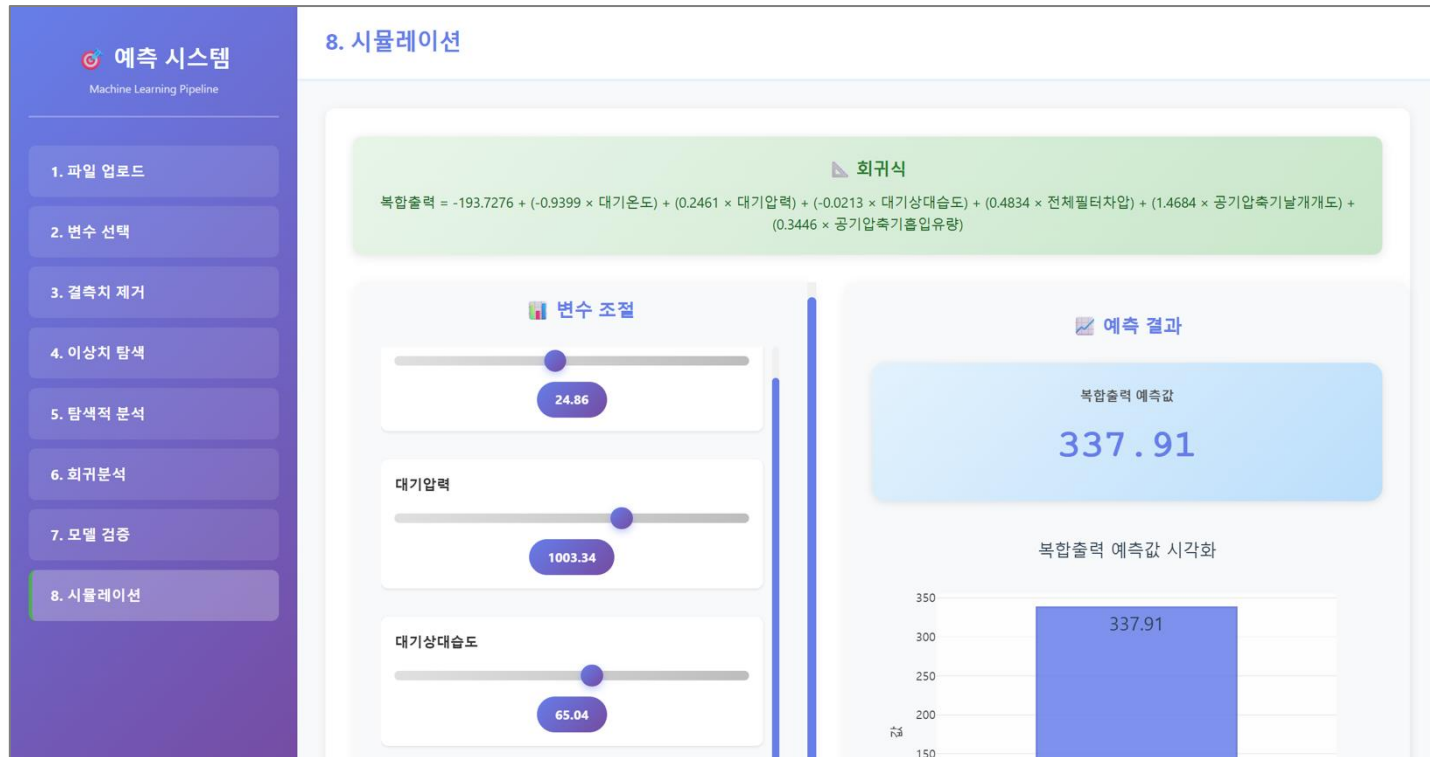
- 단순회귀 : 독립변수가 하나일때 (키와 몸무게 관계)
- 다중회귀 : 독립변수가 여러 개일때 (성적, 공부/수면시간)
- 로지스틱회귀 : 종속변수가 범주형일때 (합격/불합격)

활용예시

- 부동산 가격 예측 : 집의 크기, 위치, 층수 등으로 가격 예측
- 마케팅 : 광고 비용과 판매량의 관계를 분석
- 의료 : 특정 약물 사용량과 환자 회복 속도의 관계를 분석

웹 개발 실습 3. 발전기 복합출력 예측

▶ 실습목표: 발전기 복합출력과 연관성 있는 변수들과의 회귀분석을 통해 복합출력량 예측 모델 개발



1. 기능 : 파일업로드, 데이터분석 등
2. 메뉴 : 업로드, 변수선택, 결측치 제거 등
3. 데이터베이스 : 없음
4. 개발환경 : flask

웹 개발 실습 3. 발전기 복합출력 예측

▶ 프롬프트 초안 : 카테고리 만들기



아래 내용을 참고해서 개발환경, 노드(기능), UI로 구분해줘

=====

파일을 업로드하면 변수 변화에 따라 예측하는 시스템을 만들고 싶어

2. 개요

- 파이썬을 flask를 활용해서 예측 시스템 코드를 생성해줘
- 그래프 그릴때 한글이 안깨지도록 설정해줘
- 파일 업로드 부터 시뮬레이션 단계까지 순차적으로 진행되는데
전단계의 결과가 다음 단계에서 사용되는거야, 세션이 유지될 수 있도록
전 단계 결과를 sqlite3에 저장해서 사용해줘

3. 화면구성

- user interface의 컬러는 연한 녹색, 연한 파란색, 회색으로 만들어줘
- 화면을 2분할하는데 좌측을 20%, 우측을 80% 비율로 만들어줘, 우측 화면을 상단 10%, 하단 90%로 분할해줘
- 좌측 화면 메뉴 구성은 "1. 파일 업로드", "2. 변수 선택", "3. 결측치 제거", "4. 이상치 탐색", "5. 탐색적 분석", "6. 회귀분석", "7. 모델 검증", "8. 시뮬레이션" 으로 해줘
- 좌측 화면 메뉴는 순서대로 진행되어야 하기 때문에 처음에는 "1. 파일 업로드"를 진하게 표시하고 나머지 메뉴는 연하게 표시해줘,
하나의 메뉴가 실행된 후 다음 메뉴를 진하게 표시해줘
- 우측 화면 상단은, 좌측 메뉴를 눌렀을때 단계(예시 : 2. 변수선택)등이 표시되고 우측 하단 화면은 서브메뉴와 실행 결과가 나와야해

4. 메뉴별 구성

- "1. 파일 업로드" 메뉴를 클릭하면 폴더에서 csv, excel 파일을 업로드 할 수 있게 해줘,
파일을 업로드 하면 우측화면 하단에 pandas 패키지를 이용해서 10개의 행만 출력해줘, 그리고 각 변수의 기초 통계량을 출력해줘
- "2. 변수 선택" 메뉴를 클릭하면 우측 화면 상단에 종속변수와 독립변수를 선택하는 메뉴 만들어줘, 사용자가 변수를 선택하면 그것을 기준으로 차후 분석을 진행해야해
- "3. 결측치" 제거 메뉴를 클릭하면 결측치가 있는 행을 찾고 제거해줘,
결과 값으로 총 몇개의 행이 있었고, 결측치 행을 몇개 제거했는지, 현재 몇개행이 남아 있는지 표시해줘
- "4. 이상치 탐색" 메뉴를 클릭하면 각 변수별 ± 3 표준편차 범위 밖에 존재하는 데이터가 몇개나 있는지 표시해줘, 그리고 변수별 이상치 행을 제거해야 하는지 우측 화면 상단에서 선택할 수 있도록 표시해줘, 사용자가 이상치 제거를 선택하면 이상치 행을 제거해줘
- "5. 탐색적 분석" 메뉴를 클릭하면 우측화면 상단에 "히스토그램", "산점도", "상관관계(히트맵)" 분석 메뉴를 표시해줘, "히스토그램"을 선택하면 사용자가 변수를 선택할 수 있도록 하고, 변수 선택 후 히스토그램과 밀도 곡선을 겹쳐서 그려줘,
"산점도"를 선택하면 여러개의 변수를 선택하게 하고, 변수 선택 후 산점도를 그려줘,
"상관관계(히트맵)"을 선택하면 전체 변수의 상관계수가 표시된 히트맵을 그려줘

웹 개발 실습 3. 발전기 복합출력 예측

▶ 프롬프트 초안 : 카테고리 만들기

- "6. 회귀분석" 메뉴를 클릭하면 우측 상단 화면에 "전진선택법", "후진제거법", "릿지", "라쏘" 메뉴가 보이게 해줘,
사용자가 선택한 메뉴로 회귀분석을 실시하고, 회귀모형과, 예측 정확도를 출력해줘,
사용자가 모델의 결과값을 보고 최종 모델을 선택할 수 있도록 해줘
 - "7. 모델검증"을 클릭하면 잔차분석과 적합도검정을 진행하고 결과를 보여줘
 - "8. 시뮬레이션"을 클릭하면 만들어진 회귀모형을 시뮬레이션 할 수 있도록 해줘
우측화면의 상단에는 사용자가 선택한 회귀식을 출력해줘
인터랙티브한 그래프 출력을 위해 Plotly, Dash 패키지를 사용해줘
우측화면의 하단에는 회귀모형의 독립변수를 슬라이더바로 입력할 수 있게 하고
아래쪽에는 종속변수의 값과 막대그래프로 크기 변화를 출력해줘
가독성있게 한 화면에서 슬라이더바 입력, 종속변수 값이 출력되어야 해
5. 기타
- 추가로 필요한 기능이 있다면 나한테 물어보고 추가해줘
 - 웹 시스템은 한글로 구현해줘

웹 개발 실습 3. 발전기 복합출력 예측

▶ 프롬프트 만들기



커서로 예측 시스템을 만들건데, 아래 내용을 커서에 넣을 프롬프트로 작성해줘
파이썬 Flask를 활용한 예측 시스템을 개발하고 싶어

1. 개발환경

- Flask 패키지 사용
- 데이터 처리: pandas 패키지
- 그래프: matplotlib (한글 깨짐 방지 설정), Plotly, Dash
- 세션 관리: SQLite3 (단계별 결과 저장, 전 단계 결과를 다음 단계에서 사용)
- 파일 업로드: csv, excel 지원
- 전체 시스템 한글로 구현

2. 노드설명

- 파일 업로드부터 시뮬레이션까지 순차적으로 진행
- 각 단계의 결과는 SQLite3에 저장되며, 다음 단계에서 이전 결과를 불러와 사용
- 파일 업로드: csv, excel 업로드 후 pandas로 10개 행 출력 및 기초 통계량 출력
- 변수 선택: 종속변수 및 독립변수 선택, 이후 분석에 적용
- 결측치 제거: 결측치 행 탐색 및 제거, 총 행 수 / 제거 행 수 / 잔여 행 수 출력
- 이상치 탐색:
 - 변수별 ± 3 표준편차 범위 밖 데이터 탐색
 - 변수별 이상치 제거 여부 선택 및 제거
- 탐색적 분석:
 - 히스토그램: 변수 선택 후 히스토그램 + 밀도 곡선 출력
 - 산점도: 다중 변수 선택 후 산점도 출력
 - 상관관계(히트맵): 전체 변수 상관관계 히트맵 출력

- 회귀분석:

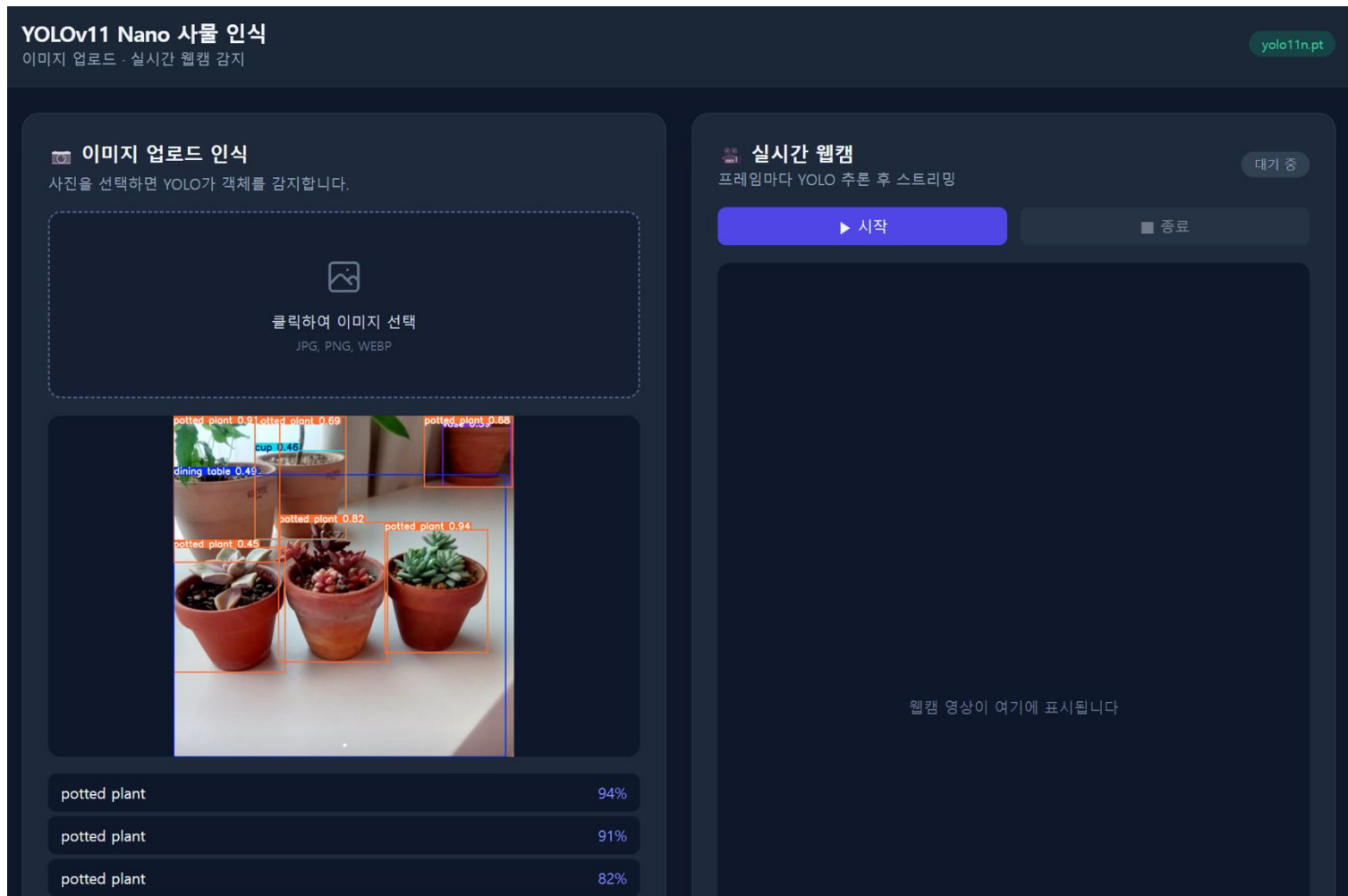
- 전진선택법 / 후진제거법 / 릿지 / 라쏘 선택
- 회귀모형 및 예측 정확도 출력, 최종 모델 선택
- 모델 검증: 잔차분석 및 적합도 검정 결과 출력
- 시뮬레이션: 선택된 회귀모형으로 시뮬레이션, 독립변수 슬라이더바 입력, 종속변수 값 및 막대그래프 출력
- 단계별 순차 진행 (전 단계 완료 후 다음 단계 활성화)
- 추가 기능 필요 시 사용자에게 확인 후 추가

3. UI설명

- 컬러: 연한 녹색, 연한 파란색, 회색- 화면 2분할: 좌측 20% / 우측 80%
- 우측 화면: 상단 10% (현재 단계 표시) / 하단 90% (서브메뉴 및 실행 결과)
- 좌측 메뉴 구성: 1.파일업로드 / 2.변수선택 / 3.결측치제거 / 4.이상치탐색 / 5.탐색적 분석 / 6.회귀분석 / 7.모델검증 / 8.시뮬레이션
- 좌측 메뉴: 현재 단계 진하게, 나머지 연하게 표시, 단계 완료 시 다음 메뉴 활성화
- 우측 상단: 현재 메뉴 단계명 표시
- 우측 하단: 서브메뉴 및 실행 결과 표시
- 탐색적 분석 서브메뉴: 히스토그램 / 산점도 / 상관관계(히트맵) 탭
- 회귀분석 서브메뉴: 전진선택법 / 후진제거법 / 릿지 / 라쏘 탭
- 시뮬레이션: 회귀식 출력, 슬라이더바 입력, 종속변수 값 및 막대그래프 한 화면 구성 (Plotly/Dash)

웹 개발 추가 예제. 영상 이미지 분석

▶ 실습목표: 카메라에 물체를 인식하여 구분하기



1. 기능 : 사물 인식
2. 메뉴 : 사진 업로드, 카메라 인식
3. 데이터베이스 : 없음
4. 개발환경 : flask, yolo 11 nano, ultralytics

웹 개발 추가 예제. 영상 이미지 분석

▶ 프롬프트 초안 : 카테고리 만들기



YOLOv11 Nano 기반의 가장 단순한 사물 인식 웹페이지를 만들거야

- flask로 최대한 단순하게 만들거야
- 필요한 패키지, 기능과 UI에 대해서 설명해줘

웹 개발 추가 예제. 영상 이미지 분석

▶ 프롬프트 만들기



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어
YOLOv11 Nano 기반의 가장 단순한 사물 인식 웹페이지를 만들거야

1. 개발 환경

- Flask, ultralytics(yolo11n.pt), opencv-python

2. 노드설명

- 이미지 업로드: 이미지를 올리면 YOLO 바운딩 박스가 그려진 결과 이미지 표시
- 웹캠 스트리밍: Flask 비디오 스트리밍을 활용해 실시간으로 YOLO 인식 화면 출력
- 복잡한 기능 없이 가장 단순하고 직관적인 코드로 한 번에 만들어줘

3. UI

- HTML 단일 페이지 (Tailwind CSS로 깔끔하고 미니멀하게 디자인)
- 이미지 업로드 구역과 실시간 웹캠 화면 구역을 직관적으로 배치

웹 개발 추가 예제. 영상 이미지 분석

▶ 프롬프트 최종

Prompt

YOLOv11 Nano와 Flask를 활용한 가장 단순하고 직관적인 '사물 인식 웹 애플리케이션'을 만들어줘. 복잡한 클래스 구조나 불필요한 추상화 없이, 파일 하나만 봐도 흐름이 다 보일 정도로 직관적이고 깔끔한 코드를 원해.

아래 요구사항과 개발 환경을 엄격히 준수해서 한 번에 동작하는 코드를 작성해줘.

1. 개발 환경 및 라이브러리

- Backend: Flask
- Vision & AI: ultralytics (yolo11n.pt 자동 다운로드 활용), opencv-python
- Frontend: Single HTML (Tailwind CSS 활용, CSS 파일 분리 금지)

2. 폴더 구조

기본적인 Flask 구조를 따라 폴더와 파일을 생성해줘.

- app.py (Flask 메인 백엔드 소스)
- templates/index.html (프론트엔드 단일 페이지)

3. 핵심 기능 요구사항

[기능 1: 이미지 업로드 인식]

- 사용자가 이미지를 업로드하면 백엔드에서 YOLOv11n 모델로 객체를 감지함.
- 감지된 바운딩 박스와 클래스명이 그려진 결과 이미지를 웹 화면에 즉시 표시함. (복잡한 파일 저장 로직 대신, 메모리 내 또는 static 폴더를 활용한 심플한 방식으로 처리)

[기능 2: 웹캠 실시간 스트리밍]

- OpenCV(cv2.VideoCapture)를 사용해 웹캠 영상을 실시간으로 캡처함.
- 프레임마다 YOLOv11n 인식을 수행하고 바운딩 박스를 그린 뒤, Flask의 Multipart Response(generator 활용)를 통해 실시간 비디오 스트리밍 프레임으로 화면에 출력함.

4. UI/UX 디자인 요구사항

- `templates/index.html` 단일 파일로 작성하고 UI는 Tailwind CSS CDN을 활용해 깔끔하고 현대적인 대시보드 스타일로 디자인해줘.
- 좌측/우측 또는 상단/하단으로 2분할 레이아웃을 적용하여 [이미지 업로드 기능 구역]과 [실시간 웹캠 스트리밍 구역]이 직관적으로 한눈에 보이게 배치해줘.
- 웹캠 스트리밍은 '시작/종료' 상태를 직관적으로 알 수 있거나, 페이지 접속 시 바로 부드럽게 출력되도록 처리해줘.

5. 코드 작성 가이드라인

- 에러를 유발할 수 있는 복잡한 DB나 외부 종속성은 모두 배제하고 오직 Flask, OpenCV, Ultralytics 기본 기능만 사용해줘.
- app.py와 index.html 전체 코드를 생략 없이 '완전한 형태(Complete Code)'로 출력해줘.

실습주제 예시

- (웹) 메모, Todo 작성(DB)
- (웹) 일정 관리
- (웹) 레트로게임 : 테트리스, 너구리, 갤러그, 미로찾기 + 알파
- (웹) 오늘날씨~일주일날씨 대시보드: 추천 옷차림+챙길 물품
- (웹) 업로드한 파일 내용 정리 요약해주는 대시보드
- (웹) 회의록을 바탕으로 회의내용을 정리하고 누가 무엇을 언제까지 해야 하는지 요약/정리
- (앱, 웹) Youtube 링크를 입력하면 영상/음원/영상+음원을 다운로드 및 저장
- (앱) 계산기
- (앱) 동호회 멤버들끼리 매일 실천하는 챌린지 앱

M2. 바이트코딩 활용 앱/웹 개발

6. 바이트코딩 스킬 활용

Spec-Driven Development (SDD)

명확한 마크다운 스펙을 먼저 작성하고, AI코딩 에이전트가 그것을 구현하게 하는 방식

■ 3개 MD 파일 적용

구분	API
architecture.md	프로젝트의 전체 아키텍처와 기술적인 설계 기준을 정의하는 파일
prompts.md	AI가 프로젝트를 개발할 때 따라야 하는 작업 규칙과 프롬프트를 정의하는 파일
ui.md	UI/UX와 디자인에 대한 기준을 정의하는 파일

architecture.md

- 프로젝트 목적
- 핵심 기능
- 기술 스택
- 디렉터리 구조
- 데이터 모델
- API 설계
- 상태 관리 방식
- 인증 및 권한
- 비즈니스 로직
- 개발 제약사항

prompts.md

- AI 역할(Role)
- 개발 원칙
- 코드 스타일
- 구현 우선순위
- 리팩토링 기준
- 에러 처리 규칙
- 테스트 작성 기준
- 응답 형식
- 금지사항
- 자주 사용하는 프롬프트 템플릿

ui.md

- 디자인 컨셉
- 컬러 시스템
- 타이포그래피
- 컴포넌트 규칙
- 레이아웃
- 반응형 기준
- 페이지별 UI 요구사항
- 인터랙션
- 접근성 기준

SDD 방법론 적용 효과



AI 기억력 보완

- MD = AI의 장기 기억 장치. @참조 한 번으로 모든 맥락을 AI에게 전달. 설명 반복 끝!



일관성 유지

- 스타일·구조·규칙이 흔들리지 않음. 10번째 기능도 1번째와 같은 스타일로 생성됨



시간 절약

- 매번 설명 반복 없이 @참조 한 번으로 끝. 설명 시간 대신 기능 추가에 집중 가능



협업 가능

- 팀원이 MD 3개만 읽으면 프로젝트 파악 완료. 온보딩 시간 대폭 단축



내 자산이 됨

- MD는 코드보다 오래 쓰는 설계 문서. 다음 프로젝트에도 템플릿으로 재사용 가능

MD 파일 3개 작성에 드는 1시간이 이후 수십 시간을 절약!

architecture.md 예시

```
# architecture.md - 기술 설계도

## 기술 스택
- - Backend : Python 3.x + Flask
- Frontend : 순수 HTML + CSS + JavaScript
- 통신 방식: JavaScript fetch API → Flask JSON 응답
- Database : SQLite (todos.db 파일, 자동 생성)

## 폴더 구조
/todo-app
  app.py          → Flask 앱 메인 파일 (라우트 + DB 처리)
  todos.db        → SQLite 데이터베이스 파일 (자동 생성)
  /static
    index.html    → 단일 페이지 (HTML)
    style.css     → 전체 스타일
    main.js       → 할일 추가·삭제·체크·필터 JavaScript

## DB 스키마 - todos 테이블
id(uuid PK) | title(text) | category(text) | due_date(date)
is_done(boolean DEFAULT false) | created_at(timestamp)

## API 설계
GET /api/todos      → 전체 조회      POST /api/todos      → 추가
PATCH /api/todos/[id] → 완료 토글    DELETE /api/todos/[id] → 삭제
```

prompts.md 예시

```
# prompts.md - AI 지시 규칙 & 프롬프트 레시피

## AI가 항상 지킬 규칙
- Python 코드는 함수 단위로 깔끔하게 분리
- Flask API는 항상 JSON으로 응답 { "data": ..., "error": null }
- 에러는 try-except로 처리, 에러 메시지는 한국어로
- HTML은 /static/index.html 한 파일
- CSS는 /static/style.css 한 파일
## 프롬프트 템플릿

### 새 기능 추가 (Plan 모드 먼저)
@ui.md @architecture.md @prompts.md 를 참고해서
[기능명] 기능을 추가하는 계획을 만들어줘.
- 수정할 파일 목록
- 각 파일에서 변경할 내용 요약
계획이 맞으면 코드를 작성해줄게.
```

ui.md 예시

```
# ui.md - 화면 설계도

## 색상 & 폰트
- 주색: #2563EB (파랑)   배경: #FFFFFF   텍스트: #1A1A1A
- 완료 항목: #94A3B8 (회색)   경고(마감 지남): #DC2626 (빨강)
- 폰트: Pretendard / 제목 20px Bold / 본문 14px Regular

## 메인 페이지 (/)
- 헤더: 로고 "오늘의 할일" (좌) + "+ 추가" 버튼 (우, 파란색)
- 카테고리 탭: [전체] [업무] [개인] [공부]
- 빈 상태: "할일이 없어요! 추가해보세요 😊"

## 할일 카드 컴포넌트
- 체크박스(좌) + 제목 + 카테고리 배지 + 마감일(우)
- 완료 시: 취소선 + 전체 회색   마감 지남: 마감일 텍스트 빨간색

## 컴포넌트 규칙
- 버튼: rounded-lg px-4 py-2   카드: shadow-sm border rounded-xl p-4
```

3개 MD 파일 완성! → Chat에서 @참조하며 기능 만들기 시작!

마크다운(MarkDown)

- 특수기호로 문서 구조를 표현하는 가벼운 문법
- 사용문법
 - # 하나면 큰 제목 됨
 - **별표 두 개**면 굵게 됨
 - -(하이픈)이면 목록 됨
 - 문법이 단순해서 실험 기록에 잘 맞음

"글을 꾸미는 것"보다 "구조를 만드는 것"에 가까움

참고. 마크다운 예시

기능	입력	결과
제목(H1)	# 제목	제목
제목(H2)	## 제목	제목
제목(H3)	### 제목	제목
굵게	**굵은 글씨**	굵은 글씨
기울임	*기울임*	<i>기울임</i>
취소선	~~삭제~~	삭제
굵게 + 기울임	***강조***	<i>강조</i>
인라인 코드	`print()`	print()
코드 블록	python print("Hello") 	코드 블록 생성
순서 없는 목록	- 사과	사과
순서 있는 목록	1. 사과	1. 사과

기능	입력	결과
체크박스	- [x] 완료	☒ 완료
체크박스	- [] 미완료	☐ 미완료
링크	[OpenAI](https://openai.com)	OpenAI
이미지	![설명](image.png)	이미지 표시
인용문	> 인용문	> 인용문
수평선	---	_____
표	이름 나이	표 생성
이스케이프	*별표*	*별표*

md 파일 작성(프롬프트)

■ STEP 1 — Cursor Chat (Ctrl+L) 열고 아래 프롬프트 입력



나는 "오늘의 할일" 앱을 만들 거야.

기능: 할일 추가, 완료 체크, 삭제, 카테고리(업무/개인/공부) 분류, 마감일 설정

- Backend: Python Flask

- Frontend: 순수 HTML + CSS + JavaScript

- Database: SQLite3

아래 3개 MD 파일 초안을 각각 작성해줘.

파일에 바로 붙여넣을 수 있게 마크다운 형식으로, 구체적으로 작성해줘.

1. ui.md: 페이지 목록, 컴포넌트 구조, 색상·폰트 규칙

2. architecture.md: 기술 스택, 폴더 구조, API 엔드포인트, DB 스키마

3. prompts.md: AI 코딩 규칙, 자주 쓸 프롬프트 템플릿

■ STEP 2 — 생성된 3개 파일 확인



프로젝트이름

/todo-app/

├── ui.md

├── architecture.md

└── prompts.md

Skill이란?

■ AI 코딩 도구(Cursor, Codex 등)가 특정 작업을 수행할 때 참조하는 구조화된 지식 문서(작업 매뉴얼)

1

매번 고민할 필요 없음

"어떻게 물어보지? 어떤 프롬프트를 쓰지?" → 스킬이 그 역할을 대신 수행

2

비개발자도 전문가 수준

질문·문서화를 비개발자도 바로 전문가 수준으로 제작 가능

3

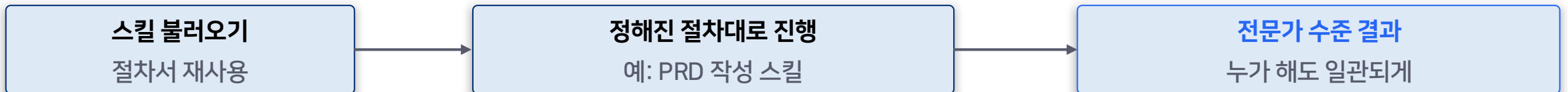
검증된 절차 재사용

좋은 절차는 이미 스킬로 만들어져 있음 → 잘 만든 것을 활용하는 것도 실력

Before (스킬 없이)



After (스킬 활용)



Skill 실제 파일 구조 예시

■ Markdown 파일(.md) 형태

```
# grill.me Skill
```

```
## 역할
```

당신은 아이디어 검증 전문가임.

```
## 규칙
```

1. 아이디어 입력받으면 7가지 질문 순서대로 함.
2. 질문은 한 번에 하나씩만 함.

```
## 질문 목록
```

1. 누가 씀? 2. 어떤 문제를 해결함?

```
## 출력 형식
```

****문제 정의****: {답변 기반으로 정리}

****대상 사용자****: {답변 기반으로 정리}

```
---
name: to-prd
description: Turn the current conversation into a PRD and publish
disable-model-invocation: true
---

This skill takes the current conversation context and codebase un
The issue tracker and triage label vocabulary should have been pro

## Process

1. Explore the repo to understand the current state of the codebas
2. Sketch out the seams at which you're going to test the feature.
Check with the user that these seams match their expectations.
3. Write the PRD using the template below, then publish it to the

<prd-template>

## Problem Statement

The problem that the user is facing, from the user's perspective.

## Solution

The solution to the problem, from the user's perspective.

## User Stories

A LONG, numbered list of user stories. Each user story should be

1. As an <actor>, I want a <feature>, so that <benefit>
```

참고. 주요 Skill

기능	특징	주요기능	웹주소
grill-me	코드 작성 전 설계를 집요하게 검증하는 사전 검토 도구	① 계획/설계를 질문으로 집요하게 검증 ② 결정 트리 분기별로 하나씩 해소 ③ 코드베이스 탐색으로 중복 질문 자동 생략	skills.sh/mattpocock/skills/grill-me
to-prd	추가 질문 없이 대화만으로 PRD 완성 후 이슈 자동 등록	① 대화 맥락만으로 PRD 자동 생성 ② 사용자 스토리·구현 결정·테스트 계획 포함 ③ 이슈 트래커에 needs-triage 라벨로 자동 게시	skills.sh/mattpocock/skills/to-prd
diagnose	6단계 체계로 어려운 버그를 끝까지 추적하는 디버깅 규율	① 피드백 루프 구축이 핵심 — 재현 가능한 신호 확보에 최우선 ② 3~5개 가설을 먼저 나열하고 순위 매긴 뒤 검증 ③ 수정 전 회귀 테스트 작성 → 수정 → 디버그 코드 전부 제거	skills.sh/mattpocock/skills/diagnose
frontend-design	Anthropic 공식, 디자인 결정을 AI가 자동 판단	① UI 색상/타이포그래피/레이아웃 가이드 적용 ② 디자인 일관성 자동 유지 ③ 컴포넌트 스타일 결정 지원	skills.sh/anthropics/skills/frontend-design
vercel-react-best-practices	Vercel 공식, 배포 환경까지 고려한 React 패턴	① Vercel 최적화 React 패턴 ② 성능/렌더링 모범 사례 ③ 배포 친화적 코드 구조	skills.sh/vercel-labs/agent-skills/vercel-react-best-practices
web-design-guidelines	접근성·반응형·UX를 종합한 웹 디자인 원칙	① 웹 접근성 및 반응형 원칙 ② UX 중심 설계 가이드 ③ 크로스브라우저 호환성 고려	skills.sh/vercel-labs/agent-skills/web-design-guidelines
systematic-debugging	감이 아닌 체계적 가설 검증으로 버그 추적	① 버그 원인을 단계별로 좁혀가는 프로세스 ② 가설 설정 → 검증 → 범위 축소 반복 ③ 디버깅 과정 문서화 지원	skills.sh/obra/superpowers/systematic-debugging

참고. 주요 Skill

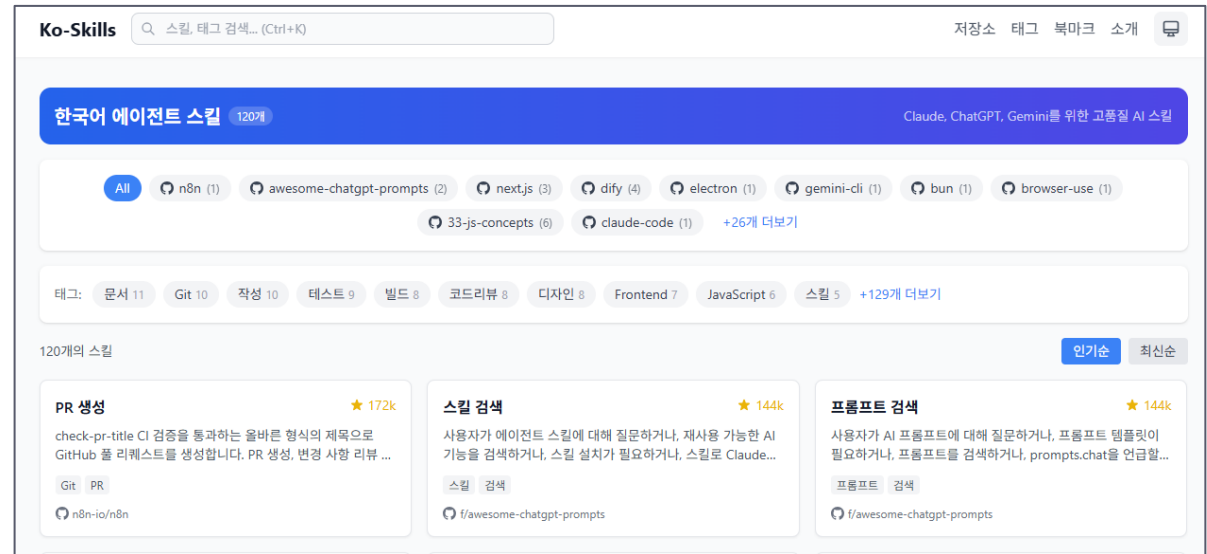
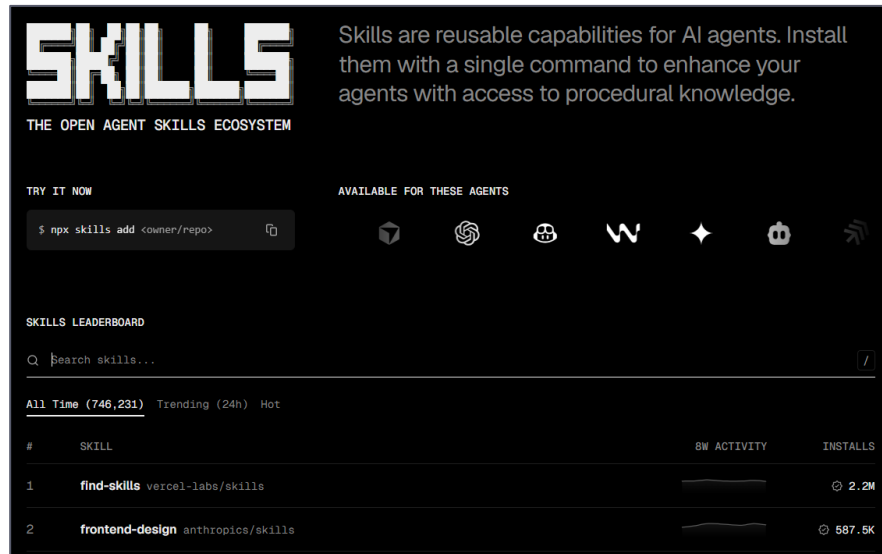
기능	특징	주요기능	웹주소
shadcn	shadcn 공식, 컴포넌트 설치부터 커스터마이징까지	① shadcn/ui 컴포넌트 올바른 설치/사용법 ② 테마 커스터마이징 가이드 ③ Radix UI 기반 접근성 패턴	skills.sh/shadcn/ui/shadcn
tdd	테스트를 먼저 쓰는 Red-Green-Refactor 루프 강제	① Red-Green-Refactor 루프 적용 ② 테스트 먼저 작성 후 구 ③ 통합·단위 테스트 전략	skills.sh/mattpocock/skills/tdd
pdf	Anthropic 공식, PDF 생성부터 추출까지 올인원	① PDF 생성/병합/분할 ② 텍스트·이미지 추출 ③ 폼 필드 작성 및 암호화	skills.sh/anthropics/skills/pdf
pptx	Anthropic 공식, PPTX 생성·편집·파싱 전반 지원	① 슬라이드 덱 생성 및 편집 ② 레이아웃·노트·템플릿 처리 ③ 기존 PPTX 파싱 및 내용 추출	skills.sh/anthropics/skills/pptx
seo-audit	SEO 구조를 자동 분석해 우선순위별 개선안 제시	① 페이지 SEO 구조 분석 ② 개선 우선순위 제안 ③ 메타/구조화 데이터 점검	skills.sh/coreyhaines31/marketingskills/seo-audit
docx	Anthropic 공식, Word 문서 생성·편집 전담	① Word 문서 생성 및 서식 적용 ② 목차·페이지번호·헤더 처리 ③ 기존 .docx 편집 및 내용 추출	skills.sh/anthropics/skills/docx

Skill 활용 방법

■ 활용 순서

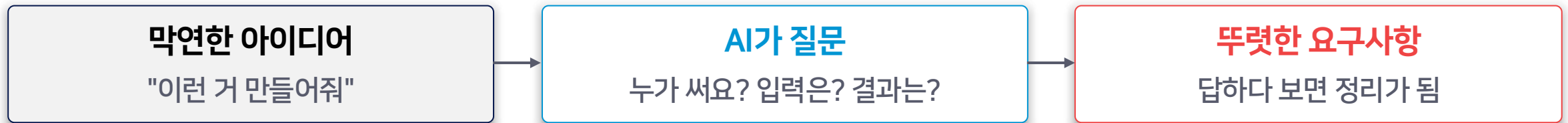


■ Skill 검색 사이트 : <https://www.skills.sh>, <https://ko-skills.pages.dev>



grill-me 스킬을 활용한 아이디어 구체화

- 구체화가 필요한 이유 : 막연한 아이디어 한 줄로 요청시, 빈칸은 AI가 제멋대로 채움
- grill-me 스킬을 활용한 아이디어 구체화
 - 내가 AI에게 요청하는 게 아니라 **AI가 나에게 필요한 기술을 물어봄**



- 스킬 설치 링크: <https://www.skills.sh/mattpocock/skills/grill-me>

1. Cursor Chat에 grill-me 스킬을 활용해서 아이디어만 입력하면 됨

 /grill-me 냉장고 재료 입력하면 만들 수 있는 요리 추천하는 앱 만들고 싶어

2. grill-me가 던지는 질문들

- 누가 씬?
- 어떤 문제가 있음?
- 기존 서비스랑 뭐가 다름?
- 젤 중요한 기능 하나만 꼽으면?
- 첫 번째 버전(MVP)은 어디까지임?
- 성공 기준이 뭐임?

3. 대답하다 보면 아이디어가 구체화되고, 최종적으로 아이디어 검증 문서가 나옴

- 문제 정의 / 대상 사용자
- 사용자 니즈 / 핵심 가치
- MVP 범위 / 제외 기능
- 성공 기준 / 위험 요소

to-spec(PRD) 스킬을 활용한 요구사항 생성

- PRD(Product Requirements Document)
- "무엇을 만들 것인가"를 적은 요구사항 목록 문서임 → AI가 판단할 근거가 되는 문서

항목	개념	예시(쇼핑앱)
왜	서비스를 만드는 이유와 목표를 정의함	모바일 중심 고객이 빠르고 간편하게 상품을 구매할 수 있는 환경 제공이 목적임
문제	현재 사용자가 겪는 불편이나 해결해야 할 문제를 정의함	오프라인·SNS 주문만 가능함 주문·재고를 수기로 관리하여 실수 잦음
사용자	서비스를 사용하는 대상과 역할을 정의함	일반 고객(상품 구매), 재구매 고객(빠른 재주문), 관리자(상품·주문·재고 관리)
범위	개발에 포함할 기능과 제외할 기능을 정의함	포함: 회원가입, 상품 조회, 장바구니, 결제, 관리자 화면제외: 리뷰, 채팅, 다국어 지원
동작 방식	서비스가 어떤 규칙과 흐름으로 동작하는지 정의함	결제 실패 시 장바구니 유지함 재고 부족 시 결제 전에 안내함 주문 완료 시 재고 자동 차감함
성공 기준	기능이 성공했는지 판단하는 기준을 정의함	결제까지 클릭 3회 이내 완료 목표 재구매율 20% 이상 달성 목표

to-spec(PRD) 스킬을 활용한 요구사항 생성

- PRD만 가지고 바로 코딩할 때 흔히 빠지는 부분 → 보완

누락 항목	발생되는 문제
데이터 모델/스키마	필드, 타입, 관계가 불명확해 AI가 임의로 설계함
에러/예외 처리 기준	실패 메시지, 재시도 방식 등을 AI가 임의로 결정
인증/권한 경계	누가 뭘 보고 수정할 수 있는지 불명확
비기능 요구사항	성능, 동시성, 데이터 규모(100명 vs 100만명)에 따라 설계가 완전히 달라짐
기존 시스템 연동 방식	기존 API·DB·인증 시스템과의 연결 방식 누락

to-spec(PRD) 스킬을 활용한 요구사항 생성

- to-spec: 아이디어 검증 결과를 바탕으로 제품 요구사항 문서(PRD)를 생성해 주는 스킬
- grill-me 스킬에서 나온 결과물을 바탕으로 개발에 필요한 기준들이 정리되어 파일로 나옴



지금까지 내용을 SPEC.md파일에 작성해줘

- 스킬 설치 링크: <https://skills.sh/mattpocock/skills/to-spec>

skills / mattpocock / skills / to-spec

to-spec

INSTALLATION Command Prompt

```
$ npx skills add https://github.com/mattpocock/skills --skill to-spec
```

SKILL .md

This skill takes the current conversation context and codebase understanding and produces a spec (you may know this document as a PRD). Do NOT interview the user — just synthesize what you already know.

The issue tracker and triage label vocabulary should have been provided to you — run `/setup-matt-pocock-skills` if not.

Process

1. Explore the repo to understand the current state of the codebase, if you haven't already. Use the project's domain glossary vocabulary throughout the spec, and respect any ADRs in the area you're touching.
2. Sketch out the seams at which you're going to test the feature. Existing seams should be preferred to new ones. Use the highest seam possible. If new seams are needed, propose them at the highest point you can. The fewer seams across the codebase, the better - the ideal number is one.

Check with the user that these seams match their expectations.

3. Write the spec using the template below, then publish it to the project issue tracker. Apply the `ready-for-agent` triage label - no need for additional triage

실습 예제. 냉장고 재료로 요리 추천하는 앱

아이디어: 냉장고에 남은 재료를 입력하면 만들 수 있는 요리를 추천해주는 앱을 만들고 싶다.

■ 커서에 입력할 프롬프트

NO.	프롬프트
1	/grill-me 냉장고의 재료로 만들 수 있는 요리 추천 앱 만들고 싶어
2	1번부터 질문을 체크할 수 있게 물어봐 줘
3	/to-spec 지금까지 내용을 SPEC.md파일에 작성해줘
4	SPEC.md 내용 검토 후 필요한 내용 추가
5	@SPEC.md 내용으로 코드를 작성해

#참고. Plan 모드 사용 및 검토 사항

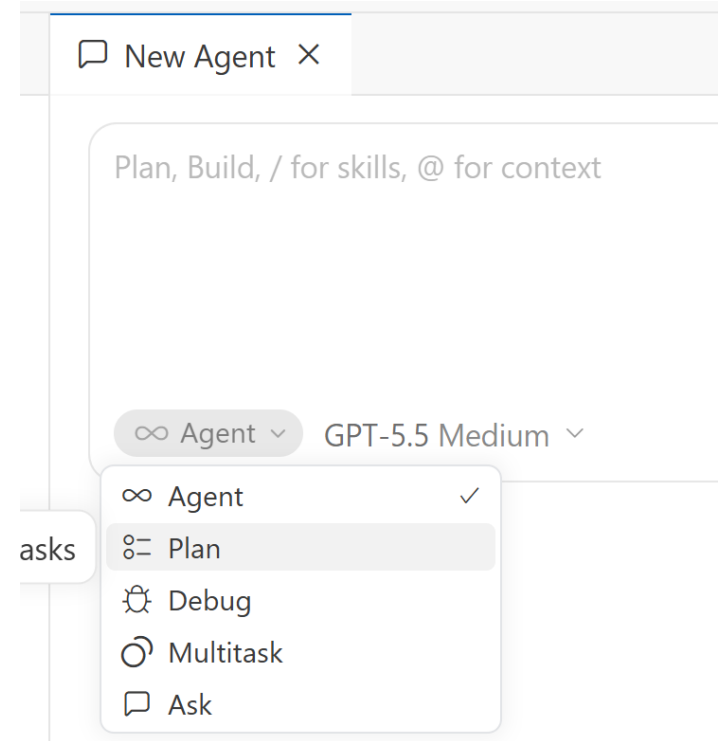
- AI가 코드를 바로 짜지 않고, "이렇게 만들겠습니다"라는 계획을 먼저 보여주는 모드

- plan모드 사용법

1. Composer 열기 (Ctrl+I)
2. 상단 드롭다운에서 "Plan" 선택
3. @MD 파일 참조 + 프롬프트 입력
4. AI가 계획을 텍스트로 보여줌 (코드는 아직 없음)
5. 계획 검토 → 문제 있으면 수정 요청 → 문제 없으면 "진행해줘" 또는 Agent 모드로 전환
6. 실제 코드 생성

- Plan 모드에서 확인할 사항

- 만들려는 파일 목록이 맞는가?
- 수정하려는 파일이 올바른가? (엉뚱한 파일 건드리면 안 됨)
- 접근 방식이 architecture.md와 일치하는가?
- 빠진 기능이나 조건은 없는가?



#참고. Plan 모드 예시

■ Plan 모드 프롬프트

@ui.md @architecture.md @prompts.md

할일 추가 기능을 만들 계획을 세워줘.

- 어떤 파일을 만들고 수정하는지
- 각 파일에서 무엇을 작성하는지
- Flask API와 JavaScript가 어떻게 연결되는지 계획만 먼저 보여줘. 코드는 아직 작성하지 마.

■ Plan 검토 체크리스트

파일 관련

- [] 만들거나 수정하는 파일 목록이 architecture.md와 일치하는가?
- [] 건드리면 안 되는 파일을 건드리지 않는가?
- [] 새로 만드는 파일이 올바른 폴더에 위치하는가?

기능 관련

- [] 내가 요청한 기능이 계획에 모두 포함됐는가?
- [] 빠진 조건이나 엣지 케이스는 없는가?
- [] API URL이 architecture.md의 설계와 맞는가?

흐름 관련

- [] JavaScript → Flask API → SQLite 흐름이 맞는가?
- [] 성공/실패 처리가 계획에 포함됐는가?
- [] 화면 업데이트(DOM) 방식이 자연스러운가?

■ 체크리스트 검토 후

- 이상 있으면: 3번 항목에서 API URL을 /api/todos로 바꿔줘. 그리고 삭제 확인 창(confirm)이 빠져있는데 추가해줘.
- 이상 없으면: 계획대로 진행해줘

M3. 바이트코딩 활용 RAG 구현

7. RAG 감 잡기

AI는 왜 우리 회사 일을 모를까

▶ LLM이 배운 것과 우리가 묻는 것 사이의 간극



LLM이 배운 것

- 인터넷에 공개된 문서 · 책 · 코드
- 학습을 끝낸 시점까지의 정보
- 누구에게나 똑같이 주어지는 일반 지식

→ 세상의 평균적인 지식



우리가 묻는 것

- 우리 회사 연차·출장 규정
- 우리 팀이 지난달 쓴 보고서
- 어제 회의록, 이번 분기 제품 사양서

→ 인터넷에 없는, 우리만의 지식

요즘 AI는 이 경계를 스스로 압니다. 모르면 "모른다"고 말합니다.
그리고 "무엇을 봐야 하는지"까지 알려줍니다.

✅ 답할 수 있는 것

"근로기준법상 연차는 15일입니다" 공개된 법·기준·일반 상식 → 학습 데이터에 들어 있음

❌ 답할 수 없는 것

"재택근무 신청 기한은 회사마다 다릅니다. 확인할 수 없습니다" → 우리 문서에만 있는 정보

🔑 AI가 덧붙이는 말

"사내 복리후생·근무제도 안내를 확인하셔야 합니다" → 필요한 게 뭔지는 AI도 압니다

【실습 ①】 AI가 답할 수 있는 것과 없는 것

▶ 전부 휴가·근무 질문인데 반응이 갈립니다

M3_07-1_no_doc 폴더만 열고 새 채팅에서 시작 — 상위 폴더를 열면 Cursor가 옆 폴더의 문서를 찾아냅니다

A · 공개된 지식

A-1 "근로기준법상 1년 이상 근무한 직원의
연차유급휴가는 며칠인가요?"

A-2 "주 52시간제에서 1주 연장근로 한도는?"

→ 답합니다. 법과 공개 기준은 학습되어 있습니다

B · 우리 문서에만 있는 지식

B-1 "재택근무는 며칠 전까지 신청하고
한 달에 최대 며칠까지?"

B-2 "하계휴가 지원금은 얼마인가요?"

→ 못 답합니다. "확인할 수 없습니다"

#	AI가 답을 줬나?	답을 줬다면 근거는? / 못 줬다면 이유는?
A-1 / A-2	답함 / 못 함	
B-1 / B-2	답함 / 못 함	

★ 마지막 단계 — B에서 AI가 "무엇을 보라"고 했나요? 그 문장을 그대로 적으세요

"정확한 신청 기한과 월 한도는 사내 복리후생·근무제도 안내나 인사 담당 확인이 필요합니다"

해답은 「오픈북 시험」

▶ 모델을 다시 가르치는 대신, 답할 때 참고자료를 쥐여준다

❑ 폐쇄형 시험 = 그냥 LLM

- 머릿속에 든 것만으로 답한다
- 모르는 것도 일단 쓴다 (빈칸은 0점이니까)
- 우리 회사 규정은 애초에 배운 적이 없다

→ 그럴듯한 오답

❑ 오픈북 시험 = RAG

- 문제를 보고 필요한 페이지를 먼저 찾는다
- 찾은 페이지를 보면서 답을 쓴다
- 책에 없으면 "없다"고 쓸 수 있다

→ 근거 있는 답

RAG (Retrieval-Augmented Generation) · 검색 증강 생성

AI를 다시 학습시키는 것이 아니라, 질문이 들어온 순간 우리 문서에서 관련 부분을 찾아 질문과 함께 건네주는 방식

※ 재학습(파인튜닝)과의 차이 — 문서가 바뀌면 다시 학습시킬 필요 없이 창고의 파일만 갈아 끼우면 됩니다.

Retrieval · 검색

우리 문서에서 관련 부분을 찾고



Augmented · 증강

질문에 그 내용을 덧붙여서



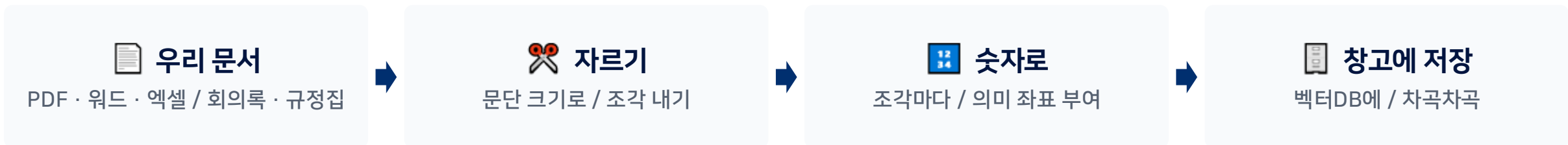
Generation · 생성

LLM이 그것을 보고 답을 만든다

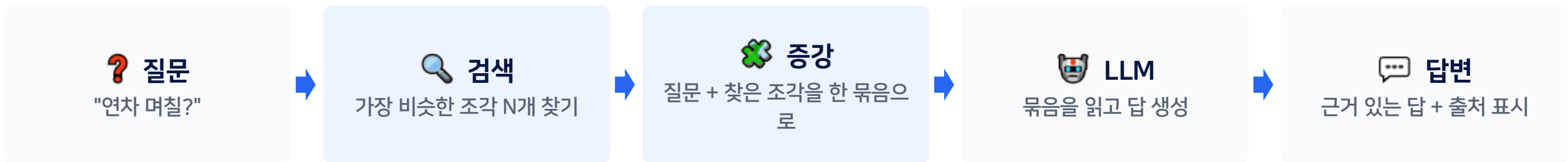
RAG는 이렇게 동작한다

▶ 사전 준비(참고 만들기)와 질문 시점(꺼내 쓰기)을 나눠서 본다

① 사전 준비 — 한 번만 해두면 되는 일



② 질문이 들어온 순간 — 매번 반복되는 일



기억할 한 문장

RAG는 AI를 똑똑하게 만드는 기술이 아니라, AI가 답하기 직전에 「정답이 적힌 페이지」를 책상 위에 올려주는 기술입니다.

그래서 문서가 최신이면 답도 최신이고, 문서에 없으면 "없다"고 답하게 만들 수 있습니다.

【실습 ②】 AI가 보라던 그 문서를 준다

▶ 실습 ①의 B 질문을 그대로 다시

STEP 1. 폴더 전환

M3_07-1_no_doc 닫고 M3_07-2_with_doc 열기 한빛소재 규정 PDF (9쪽)

STEP 2. 같은 질문 다시

PDF를 채팅창에 끌어다 놓고 실습 ①의 B-1, B-2를 토씨 하나 바꾸지 말고 그대로

STEP 3. 답변 비교

문서 없을 때 vs 있을 때 무엇이 달라졌는지 아래 표에 기록

STEP 4. 근거 요구

"몇 조 몇 항인지 알려줘"를 덧붙이고 PDF에서 직접 찾아 맞는지 확인

비교 항목	문서를 주지 않았을 때	문서를 주었을 때
B-1 재택근무	"확인할 수 없습니다"	
B-2 하계휴가 지원금	"확인할 수 없습니다"	
출처(조항 번호)	없음	

✅ 여기서 느껴야 할 것

바뀐 것은 AI가 아닙니다. 같은 AI, 같은 질문. AI 손에 문서가 있느냐 없느냐 하나뿐입니다.

★ 마지막에 꼭 물어볼 질문

"사내 어린이집 운영 시간은?" — 문서에 없는 내용입니다. "없다"고 할까요, 지어낼까요?

왜 문서를 통째로 넣으면 안 될까

▶ AI의 책상은 생각보다 좁습니다 — 컨텍스트 윈도우

AI가 한 번에 볼 수 있는 양은 정해져 있다

- 모델마다 "한 번에 읽을 수 있는 글자 수" 상한이 있음
- 이 상한에는 질문 + 첨부 문서 + 이전 대화가 모두 포함
- 넘치면 앞부분이 잘려 나가거나 오류가 납니다
- 참고: 널리 쓰이는 모델 기준 대략 A4 100~수백 장 수준

통째로 넣으면 생기는 3가지 문제

- ① 잘림 — 정작 중요한 뒷부분이 사라진다
 - ② 비용 — 넣는 글자 수만큼 요금이 붙는다
 - ③ 산만 — 관계없는 내용까지 읽느라 답이 흐려진다
- ※ 바이트코딩 중 "AI가 앞에서 정한 규칙을 까먹는" 현상도 같은 이유

그래서 "문서 전부"가 아니라 "질문과 관련된 부분만" 골라 넣습니다.
골라 넣으려면 ① 잘라두고 ② 찾을 수 있게 만들어 줘야 합니다.

① 자른다 — 청크(Chunk)

두꺼운 매뉴얼을 문단 단위로 잘라 「조각 카드」를 만드는 일

- 너무 크게 자르면 → 필요 없는 내용까지 딸려 옴
- 너무 잘게 자르면 → 문장이 끊겨 뜻을 잃음
- 조각끼리 앞뒤를 조금 겹쳐 둔다(overlap) → 문맥 유지

② 숫자로 바꾼다 — 임베딩(Embedding)

조각마다 「의미의 좌표」를 매겨 컴퓨터가 비교할 수 있게 만드는 일

- 뜻이 비슷한 문장은 좌표도 가깝게 찍힌다
- 그래서 단어가 달라도 뜻이 같으면 찾아낼 수 있다
- 이 좌표들을 담아두는 창고가 벡터DB(Vector Store)

키워드 검색과 무엇이 다른가

▶ 단어가 달라도 뜻이 같으면 찾아내는 것 — 의미 검색

	기존 키워드 검색 (Ctrl+F)	의미 검색 (임베딩 기반)
찾는 방식	글자가 똑같이 들어있는 곳을 찾음	뜻이 가까운 곳을 찾음
"연차" 검색 시	"연차"가 적힌 문장만 나옴	"휴가 일수", "유급 휴가 부여" 문장도 함께 나옴
오타·유사어	못 찾음 ("년차"로 치면 0건)	대체로 찾아냄
한계	단어를 정확히 알아야 함	엉뚱하게 가까운 문장이 섞이기도 함

비유 — 도서관에서 책 찾기

키워드 검색은 「제목에 그 단어가 든 책」을 찾아주는 사서이고, 의미 검색은 「그 주제를 다루는 책」을 찾아주는 사서입니다.
RAG의 검색 단계는 후자의 사서를 쓰기 때문에, 질문에 쓴 단어가 문서에 없어도 답을 찾아냅니다.

RAG 실습 시간에 이 단어들이 이렇게 쓰입니다

"PDF를 500자 단위로 잘라(청크) 임베딩한 다음 벡터DB에 저장하고, 질문과 가장 비슷한 조각 5개를 찾아(검색) 답하게 만들어줘"
— 이 한 문장을 Cursor에게 말할 수 있으면, 오늘 7번의 목표는 달성입니다.

【실습 ③】의미 검색 체험하기

▶ 한빛소재 규정집의 실제 문장으로 · M3_07-3_semantic_search 폴더

라운드 1 · "재택근무는 며칠 전에 신청해야 하나요?"

- ① 해외 출장자는 출발 14일 전까지 출장계획서를 제출
 - ② 임직원은 월 8일의 범위에서 재택근무를 할 수 있다
 - ③ 중식은 구내식당에서 무료로 제공하며 ...
 - ④ 재택근무를 하려는 자는 희망일의 3근무일 전까지 신청
- ①은 "○일 전까지 신청" 구조가 똑같은 함정입니다
- ②는 "재택근무"가 들어있지만 답은 아닙니다

라운드 2 · "휴가 갈 때 회사에서 돈을 주나요?"

질문에 쓰인 단어가 문서에 하나도 없습니다.

"돈"으로 Ctrl+F 하면 → 0건

그런데 AI는 하계휴가 지원금(제12조)과
리프레시 휴가 지원금(제11조)을 골라냅니다.

→ 휴가 + 돈, 두 조건을 모두 본 것입니다

	1순위	2순위	3순위	4순위	내 순위와 달랐던 지점
내 답					
AI 답					

Ctrl+F로는 0건이었을 질문에서, 의미 검색은 정확한 조항을 찾아냅니다.
단어가 아니라 뜻으로 찾는다 — 이것이 임베딩이 하는 일의 전부입니다.

RAG 5단계 · 한 장 정리

▶ 오늘 배운 것을 이 그림 한 장으로 가져가세요



① ~ ③ 은 한 번만 해두는 준비 / ④ ~ ⑤ 는 질문할 때마다 반복

이 5개 이름만 알면 충분한 이유

10번 시간에 코드를 쓰는 사람은 Cursor입니다.
여러분은 5단계 중 어디를 바꿔달라고 말하면 됩니다.
예) "조각을 더 작게 잘라줘" / "10개 찾아줘"

스스로 점검 (각 항목 한 문장으로)

- ☐ AI가 우리 회사 일을 모르는 이유를 말할 수 있다
- ☐ 문서를 주면 답이 왜 달라지는지 말할 수 있다
- ☐ 청크 / 임베딩 / 벡터DB / 검색을 각각 한 문장으로 설명한다
- ☐ 다음 시간에 Cursor에게 시킬 문장을 써 두었다

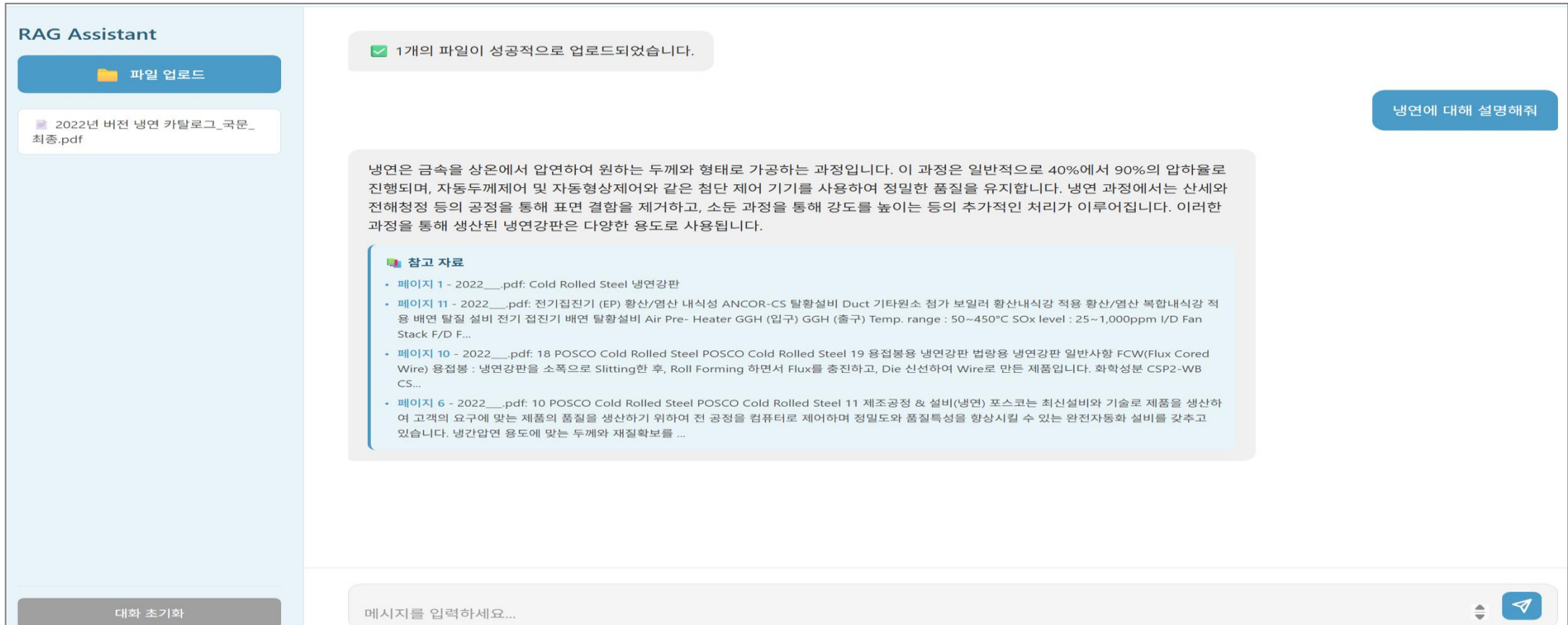
M3. 바이트코딩 활용 RAG 구현

8. RAG 구현

실습 1 : General Rag 구현

▶ 개요 : PDF 파일을 업로드하면 벡터 DB를 구성하고, 사용자의 질문에 대해 Chunk를 찾아서 답변 생성

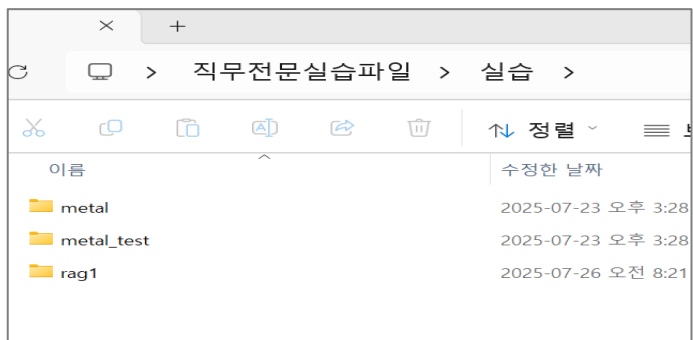
- GPT와 유사한 웹으로 구성, 텍스트는 500글자로 분할, 임베딩 모델 text-embedding-3-small , llm gpt-4o-mini
- chunk를 찾지 못하면 '모르겠습니다'라고 답변, 답변과 관련한 chunk 정보 출력, 대화 메모리 기능 사용



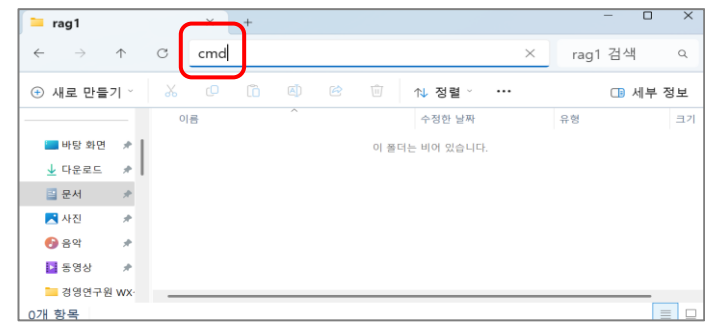
실습 1 : General Rag 구현

▶ 개발 환경 설정

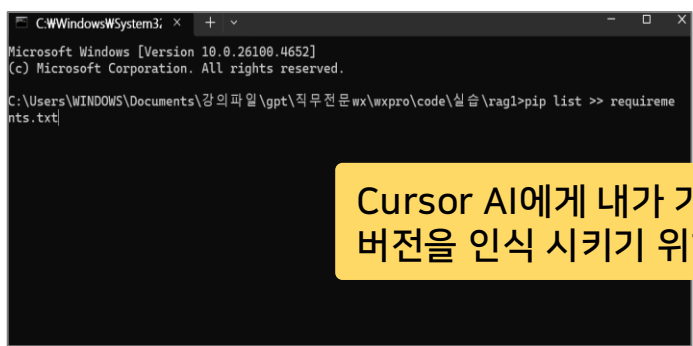
① 탐색기 : 'M3_08-1_general_rag' 폴더 확인



② 탐색기로 해당 폴더로 이동 → 주소창에 "cmd" 입력



③ "pip list >> requirements.txt" 입력



④ Cursor : File → Open folder → "M3_08-1_general_rag" 클릭



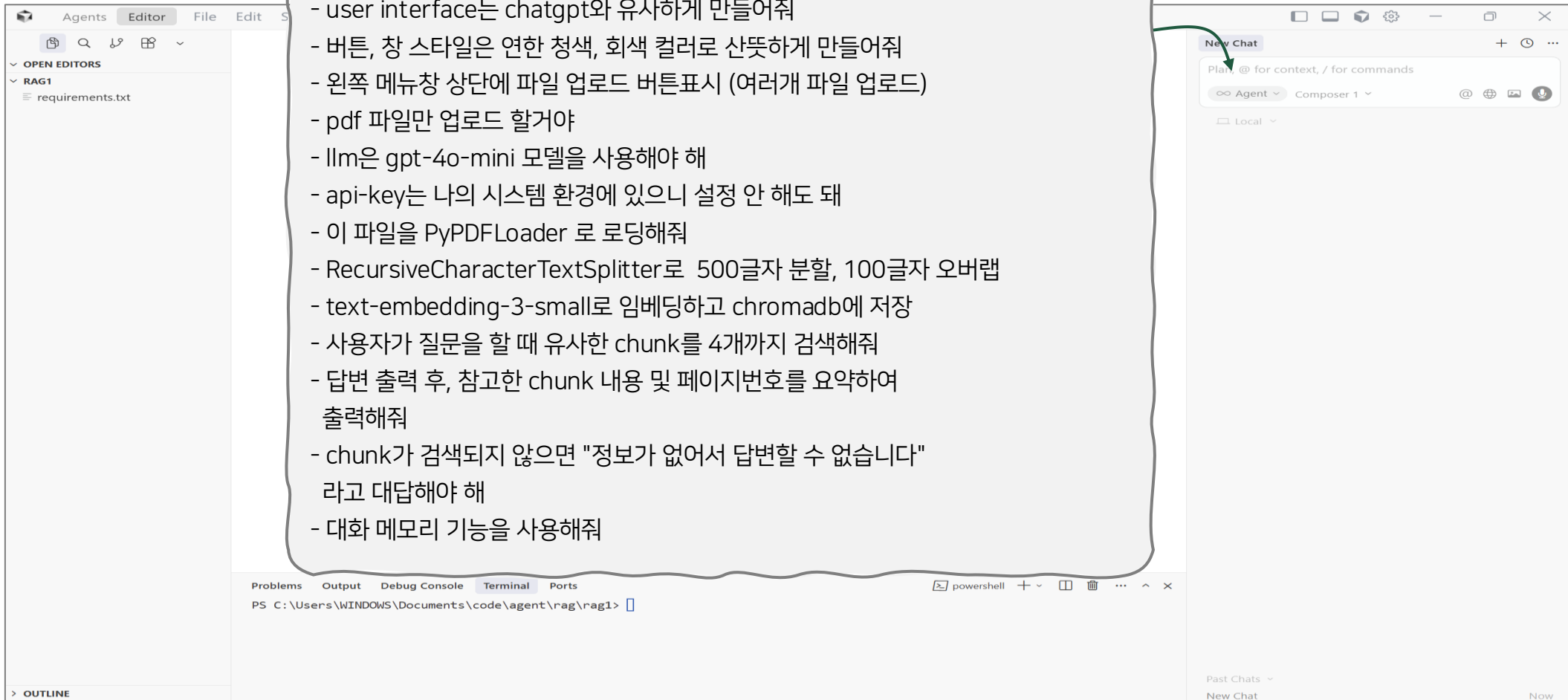
실습 1 : General Rag 구현

▶ 개발 환경 설정

⑤ -1 프롬프트 초안

Prompt

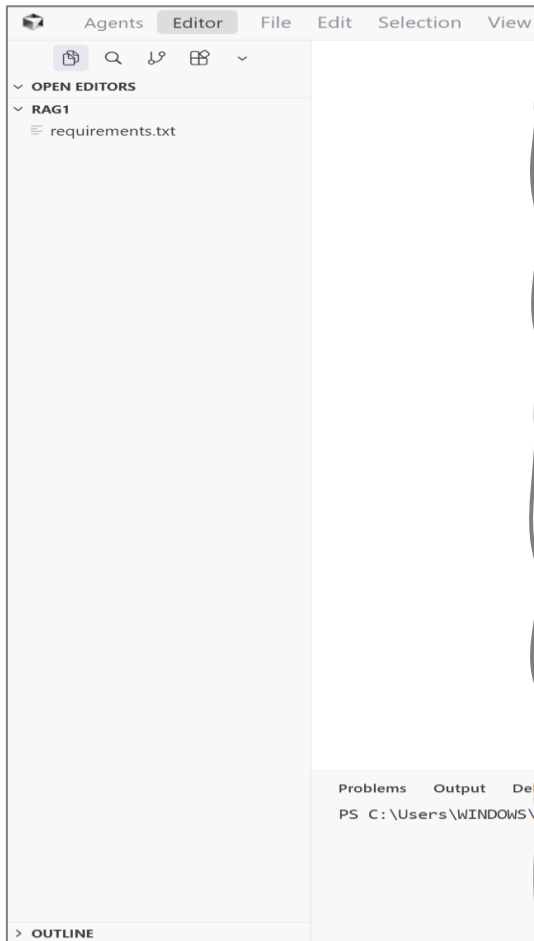
- 파이썬 langchain 패키지를 이용해서 rag를 만들어줘
- flask 패키지를 이용해 웹페이지를 만들어줘
- user interface는 chatgpt와 유사하게 만들어줘
- 버튼, 창 스타일은 연한 청색, 회색 컬러로 산뜻하게 만들어줘
- 왼쪽 메뉴창 상단에 파일 업로드 버튼표시 (여러개 파일 업로드)
- pdf 파일만 업로드 할거야
- llm은 gpt-4o-mini 모델을 사용해야 해
- api-key는 나의 시스템 환경에 있으니 설정 안 해도 돼
- 이 파일을 PyPDFLoader 로 로딩해줘
- RecursiveCharacterTextSplitter로 500글자 분할, 100글자 오버랩
- text-embedding-3-small로 임베딩하고 chromadb에 저장
- 사용자가 질문을 할 때 유사한 chunk를 4개까지 검색해줘
- 답변 출력 후, 참고한 chunk 내용 및 페이지번호를 요약하여 출력해줘
- chunk가 검색되지 않으면 "정보가 없어서 답변할 수 없습니다" 라고 대답해야 해
- 대화 메모리 기능을 사용해줘



실습 1 : General Rag 구현

▶ 개발 환경 설정

⑤ -2 대화창에 프롬프트 입력



Prompt(1/2)

RAG 기반 Flask 웹앱 개발 프롬프트

1. 목표

- 이 프로젝트는 PDF 문서를 업로드하고, 문서 내용을 기반으로 질의응답이 가능한 ****RAG 기반 웹 챗봇****을 만드는 것이다.
- 사용자 시나리오: ① 왼쪽 메뉴창에서 PDF 파일 업로드(다중 업로드 가능) → ② 채팅창에서 질문 입력 → ③ 답변 + 참고한 chunk 내용과 페이지 번호 확인
- UI/UX는 ChatGPT와 유사한 형태(왼쪽 사이드바 + 오른쪽 채팅 영역)로 구성한다.

2. 기술 스펙

환경 및 프레임워크

- Python 패키지: LangChain 사용
- 로컬 환경에 설치된 패키지 및 버전은 현재 폴더의 'requirements.txt'를 참고할 것 (이미 설치된 패키지는 업데이트하지 말 것)
- 웹 프레임워크: Flask
- LLM: 'gpt-4o-mini' (API Key는 시스템 환경변수에 설정되어 있으므로 코드 내 별도 설정 불필요)

RAG 파이프라인

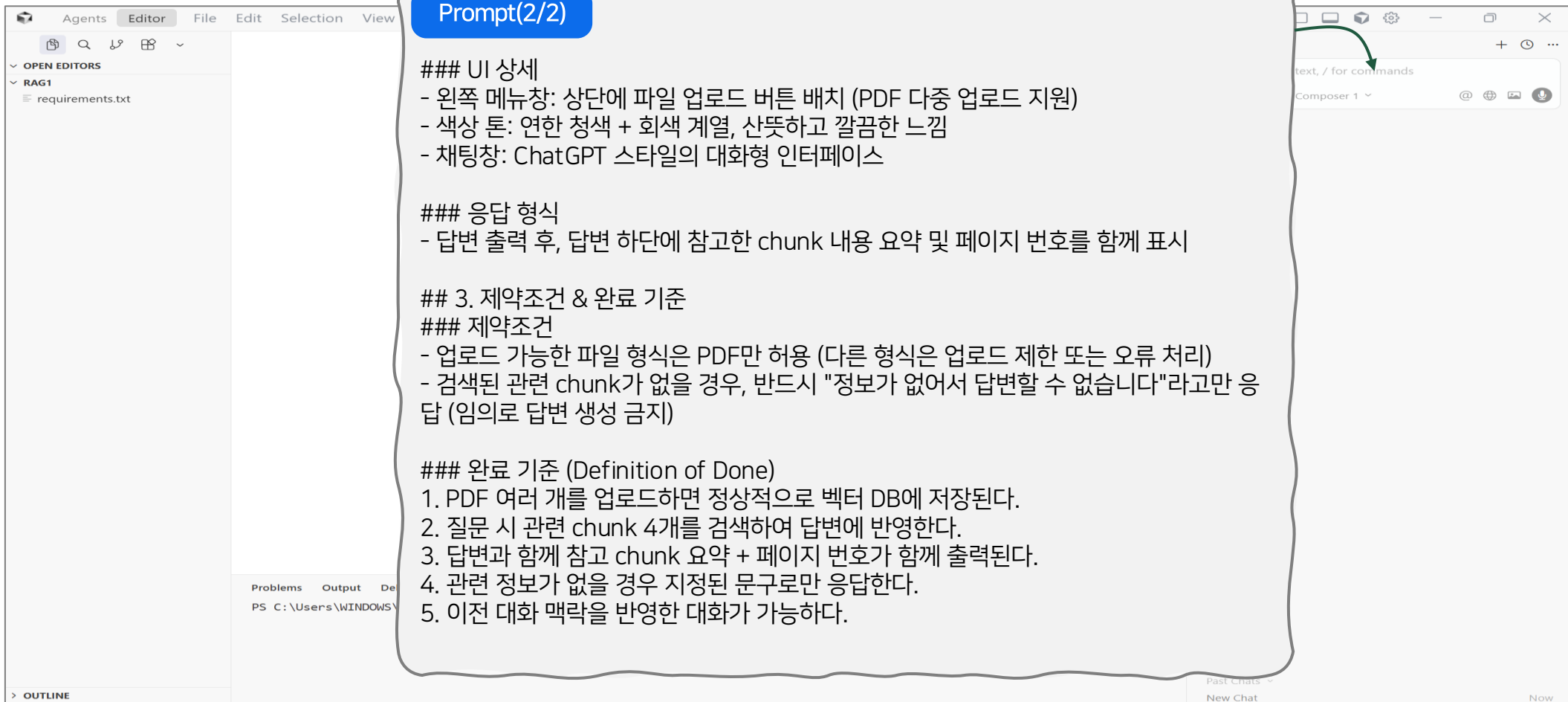
- 문서 로딩: 'PyPDFLoader'
- 청킹: 'RecursiveCharacterTextSplitter' (chunk size 500자, overlap 100자)
- 임베딩: 'text-embedding-3-small'
- 벡터 DB: ChromaDB
- 검색: 질문 시 유사도 기준 상위 4개 chunk 검색
- 대화 메모리 기능 포함 (이전 대화 맥락 유지)



실습 1 : General Rag 구현

▶ 개발 환경 설정

⑤ -2 대화창에 프롬프트 입력



The screenshot shows a VS Code editor window with a file named 'requirements.txt' open. A blue prompt box titled 'Prompt(2/2)' is overlaid on the editor, containing the following text:

UI 상세

- 왼쪽 메뉴창: 상단에 파일 업로드 버튼 배치 (PDF 다중 업로드 지원)
- 색상 톤: 연한 청색 + 회색 계열, 산뜻하고 깔끔한 느낌
- 채팅창: ChatGPT 스타일의 대화형 인터페이스

응답 형식

- 답변 출력 후, 답변 하단에 참고한 chunk 내용 요약 및 페이지 번호를 함께 표시

3. 제약조건 & 완료 기준

제약조건

- 업로드 가능한 파일 형식은 PDF만 허용 (다른 형식은 업로드 제한 또는 오류 처리)
- 검색된 관련 chunk가 없을 경우, 반드시 "정보가 없어서 답변할 수 없습니다"라고만 응답 (임의로 답변 생성 금지)

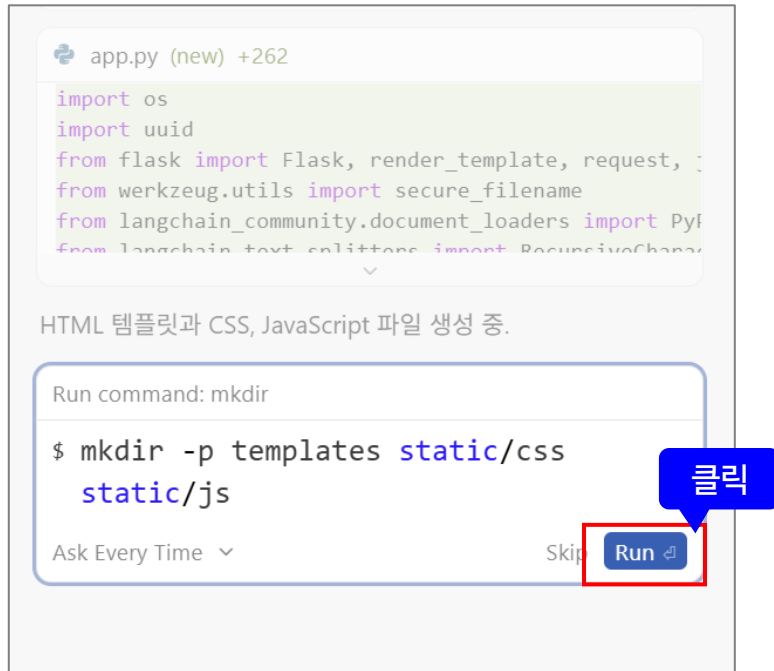
완료 기준 (Definition of Done)

1. PDF 여러 개를 업로드하면 정상적으로 벡터 DB에 저장된다.
2. 질문 시 관련 chunk 4개를 검색하여 답변에 반영한다.
3. 답변과 함께 참고 chunk 요약 + 페이지 번호가 함께 출력된다.
4. 관련 정보가 없을 경우 지정된 문구로만 응답한다.
5. 이전 대화 맥락을 반영한 대화가 가능하다.

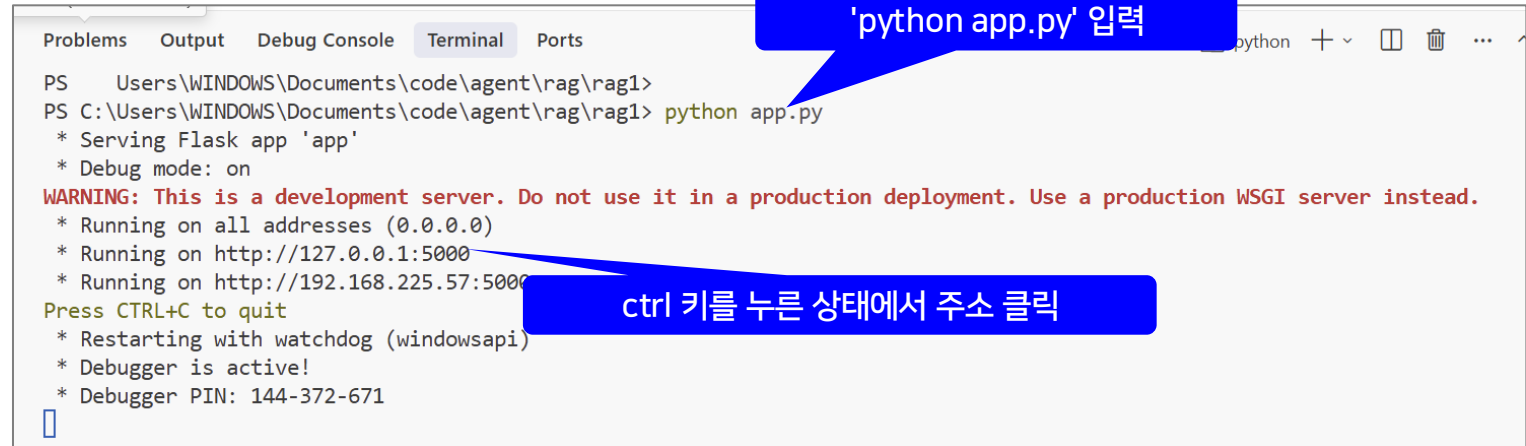
실습 1 : General Rag 구현

▶ 개발 환경 설정

⑥ 폴더생성, 패키지 설치 등 승인



⑦ 코딩이 완료되면 터미널 창에서 웹 실행



실습 1 : General Rag 구현

▶ 개발 환경 설정

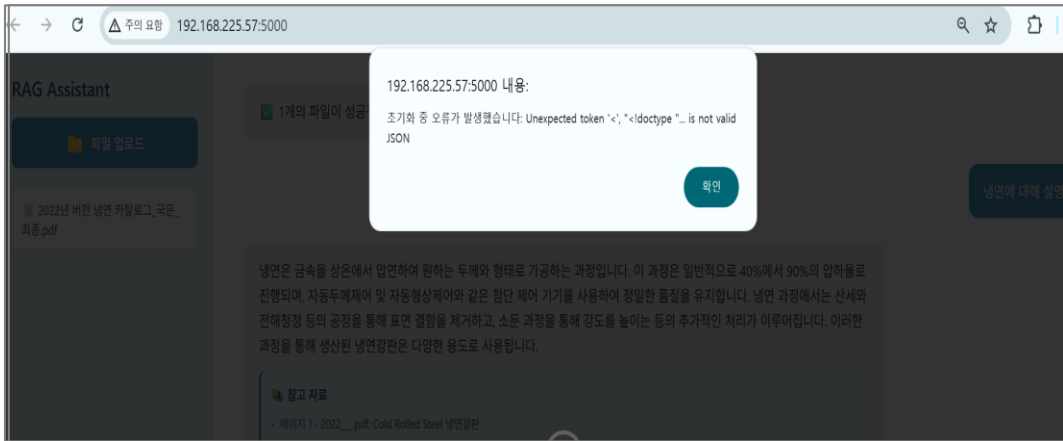
⑧ 웹 동작 상태 확인, 에러 발생시 수정

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5000'. The page title is 'RAG Chat Assistant'. The main content area is titled 'RAG Assistant' and features a 'PDF 파일 업로드' (PDF File Upload) button. A blue callout bubble points to this button with the text: '2022년 버전 선재 카탈로그_국문_최종.pdf' 업로드. Below the upload button, the filename '2022년 버전 선재 카탈로그_국문_최종.pdf' is listed. To the right of the main content area, there is a button labeled '피아노선재에 대해 알려줘' (Tell me about Piano Wire). The chat area displays a response from the assistant: '피아노선재는 고강도 Bead Wire, PC강선, 교량케이블 등에 주로 사용되는 고탄소 청정강입니다. 이 선재는 극세선 신선성, 고강도, 내피로성이 우수한 특성을 가지고 있습니다. 주요 규격으로는 POSMICRO62, POSCABLE82, POSCABLE86, POSCABLE90, POSCABLE92, POSCABLE98 등이 있으며, JIS 규격으로는 SWRS62A/B부터 SWRS92A 까지 다양합니다. 피아노선재는 주로 스프링강과 같은 고탄소강 제품으로, 높은 강도와 신선 가공성을 필요로 하는 용도에 적합합니다.' Below the response, there is a section titled '참고 자료:' (Reference Materials) with three entries: '페이지 13: Micro Cable용 피아노 선재', '페이지 10: 16 POSCO Wire Rod', and '페이지 6: 8 POSCO Wire Rod'. The bottom of the interface shows a text input field with the placeholder '메시지를 입력하세요' (Enter message) and a '전송' (Send) button.

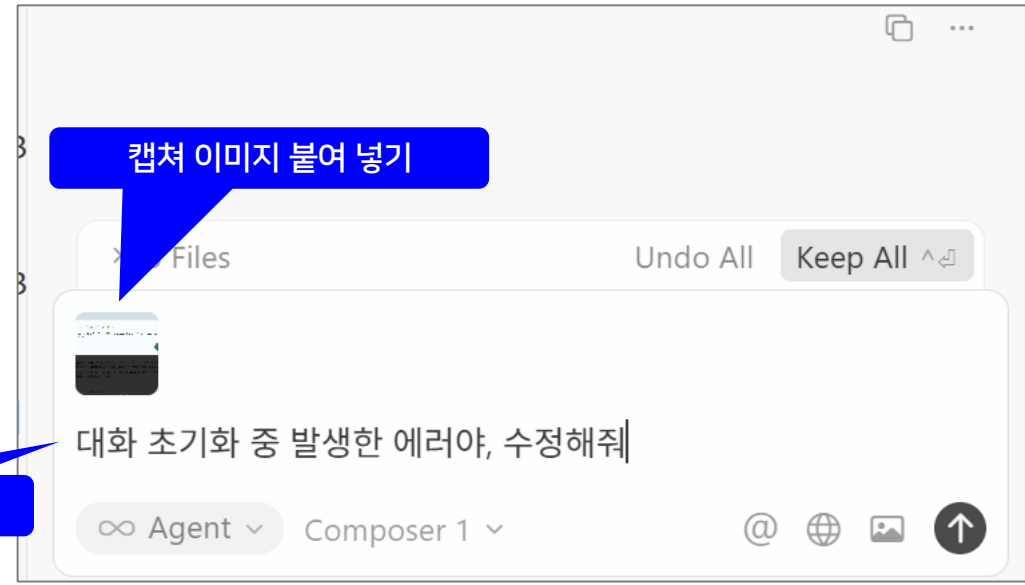
실습 1 : General Rag 구현

▶ 개발 환경 설정

⑨ 오류 수정: 오류가 발생한 화면을 캡처



→ cursor AI 대화창에 캡처 이미지 붙여 넣고 오류 상황 입력



수정 프롬프트

※ 코드 수정 완료 후 웹 페이지 새로고침하여 수정 여부 확인

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ Local LLM, 임베딩 모델



- 모델명 : [BAAI/bge-m3](#) (임베딩)
- 크기 : 1GB 이내
- 문서 벡터화, 검색, RAG Retriever
- 다국어, 한국어 지원 우수
- CPU/GPU 임베딩 사용
- 빠르고 정확한 검색 품질, 가벼움



- 모델명 : [EXAONE-3.5-2.4B-Instruct-GGUF](#) (LLM)
- 크기 : 1.8GB 이내
- 대화형 Instruct LLM, 오프라인 대화
- 한국어 강점, 한국어, 영어 이중언어 최적화
- GGUF 포맷, CPU 에서도 로컬 실행 가능
- 소형 모델이라 복잡한 추론에 한계

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ Local LLM, 임베딩 모델

종류	특징
EXAONE-3.5	LG AI Research의 EXAONE 3.5 모델 시리즈
2.4B	24억(2.4 billion) 개의 파라미터를 가진 모델
Instruct	지시문(Instruction)을 따르도록 추가 학습된 버전. 일반적인 채팅이나 질의응답에 적합
Q5_K_M	5비트 양자화(Quantization) 방식의 모델
.gguf	GGUF(GPT-Generated Unified Format) 파일 형식. llama.cpp, Ollama, LM Studio 등에서 사용하는 모델 포맷

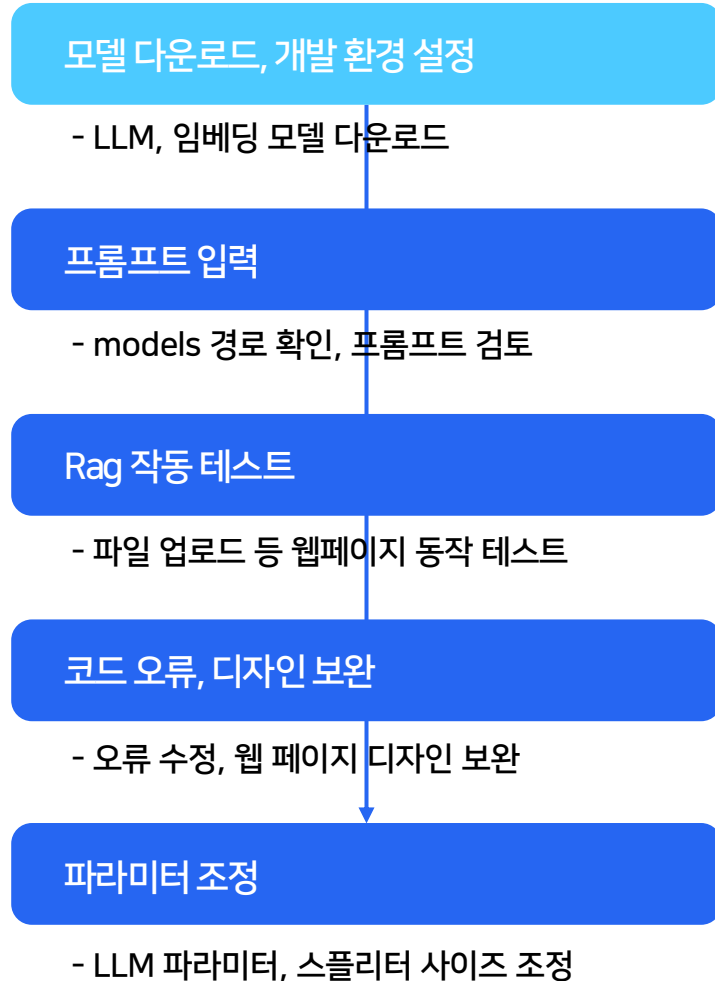
실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ 모델 다운로드 및 개발 환경 설정

- ① **폴더 확인** : 배포된 실습 폴더에 models 폴더 있음 ex) Rag 폴더 : `.WM3_08-2_local_rag`, 모델 저장 폴더 : `.WM3_08-2_local_rag\models`
- ② **터미널 실행** : 탐색기로 `.WM3_08-2_local_rag\models` 폴더로 이동 후, 주소창에 'cmd' 입력
- ③ **허깅페이스 패키지 설치** : 터미널에 `'pip install huggingface_hub'` 입력
- ④ **임베딩 모델 다운로드** : 터미널에 명령어 입력
 - `.WM3_08-2_local_rag\models>위치에서 hf download BAAI/bge-m3 --local-dir bge-m3`
- ⑤ **llm 다운로드** : 터미널에 명령어 입력
 - `.WM3_08-2_local_rag\models>위치에서 hf download LGAI-EXAONE/EXAONE-3.5-2.4B-Instruct-GGUF --include "*Q5_K_M.gguf" --local-dir exaone_2.4b`
- ⑥ **requirements 파일 생성** : `.WM3_08-2_local_rag` 폴더에서 명령어 입력
 - `.WM3_08-2_local_rag>위치에서 pip list >> requirements.txt`
- ⑦ **Cursor AI 실행** : file → Open folder → "`.WM3_08-2_local_rag`" 폴더 클릭

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ Local rag 구현



Prompt 초안

파이썬 langchain 패키지를 이용해서 rag를 만들어줘

- flask 패키지를 이용해 웹페이지를 만들어줘
- user interface는 chatgpt와 유사하게 만들어줘
- 버튼, 창 스타일은 연한 청색, 회색 컬러로 산뜻하게 만들어줘
- 왼쪽 메뉴창 상단에 파일 업로드 버튼표시 (여러개 파일 업로드)
- pdf 파일만 업로드 할꺼야
- llama-cpp-python 사용해줘 (llm 과 임베딩 모델 모두 local 에 저장되어 있음)

- llm은 `"/models/exaone_2.4b/EXAONE-3.5-2.4B-Instruct-Q5_K_M.gguf"` 이용해
- 임베딩 모델은 `"/models/bge-m3"` 을 이용해서 500글자 분할, 100글자 오버랩, chromadb에 저장
- 사용자가 질문을 할 때 유사한 chunk를 4개까지 검색해줘
- chunk가 검색되지 않으면 "정보가 없어서 답변할 수 없습니다" 라고 대답해야 해
- 대화 메모리 기능을 사용해줘
- 답변을 출력한 후, 참고한 chunk 정보도 같이 출력해줘

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

Prompt 1/2

RAG 기반 Flask 웹앱 개발 프롬프트

1. 목표 & 컨텍스트

- 이 프로젝트는 PDF 문서를 업로드하고, 문서 내용을 기반으로 질의응답이 가능한 ****RAG 기반 웹 챗봇****을 만드는 것이다.
- 사용자 시나리오: ① 왼쪽 메뉴창에서 PDF 파일 업로드(다중 업로드 가능) → ② 채팅창에서 질문 입력 → ③ 답변 + 참고한 chunk 내용과 페이지 번호 확인
- UI/UX는 ChatGPT와 유사한 형태(왼쪽 사이드바 + 오른쪽 채팅 영역)로 구성한다.

2. 기술 스펙

환경 및 프레임워크

- Python 패키지: LangChain 사용
- 로컬 환경에 설치된 패키지 및 버전은 현재 폴더의 'requirements.txt' 파일에 있으며, 반드시 여기에 있는 패키지 및 버전만 사용할 것. 이미 설치된 패키지는 업데이트하지 말 것.
- 웹 프레임워크: Flask
- LLM 및 임베딩 실행 방식: 'llama-cpp-python' 사용 (LLM, 임베딩 모델 모두 로컬 파일로 보유)
- LLM: 로컬 GGUF 모델 사용 — 경로: './models/exaone_2.4b/EXAONE-3.5-2.4B-Instruct-Q5_K_M.gguf'

RAG 파이프라인

- 문서 로딩: 'PyPDFLoader'
- 청킹: 'RecursiveCharacterTextSplitter' (chunk size 500자, overlap 100자)
 - 분할 기준(separators)은 "문단 → 문장" 순으로 우선 적용
 - 예: 'separators=["\n\n", "\n", ".", "?", "!", ":", ";"]'
 - 문단 구분(빈 줄, 줄바꿈)을 최우선으로 분할하고, 문단 내에서는 문장 단위(마침표, 물음표, 느낌표 뒤)로 분할되도록 구성
- 임베딩: 로컬 임베딩 모델 사용 — 경로: './models/bge-m3' ('llama-cpp-python'으로 로딩)
- 벡터 DB: ChromaDB
- 검색: 질문 시 유사도 기준 상위 4개 chunk 검색
- 대화 메모리 기능 포함 (이전 대화 맥락 유지)

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

Prompt 2/2

UI 상세

- 왼쪽 메뉴창: 상단에 파일 업로드 버튼 배치 (PDF 다중 업로드 지원)
- 색상 톤: 연한 청색 + 회색 계열, 산뜻하고 깔끔한 느낌
- 채팅창: ChatGPT 스타일의 대화형 인터페이스

응답 형식

- 답변 출력 후, 답변 하단에 참고한 chunk 내용 요약 및 페이지 번호를 함께 표시

3. 제약조건 & 완료 기준

제약조건

- 업로드 가능한 파일 형식은 PDF만 허용 (다른 형식은 업로드 제한 또는 오류 처리)
- 검색된 관련 chunk가 없을 경우, 반드시 "정보가 없어서 답변할 수 없습니다"라고만 응답 (임의로 답변 생성 금지)

완료 기준

1. PDF 여러 개를 업로드하면 정상적으로 벡터 DB에 저장된다.
2. 질문 시 관련 chunk 4개를 검색하여 답변에 반영한다.
3. 답변과 함께 참고 chunk 요약 + 페이지 번호가 함께 출력된다.
4. 관련 정보가 없을 경우 지정된 문구로만 응답한다.
5. 이전 대화 맥락을 반영한 대화가 가능하다.

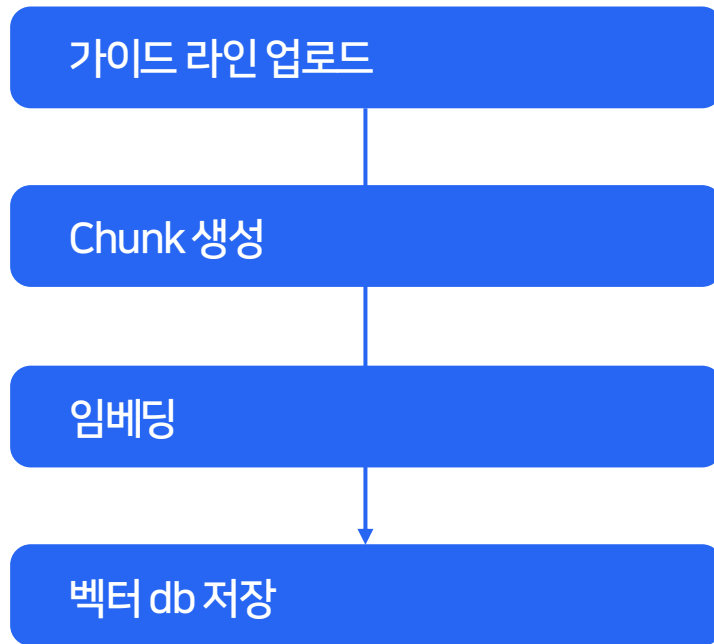
- GPT와 유사한 웹으로 구성, 계약서 약관/ 계약서 PDF 업로드, 약관은 text-embedding-3-small로 임베딩 , llm gpt-4o-mini 사용
- 계약서 문장을 한 줄씩 읽은 후 Chunk 검색 및 비교, 오류가 있다면 원문과 수정 문구 출력, 오류 없을 시 원문만 출력

166

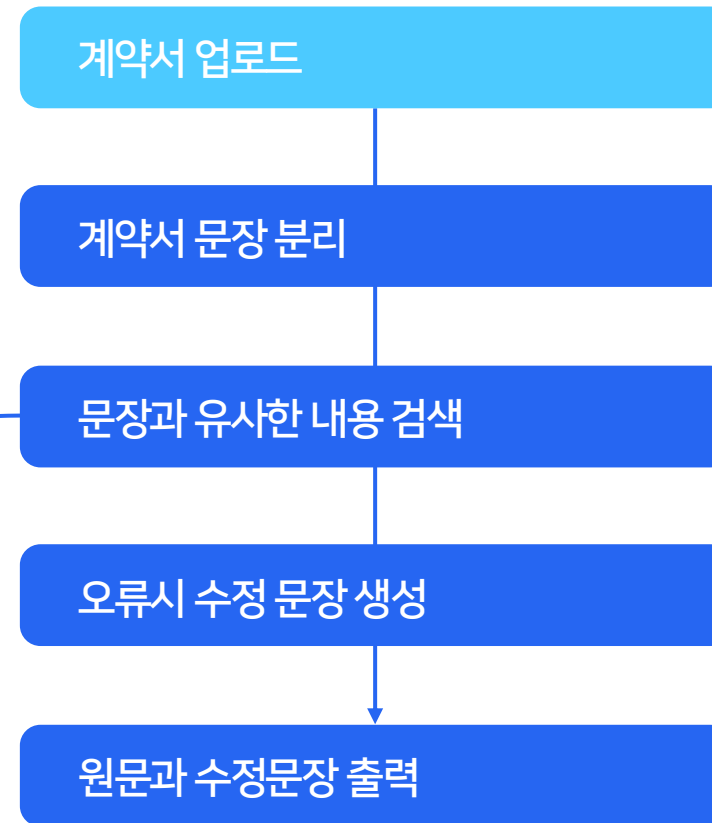
실습 3 : 계약서 검토 Rag 구현

▶ 흐름도

- 벡터db 흐름도



- 계약서 검토 흐름도



실습 3 : 계약서 검토 Rag 구현

▶ 프롬프트

Prompt

내가 만들려고 하는 웹 시스템은 아래와 같아

0. 제목 : LLM 기반 계약서 검토 웹

1. 개요

- 파이썬 langchain 0.3X 버전 패키지를 이용해서 LLM 기반 계약서 검토 웹 코드를 생성해줘
- 계약서 가이드라인이나, 약관을 업로드해서 rag를 구현하고, 사용자가 계약서를 업로드하면 한 문장씩 읽고, rag에서 유사 문서를 검색 후, 가이드라인이나 약관에 위배사항이 있는지 검토하고 계약서를 수정해서 출력하는 웹서비스를 구현하는 거야

2. 패키지 버전 및 환경

- flask 패키지를 이용해, llm은 gpt-4o-mini 모델을 사용해
- langchain 등 패키지 버전은 내 시스템에 설치된 'requirements.txt' 파일을 참조해줘, 이미 설치된 패키지는 설치된 버전 그대로 쓰고 업데이트하지 마
- api-key는 나의 시스템 환경에 있으니 설정 안 해도 돼
- 시스템 구현을 위해 추가적으로 필요한 패키지 버전은 알려줘
- 코드들은 이해하기 쉽게 주석을 추가해줘

3. 메뉴 구성

- user interface는 chatgpt와 유사하게 만들어줘 (왼쪽 : 메뉴, 오른쪽 : 채팅 및 결과)
- 화면 구성은 연한 파랑색과, 연한 회색을 이용해 줘
- 왼쪽 메뉴는 'rag 파일 업로드' 버튼 과 '계약서 업로드' 버튼이 있어야 해
- '계약서 업로드'가 완료되면 '계약서 검토' 버튼이 보여야 해
- 오른쪽 화면은 답변을 보여주고, 하단에는 일반적인 질문을 할 수 있는 질문창을 보이게 해줘
- rag 파일 업로드, 계약서 업로드시 답변 창에 진행율을 보여주고 완료 결과를 출력해줘 (tqdm 패키지 활용, 바 출력)

4. rag 구현

- 'rag 파일 업로드' 버튼을 누르면 pdf 파일을 업로드 할 수 있도록 구성해줘
- PyPDFLoader로 여러 개의 파일을 업로드 할 수 있어야 해
- 업로드한 문서를 RecursiveCharacterTextSplitter로 30글자로 분할, 5글자로 오버랩 해줘, separators는 ["\n", "\n\n"]
- text-embedding-3-small 로 임베딩하고 chromadb에 저장해

실습 3 : 계약서 검토 Rag 구현

▶ 프롬프트(계속)

5 계약서 검토 구현

- '계약서 업로드' 버튼을 누르면 PyPDFLoader로 pdf 파일을 업로드 할 수 있게 해줘
- 계약서를 RecursiveCharacterTextSplitter로 30글자로 분할, 오버랩 하지만, separators는 ["\n", "\n\n"]
- 계약서 문장과 rag 문서를 비교하여 계약서 문장에 오류 또는 보완이 필요한 부분을 찾은 후 수정해줘
- 계약서 문장을 '[원문]' 이라고 표시 후 출력하고 오류를 수정한 문장을 '[수정 문구]' 라고 표시한 후 진한 파란 글씨체로 원문의 바로 밑에 출력해줘 (7. 출력 예시 참조)
- 계약서 문장에 이상이 없다면 원문만 출력하면 됨
- 화면에 문장 하나씩 하나씩 출력해줘, 한꺼번에 출력되면 검토하는 동안 진행 여부를 확인할 수 없어

6. 기타

- 사용자가 일반적인 질문을 하게 되면 rag를 이용하지 말고 llm에 직접 요청해서 답변을 생성하면 되
- 만약 성능 향상을 위해 에이전트가 필요하다면 langgraph 패키지를 이용해서 에이전트를 구현해줘

7. 출력 예시

[원문] 3. 계약체결 및 작업완료 후라도 본 계약금액의 산출내역중 적용물량, 적용단가, 제비용 등의 적용이 과다 또는 과소하게 책정된 것이 발견된 때에는 도급인과 수급인은 협의하여 이를 감액 또는 증액 조정할 수 없다.

[수정문구] 3. 계약체결 및 작업완료 후라도 본 계약금액의 산출내역중 적용물량, 적용단가, 제비용 등의 적용이 과다 또는 과소하게 책정된 것이 발견된 때에는 도급인과 수급인은 협의하여 이를 감액 또는 증액 조정할 수 있다.

실습 3 : 계약서 검토 Rag 구현

▶ 실습절차

① 실습 폴더 확인

- 실습 폴더 아래 M3_08-3_contract_review 폴더 열기 (배포됨)

② New Chat 클릭

- 사이드바에서 New Chat 클릭

Chunk 를 어떻게 나눌 것인가?

계약서 문장 분리를 어떻게 할 것인가? 에 따라 검토 품질이 달라짐

③ 프롬프트 입력

- 대화창에 프롬프트 입력, 진행 여부 등에 대해 답변

④ 웹 테스트

- 실습파일 업로드 후 웹 테스트, 이상 여부 확인
※ rag 문서 : 시설공사계약약관, 협력계약일반약관, 검토 계약서 : 계약서_오류정정실습)

⑤ 에러 보완

- 에러 수정 프롬프트 생성, 코드 생성 후 적용, 재 실행

M4. 종합 실습

9. 막혔을 때 빠져나오는 법

초보가 막히는 4가지 패턴

▶ 해결하지 못해서가 아니라, 전달하지 못해서 막힙니다



패턴 ① 화면만 본다

빨간 글씨를 눈으로만 보고
"에러 났어요"라고 말합니다.
AI는 그 화면을 못 봅니다.



패턴 ② "안 돼요"만 말한다

무엇을 하려다 어디서 안 됐는지
없이 결과만 전달합니다.
AI는 추측으로 헛수고합니다.



패턴 ③ 같은 말만 반복

"다시 해줘"를 10번 누릅니다.
코드는 점점 복잡해지고
원래 되던 것까지 망가집니다.



패턴 ④ 돌아갈 곳이 없다

잘 되던 시점의 복사본이 없어
처음부터 다시 만듭니다.
2시간이 날아갑니다.

바이브코딩에서 잘하는 사람과 못하는 사람의 차이는 코딩 지식이 아니라

① 상황을 정확히 전달하는 능력 ② 언제든지 되돌아갈 수 있는 습관 — 이 둘입니다.

오늘의 약속

막히면 혼자 20분 헤매지 말고 — ① 멈추고 ② 복사하고 ③ 설명한다. 이 3단계를 몸에 붙이는 것이 이 시간의 전부입니다.

에러 대응 3단계 — STOP · COPY · TELL

▶ 막힌 순간 30초 안에 할 일

STOP 멈춘다

- 더 이상 새 요청을 넣지 않는다
- 화가 난 상태로 "다시!"를 누르지 않는다
- 지금 화면을 그대로 둔다

COPY 통째로 복사한다

- 터미널의 빨간 글씨 전문(全文)
- 텍스트가 안되면 스크린샷으로
- 마지막으로 실행한 명령어도 함께

TELL 상황을 설명한다

- 아래 5요소 템플릿을 채운다
- 추측("아마 DB 문제인 듯")은 쓰지 않는다
- 사실만 적는다

상황 설명 5요소 템플릿

- ① 목표 — 무엇을 만들려고 했나
- ② 방금 한 일 — 어떤 요청/명령을 넣었나
- ③ 기대 — 무엇이 나오길 바랐나
- ④ 실제 — 무엇이 나왔나 (화면 문구 그대로)
- ⑤ 에러 전문 — 빨간 글씨 전체 붙여넣기

※ ⑤만 있어도 절반은 해결됩니다. ①~④는 해결 속도를 높입니다.

❌ 이렇게 말하면 AI도 헤맬니다

"안 돼요. 고쳐주세요"

"아까는 됐는데 지금은 이상해요"

✅ 이렇게 말하면 한 번에 잡힙니다

"PDF 번역 웹페이지를 만들고 있어. 방금 '파일 업로드 기능 추가해줘'라고 요청하고 실행했어. 업로드 버튼이 보이길 기대했는데 화면이 하얗게 비어. 터미널 에러 전문은 아래와 같아: (붙여넣기)"

【실습】고장난 앱 고치기

- ▶ 강사가 배포한 프로젝트 폴더에는 버그 3개가 심어져 있습니다. 코드는 직접 고치지 않습니다
— 상황을 전달하고 Cursor에게 시키세요.

버그 ①

실행하자마자 멈춤

증상 : ModuleNotFoundError
No module named ...

할 일 : 에러 전문을 그대로 붙이고
"무엇이 문제이고 어떻게 고치는지"
물어본 뒤 Cursor에게 시키기

버그 ②

화면은 뜨는데 답이 안 옴

증상 : 401 Unauthorized 또는
Incorrect API key

할 일 : 401이 무슨 뜻인지 묻고,
노트북 환경변수의 키를 쓰도록
Cursor에게 고치게 하기

버그 ③

수정했는데 화면이 그대로

증상 : 변경이 반영되지 않음
(에러 메시지가 전혀 없음)

할 일 : "방금 무엇을 바꿨고 무엇이
그대로인지" 5요소로 설명하고
확인 방법 물어보기

진행 방법

- ① 개인 10분 — 혼자 힘으로 3개 모두 시도
- ② 짝 5분 — 못 잡은 것을 옆 사람에게 설명하고 협업 해결
- ③ 전체 5분 — 팀별로 "어떻게 물었더니 풀렸는지" 공유

실습 중 금지사항

- 소스 파일을 열어 직접 고치기 (오늘은 Cursor를 시키는 연습)
- API 키·사번·개인정보를 채팅창에 붙여넣기
- 같은 요청 3회 이상 반복 (다음 장 참고)

목표는 "3개를 다 고치는 것"이 아니라 "막혔을 때 물어보는 법이 몸에 붙는 것"입니다.

같은 자리에서 3번이면 전략을 바꿔라

▶ 반복 요청은 문제를 푸는 게 아니라 코드를 망가뜨립니다

- 1 새 채팅으로 옮긴다**
대화가 길어지면 AI가 앞의 실패한 시도들에 끌려다닙니다.
새 채팅에 목표와 현재 코드만 다시 주세요.
- 2 문제를 작게 만든다**
전체 앱 말고 안 되는 기능 하나만 떼어 최소 예제로.
"이 부분만 따로 동작하는 코드를 만들어줘"
- 3 되돌린다**
마지막으로 정상 동작하던 시점으로 복귀.
Cursor의 되돌리기 또는 복사해 둔 폴더 사용.
- 4 다른 방식을 요청한다**
"같은 방법으로 고치지 말고, 아예 다른 방식으로 다시 설계해줘"라고 명시적으로 요청.

⊖ 하지 말아야 할 것

"다시 해줘 / 아직도 안 돼 / 제대로 좀 해줘"를 반복하면 AI는 기존 코드 위에 땀질을 쌓습니다.

🕒 3분 룰

같은 에러를 3번 봤거나 3분이 지났으면, 위 4수 중 하나로 반드시 전환합니다.

되돌릴 수 있게 만들어 두세요 — 잘 동작하는 순간마다 폴더를 통째로 복사(v1_작동, v2_업로드추가).
5분 전으로 돌아갈 수 있으면, 어떤 실수도 5분짜리 실수가 됩니다.

먼저 쓰면 덜 막힌다 — 요구사항 5줄

▶ 막힘의 절반은 "무엇을 만들지"가 흐릿해서 생깁니다

요구사항 5줄 템플릿

- ① 누가 쓰나 — 사용자는 누구인가
- ② 무엇을 하나 — 한 문장으로
- ③ 입력 — 무엇을 넣으면
- ④ 출력 — 무엇이 나오나
- ⑤ 안 할 것 — 이번엔 만들지 않는 기능

⑤가 가장 중요합니다. 범위를 막지 않으면 AI가 기능을 계속 늘려 감당 못 할 코드를 만듭니다.

작성 예시 — 회의록 정리 도구

- ① 우리 팀 회의 주관자
- ② 회의록 텍스트에서 할 일만 뽑아 표로 정리한다
- ③ 회의록 txt 파일 1개
- ④ 할 일 / 담당자 / 기한 3열 표, todo.md로 저장
- ⑤ 이번엔 메일 발송·캘린더 연동은 만들지 않는다

→ 이 5줄을 Cursor 첫 메시지로 그대로 붙여넣습니다.

막혔을 때 순서 체크리스트 (인쇄해서 모니터 옆에 두기)

- ① 멈춘다 — 새 요청을 넣지 않는다 ② 에러 전문을 텍스트로 복사한다
- ③ 5요소로 설명한다(목표·한 일·기대·실제·에러) ④ 답을 받으면 한 번에 하나씩만 적용한다
- ⑤ 되면 즉시 폴더 복사본을 만든다 ⑥ 3번/3분 넘으면 새 채팅·문제 축소·되돌리기·다른 방식
- ⑦ 그래도 안 되면 옆 사람에게 5요소로 설명한다 (설명하다가 스스로 찾는 경우가 많습니다)

M4. 종합실습

10. AI Agent 맛보기

챗봇과 에이전트는 무엇이 다른가

▶ 레시피를 읽어주는 AI vs 주방에 들어가 요리하는 AI

챗봇 (Chatbot)

"김치찌개 만드는 법 알려줘"

→ 레시피를 알려준다. 끝.

요리는 여전히 내가 한다. 재료도 내가 꺼내고,
불도 내가 켜고, 맛도 내가 본다.

에이전트 (Agent)

"김치찌개 끓여줘"

→ 냉장고를 열고, 재료를 꺼내고, 불을 켜고,
중간에 맛을 보고, 싱거우면 소금을 더 넣고,
다 되면 "완성됐습니다"라고 보고한다.

	챗봇	에이전트
하는 일	묻는 말에 답한다	목표를 받아 스스로 단계를 밟는다
결과물	말(텍스트)	실제로 바뀐 파일·시스템·결과물
도구 사용	없음	파일 읽기/쓰기, 실행, 검색 등
사람 개입	매 턴마다 필요	중요한 순간에만 확인·승인

한 줄 정의 — 에이전트는 "목표를 주면, 될 때까지 도구를 써가며 스스로 반복하는 AI"입니다.
핵심 차이는 똑똑함이 아니라 「다음에 무엇을 할지 스스로 정하는가」입니다.

에이전트의 4단계

▶ 한 바퀴가 아니라, 될 때까지 돕니다



① 계획

지금 무엇을 먼저 할지 정한다
결과를 보고 다음 수를 바꾼다



② 도구 사용

필요한 도구를 골라 쓴다
안 써도 되면 건너뛴다



③ 실행

실제로 파일을 읽고 쓰고
명령을 돌려 결과를 받는다



④ 검수

결과가 기준에 맞는지 스스로 본다
아니면 ①로 돌아간다

 여러분은 이미 에이전트를 썼습니다 — Cursor에게 "로그인 기능 추가해줘"라고 했을 때

①계획 무엇부터 할지 정하고 → ②도구 기존 파일을 열어 읽고 → ③실행 코드를 고쳐 쓰고 돌려보고
→ ④검수 에러가 났는지 확인하고 → ①계획 방금 실패한 방법을 기억한 채 다시 정하고 → 완료 보고
챗봇이었다면 ①에서 코드만 알려주고 끝났을 것입니다. 나머지는 전부 여러분 몫이었겠죠.

 4단계가 보인다고 다 에이전트는 아닙니다

화면에 [계획] [도구] [실행] [검수]를 찍는 것은 고정된 코드로도 됩니다. 갈리는 지점은 하나입니다.

다음에 무엇을 할지를 「파이썬 코드」가 정하면 자동화 스크립트, 「AI」가 매 단계 정하면 에이전트입니다.

도구(Tool)와 MCP

▶ AI에게 「손」을 달아주는 표준 규격

MCP란 무엇인가

AI가 외부 도구·데이터에 접근하는 방식을 통일한 규격입니다.

- 비유 — USB-C. 기기마다 다른 충전기를 쓰던 것을 하나로 통일
- 덕분에 새 도구를 붙일 때마다 새로 개발할 필요가 없음
- Cursor·Claude 등에서 설정만으로 도구를 추가할 수 있음

붙일 수 있는 도구의 예

- 내 PC의 파일과 폴더 (읽기·쓰기·정리)
- 사내 문서 저장소 · 위키 · 메일
- 데이터베이스 조회
- 웹 검색 · 사내 시스템 API
- 그리고 7번에서 배운 RAG 검색기도 「도구」의 하나입니다

도구를 붙이기 전에 반드시 정할 것 — 권한과 범위

에이전트에게 「손」을 준다는 것은 실제로 파일을 바꾸고 명령을 실행할 수 있게 한다는 뜻입니다.

- 읽기만 허용할지, 쓰기까지 허용할지 · 어느 폴더까지 접근하게 할지 · 삭제·전송은 사람 승인 후에만
- 사내 데이터를 외부로 보내는 도구는 보안 검토를 거친 것만 사용

오늘 실습에서는

별도 MCP 설정 없이, Cursor가 기본으로 가진 「내 프로젝트 폴더의 파일을 읽고 쓰는 도구」만 사용합니다. 그것만으로도 충분한 에이전트가 됩니다.

좋은 요구사항서의 4요소

▶ 에이전트에게는 "무엇을"보다 "어디까지"가 더 중요합니다

1. 역할 — 너는 무엇을 하는 담당인가

"너는 출장비 정산서를 사내 규정에 따라 판정하는 담당이야"

2. 도구·범위 — 무엇을 쓸 수 있는가

"도구 7개를 만들고, 에이전트가 골라 쓰게 해"

3. 규칙 — 지킬 것과 하지 말 것

"규정에 없는 비용은 추측하지 마. 모르면 사람에게 넘겨"

4. 종료조건 — 언제 끝난 것으로 보는가

"건마다 최대 15단계. 못 끝내면 사람에게 넘기고 다음 건으로"

❌ 나쁜 요구사항

"출장비 좀 검토해줘"

- 어떤 기준으로, 어디까지 판단할지, 언제 끝인지 알 수 없습니다.
- 엉뚱한 파일을 건드리거나 끝없이 되묻습니다.

✅ 좋은 요구사항

4요소가 모두 들어간 지시서

- 한 번에 끝나고 결과가 예측 가능합니다.
- 무엇을 확인해야 하는지도 분명합니다.

그리고 에이전트에게만 필요한 다섯 번째 항목 —

「다음에 무엇을 할지는 코드가 아니라 네가 정해라」. 이 한 줄이 없으면 에이전트가 안 됩니다.

【실습】출장비 정산 검토 에이전트 만들기

▶ 계획 → 도구사용 → 실행 → 검수가 스스로 도는 프로그램을 만듭니다

배포 폴더에 코드는 없습니다. 정산서 8건 · 규정집 · 세부지침 · 명부 · 요구사항서뿐 — 프로그램은 여러분이 만듭니다.

만들 때 — 도구 7개를 가진 에이전트

- ① 정산서_요약 정산서의 사실을 정리한다
 - ② 규정_검색 · 임직원_조회 근거 조항·신청자를 찾는다
 - ③ 정산기한_확인 · 환율_조회 · 계산 날짜·금액은 도구로
 - ④ 검토결과_저장 판정을 저장 (검수는 파이썬이)
- 정산서는 한 건씩 넘기고, 건 안의 순서는 AI가 정합니다

정산서에 심어둔 장치 — AI가 판단해야만 풀립니다

- 서울 숙박 125,000원 → 서울·경기 한도 적용 → 승인
- 9/30 제출 → 추석 연휴를 빼고 세면 기한 내
- 규정에 없는 접대비 + 동명이인 → 사람 확인 필요

실행할 때 — 브라우저에 뜨는 것

-  계획 다음에 무엇을 할지 AI가 말한다
 -  도구 7개 중 무엇을 골랐는지 + 왜
 -  실행 도구가 돌아간 결과
 -  검수  통과(초록) /  실패(빨강)
- 마지막에 "8건 검토 완료" 가 나오면 성공입니다

검수에서 볼 것

- 근거 조항이 실제 규정과 맞는가
- 기한 초과 건을 반려하지 않고 비고에 적었는가
- 원본 파일이 그대로인가 / 키가 코드에 없는가

오늘 만든 것은 "AI를 한 번 부르는 프로그램"이 아닙니다.

스스로 계획하고, 도구를 고르고, 자기 결과를 검수해 다시 도는 프로그램입니다.

내 업무에 붙일 곳 찾기

▶ 오늘 배운 내용을 조합하면 대부분의 사무 업무에 닿습니다



회의록 → 실행과제

녹취/메모에서 할 일·담당자·기한 추출



보고서 초안 자동화

정해진 양식에 데이터만 갈아 끼우기



자료 폴더 정리

규칙에 따라 파일명 변경·분류·중복 정리



사내 규정 Q&A 봇

RAG로 규정집을 근거로 답하게 하기



데이터 취합

흩어진 엑셀·CSV를 하나로 모아 요약



메일·공지 초안

상황만 주면 톤에 맞춰 초안 생성



점검 리스트 자동화

체크 항목 생성 → 결과 기록 → 요약



반복 문서 양식화

매주 같은 문서를 템플릿+데이터로 자동 생성



지금 1분 — 내 업무에서 하나만 고르기

내가 매주 반복하는 일 중, 규칙이 분명하고 결과물이 파일인 것: _____

그것을 에이전트에게 맡긴다면 지시서 4요소는? 역할 _____ 범위 _____ 규칙 _____ 종료조건 _____

「반복적이고 · 규칙이 분명하고 · 결과가 파일인」 업무가 에이전트로 가장 좋습니다.